Graphical Design Tutorial

for HDL Author and HDL Designer

Software Version 2005.2

15 September 2005

Copyright © Mentor Graphics Corporation 1996-2005
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OR MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202-3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.

This is an unpublished work of Mentor Graphics Corporation.

Contacting Mentor Graphics Corporation

Telephone:503.685.7000
Toll-Free Telephone:800.592.2210
Website:www.mentor.com
SupportNet: www.mentor.com/supportnet
Documentation Feedback: www.mentor.com/supportnet/documentation/reply_form.cfm

Table of Contents

About This Manual ATM-1

Introduction ATM-1
Other Documentation ATM-2
Copying Text From the Acrobat Viewer ATM-3
Example Designs ATM-3
Typographic Conventions ATM-4

Chapter 1
VHDL Timer Exercise 1-1

Specification 1-1
Set Library Mapping 1-2
Set the Default Language 1-4
Create a Block Diagram 1-6
Edit the Title Block Template 1-7
Add Blocks 1-8
Add Embedded Blocks 1-9
Add Ports and Signals 1-10
Add a Bundle and Global Connector 1-12
Save the Block Diagram 1-13
Edit the Block and Signal Names 1-15
Add an Embedded HDL Text View 1-19
Add a Panel and Edit the Title Block 1-21
Create a Child State Diagram 1-23
Add States and Transitions 1-26
Save the State Diagram 1-27
Edit the States 1-28
Edit the Transitions 1-30
Create a Hierarchical State Diagram 1-32
Complete the Hierarchical State Diagram 1-34
Editing State Machine Properties 1-36
Set Generation Properties 1-39
Set Checks for HDL Generation 1-41
Generate HDL for the State Machine 1-42
HDL Files in the Design Explorer 1-43
Import the BCDCounter Design Unit 1-44
Create a Child Block Diagram 1-46
Edit the Generic Mapping 1-52
Add ModuleWare Components 1-53
Add a User Declaration 1-55
Create a Truth Table 1-57
Edit the Truth Table 1-58

Set Truth Table Properties 1-59
Browse the Timer Design 1-60
Generate HDL for the Hierarchy 1-62
Edit the Timer Symbol 1-64
Create a Test Bench 1-65
Import the Tester Design Unit 1-68
Instantiate the Imported Tester 1-69
Generate HDL for the Test Bench 1-71
Browse the Completed Design 1-73
Setup the Downstream Tools 1-73
Run the ModelSim Flow 1-75
Setup the Simulator Windows 1-77
Add Simulation Probes 1-78
Enable Animation 1-79
Simulate the Design 1-81
Review the Animation 1-83
Debugging From ModelSim 1-84
Run the LeonardoSpectrum Flow 1-86
Using the Example VHDL Design 1-90

Chapter 2
Creating a VHDL Flow Chart 2-1

Introduction 2-1
Create the Tester Flow Chart 2-2
Set Flow Chart Properties 2-3
Add a Start Point and Action Box 2-6
Add a Loop and an Associated Comment 2-8
Add an Action Box 2-10
Add a Hierarchical Action Box 2-11
Add a Decision Box 2-12
Add Wait Boxes 2-14
Copy the Decision Tree 2-16
Completing the Flow Chart 2-17
Using HDL2Graphics 2-19

Trademark Information

End-User License Agreement

About This Manual

Introduction

This manual provides a self-paced tutorial with step-by-step procedures for creating a simple timer design and test bench using VHDL graphical views.

You are advised to perform the separate the *Design Management Tutorial* before attempting this tutorial to learn about creating projects and managing your design data.

You can learn about using tabular Interface-Based Design views by performing the separate *Interface-Based Design Tutorial*.

The tutorial covers the basic procedures required to fully define and verify a **VHDL** design using graphical views. The full tutorial can be completed by users of the HDL Author or HDL Designer tools using block diagram, symbol, state diagram, truth table and flow chart views.

The completed design can be compiled and simulated if Model*Sim* is available and synthesized if the LeonardoSpectrum tools are available.

The main tutorial includes a test bench which uses a flow chart view. The flow chart can be imported from example HDL code or can optionally be created by following the procedures in Chapter 2.

Although the procedures do not describe the use of other tools, the HDL Designer Series includes support for many alternative downstream tool interfaces and it should be possible to use the generated HDL with any of these interfaces.

However, you must consider any limitations of your external tool. In particular, some VHDL tools may not support the standard IEEE packages used in the tutorial and you should substitute an appropriate alternative package.

The tutorial assumes that users have some knowledge of the issues for digital hardware design and experience of the VHDL or Verilog language.

It is possible to complete the tutorial without this knowledge by carefully copying the language syntax given in the procedures. However, a separate VHDL training course is recommended in order to fully appreciate how the power of HDL design can be exploited using graphical design methods.



The illustrated examples have been laid out for maximum readability in the Acrobat viewer or on the printed page. When creating your own design, it is advisable to allow extra space between diagram objects so that you can easily route signals between them.

All user commands in the tutorial procedures are referenced using menu paths (shown in bold text) or toolbar buttons. However, many commands can also be accessed using the shortcut bar or keyboard shortcuts. Refer to Shortcut Keys in the **Quick Reference Index** which can be accessed from the **Help** menu for lists of the available shortcut keys.

The tutorial is divided into sections containing numbered procedures which must be performed for each section of the tutorial to be completed successfully. If you are working from a printed copy of the tutorial, you are recommended to tick off each numbered procedure as you complete it to ensure no steps are omitted.

Unnumbered paragraphs provide additional information or give advice on other ways to perform an operation. The procedure numbers are continuous through each major section and its subsections but restart for each major section, for example when introducing a new editor.

Other Documentation

The HDL Designer Series documentation (including the release notes and this tutorial) can be accessed from the *HDL Designer Series Bookcase* which is available from the **Help** menu in each application window.

The tutorials can be accessed from the *HDL Designer Series Tutorials* bookcase which can be accessed from a link in the main bookcase or directly from the **Help** menu.

All documents are provided in Adobe Acrobat portable document format (PDF). The Acrobat reader can be installed from the HDL Designer Series CD-ROM if it is not already available.

The *Release Notes* provide information about new features, issues and enhancements in the latest release.

The *Start Here Guide* provides general information about the HDL Designer Series tools, supported platforms and configurations.

The *Transition Guide* provides information about updating from a previous release.

The *HDL Designer Series User Manual* provides information about the design manager and procedures common to all HDL Designer Series tools.

The *Graphical Editors User Manual* provides general information about the graphical editors. It also includes procedures for using the block diagram, IBD view, symbol, tabular IO, flow chart and truth table editors.

The *State Machine Editor User Manual* provides information about the graphical state diagram editor.

The *DesignPad Text Editor User Guide* describes procedures for using the built-in HDL text editor.

The *Debug Detective User Guide* describes procedures for using Debug Detective.

The *ModuleWare Reference Guide* describes procedures for using the ModuleWare library.

A number of quick reference help pages can be accessed by choosing the **Quick Reference Index** from the **Help** menu. Further quick reference pages can be accessed using the Help buttons provided on many of the application dialog boxes.

Copying Text From the Acrobat Viewer

You can copy text from this document by choosing the  (**Text Select)** tool button or V shortcut key in the Acrobat viewer and choosing **Copy** from the Acrobat **Edit** menu (or using the Ctrl + C shortcut).

The text can be pasted into a text editor (or application dialog box) using the Ctrl + V shortcut or the **Paste** menu option if one is provided in the destination window. In the graphic editors, you can use the **Paste Special** option to explicitly paste text from the system clipboard.



If you copy HDL text from a tutorial page, check that punctuation characters are copied correctly. In particular, line feed characters may not be translated on UNIX systems and may need to be re-entered.

Example Designs

Completed examples of the VHDL and Verilog tutorial designs are provided in the HDL Designer Series installation. These can be browsed for reference or can be compiled, simulated and animated directly if you modify their downstream library mapping to use a writable downstream directory.

Refer to the section “Using the Example VHDL Design” on page 1-90 for more information.

Typographic Conventions

The following conventions have been used in this manual:

Style	Usage
Bold	Menu options and command line switches.
Italic	Pathnames (and object names derived from file names). Also used for manual titles (for example, *ModuleWare Reference Guide*).
`Courier font`	Monospaced Courier font is used for code examples.
Links	Cross references within the text are shown in blue.



When pathnames (or window titles derived from pathnames) are shown in this tutorial, the PC convention (\) is used.

Chapter 1
VHDL Timer Exercise

This exercise creates a simple timer using block diagrams and a control block described as a hierarchical state machine. A simple truth table is used to decode four-bit binary codes from the ten-bit input bus. The design is completed using a re-usable component described by a HDL text view.

A test bench is created using a flow chart which can be used as a test harness to simulate the generated VHDL for the timer design. The simulation results can be displayed as animation on the flow chart and state machine to assist in debugging the design. The verified timer design is then synthesized.

The instructions assume that a Model*Sim* simulator and the LeonardoSpectrum synthesis tools are available. However, the VHDL generated from the diagrams can also be used by other compatible downstream tools that are available on your system.

Specification

The timer outputs time data on two four-bit buses representing low and high values. There is also a logic output signal which triggers an audible alarm. The data input is provided on a ten-bit bus and control is provided by start, stop, reset and clock signals. These signals are summarized in the following table:

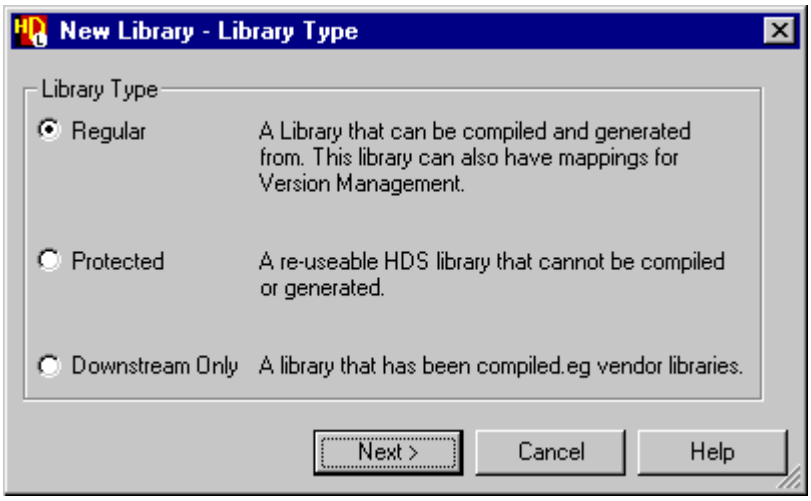
Inputs
start (logic signal)
stop (logic signal)
reset (logic signal)
clk (logic signal)
d (10-bit bus)
Outputs
high (4-bit bus)
low (4-bit bus)
alarm (logic signal)

Set Library Mapping

For this tutorial, you can use an existing project such as the project you created for the Design Management tutorial but you should create new library mapping for this design.

The library mapping defines the logical location of the directories containing your design data. The source graphical objects, HDL and downstream data can be stored at any writable locations on your available file system but are typically saved below a common root directory.

1. To set library mapping, click the **New Library** icon in the Project group on the shortcut bar or choose **Library** from the **New** cascade of the **File** menu in the design manager window to display the New Library wizard:



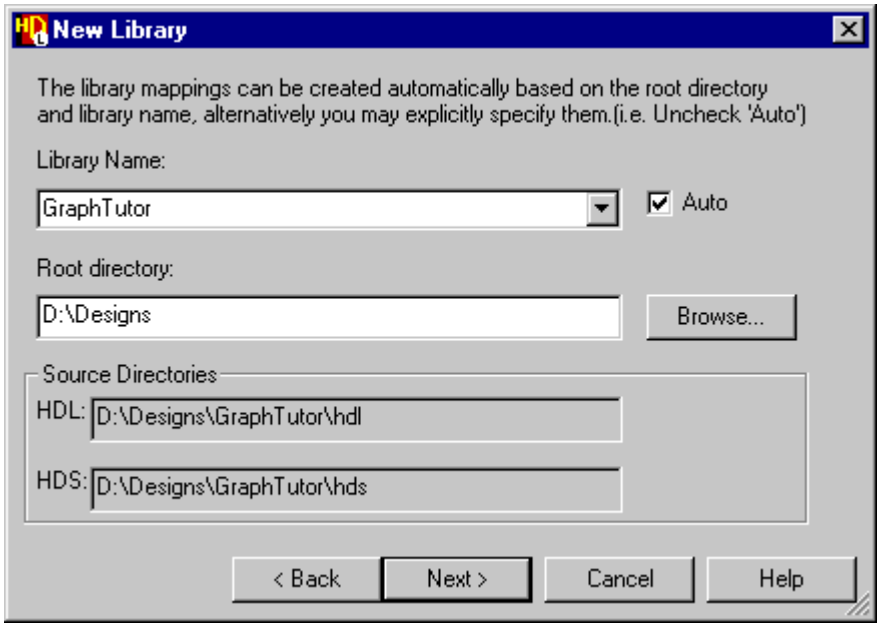
2. Choose the **Regular** library type and click the Next > button to display the library name page.

3. Enter a logical library name (for example: *GraphTutor*) and check that the **Auto** option is set in the dialog box.

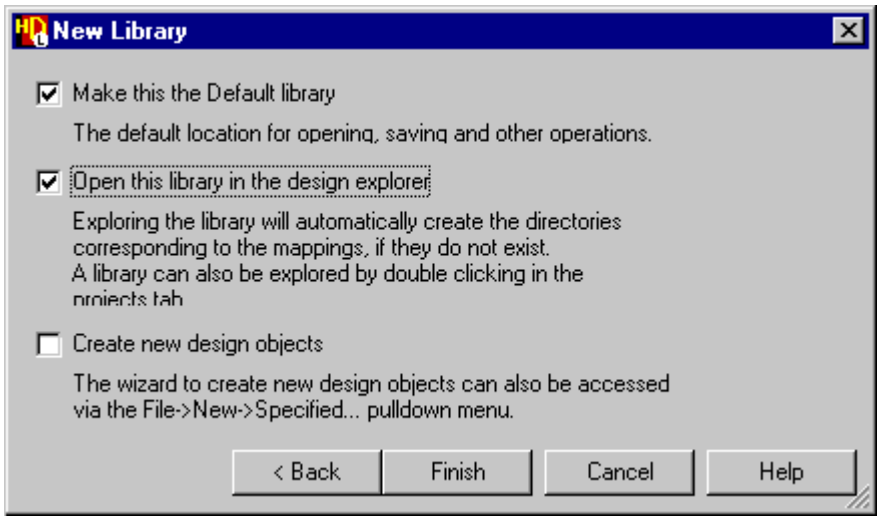
4. Specify or browse for the pathname for the root directory that will contain your library data (for example, *D:\Designs* or *$HOME/designs*).

Library names and pathnames can be entered using upper, lower or mixed case but note that UNIX systems are case sensitive and the case used for pathnames should match the file structure. (On a PC, library names are case sensitive but pathnames are case insensitive.)

Notice that the source HDL directory is named *<root_directory>\GraphTutor\hdl* and the source graphics directory is named *<root_directory>\GraphTutor\hds*.



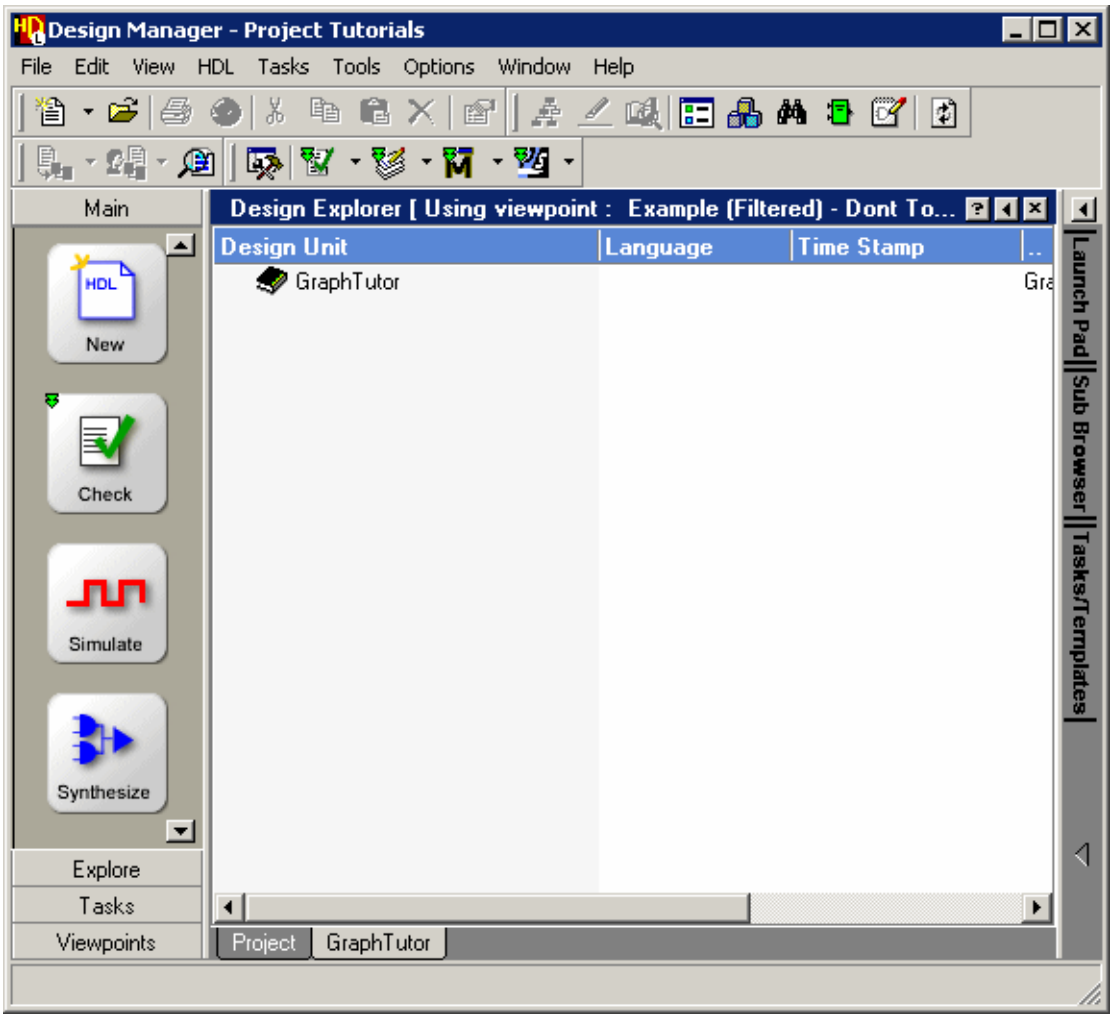
5. Click the Next > button to display the last page of the wizard.



6. Set the **Make this the Default library** and **Open this library in the design explorer** options.

7. Click the Finish button to close the wizard.

The source design data directories you specified in the wizard are created and the library (shown as a closed “book”) is opened in a new design explorer window:



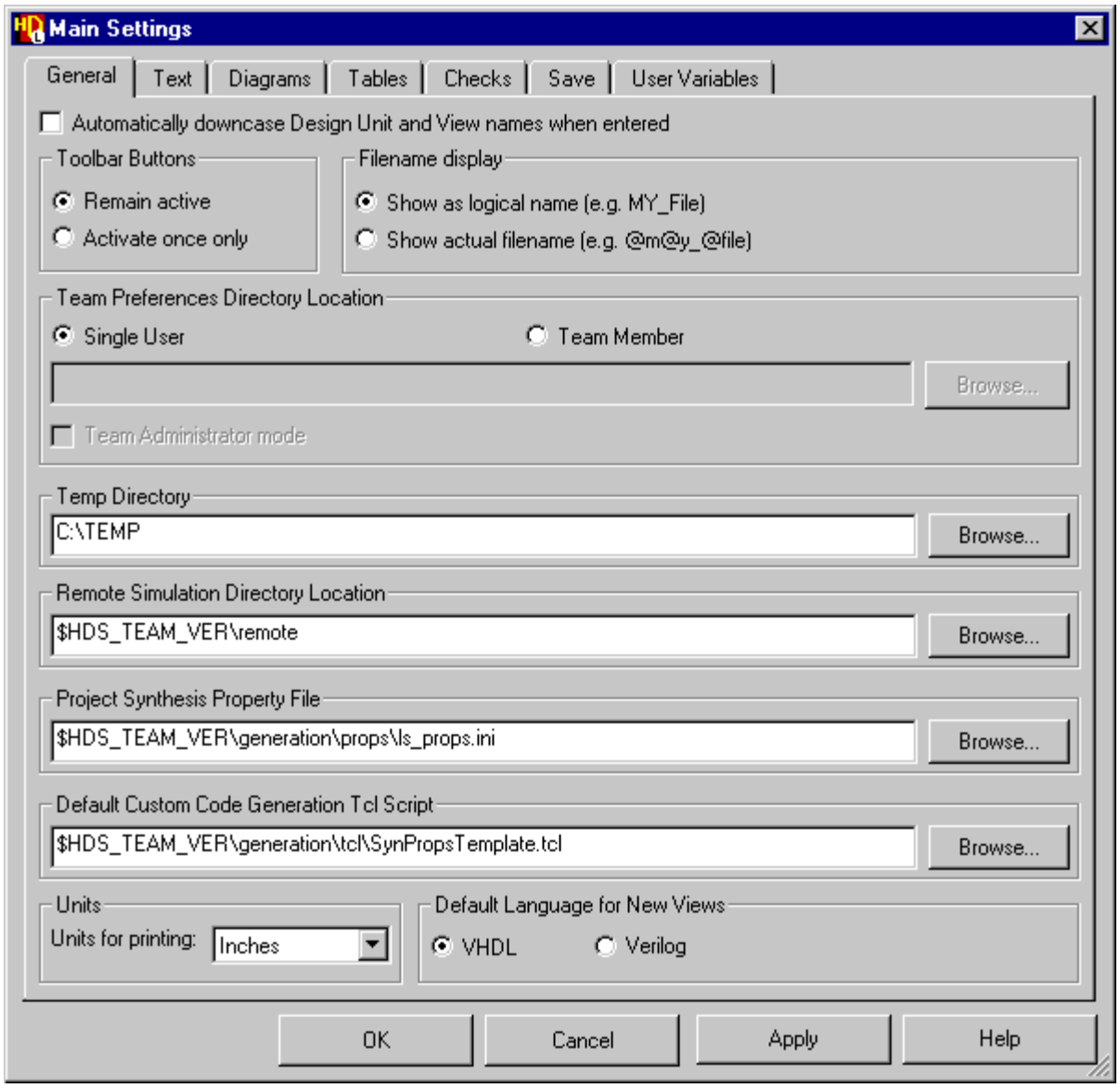
The mapping for the location of compiled data will be defined automatically when the design is compiled.

Set the Default Language

A set of preferences are loaded when you invoke a HDL Designer Series tool for the first time according to what you specify in the HDS Setup Assistant wizard. There are separate tabbed dialog boxes for the main settings, VHDL and Verilog options, HDL2Graphics import options, version management, animation settings and master preferences for each type of graphical diagram.

The preference dialog boxes can be accessed from the **Options** menu.

1. Choose **Main** from the **Options** menu to display the Main Settings dialog box.



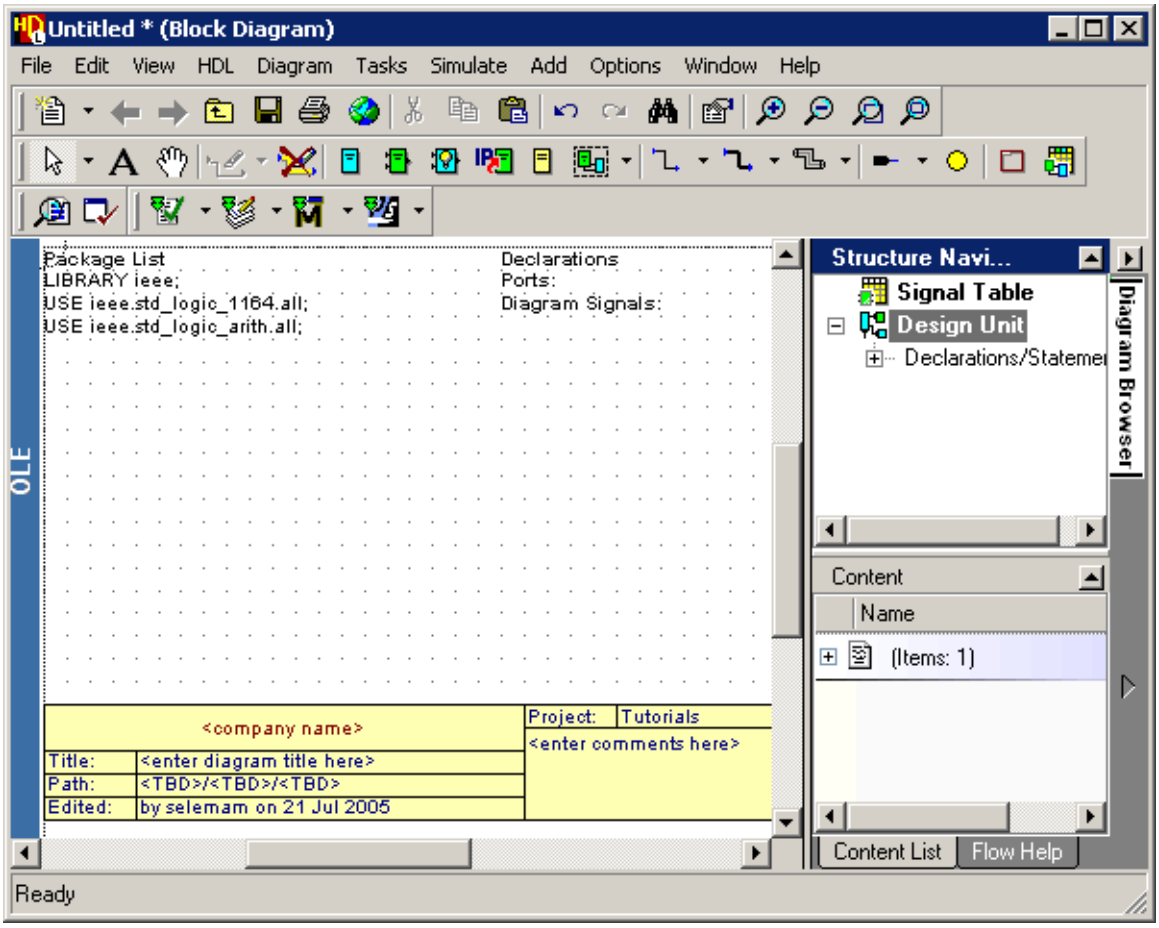
2. Select the **General** tab and ensure that **VHDL** is set as the default language to be used for new graphic editor views. Use the OK button to confirm your choice.

All other preferences can be left with their default values for this tutorial.

Create a Block Diagram

1. Use the  button in the design manager window and select **Block Diagram** from the **Graphical View** cascade of the dropdown menu to create a new untitled block diagram.

2. Use the  button to view the entire diagram:



Notice the toolbars at the top of the diagram. The toolbar buttons provide quick access to many of the most frequently used editing and formatting commands.

The block diagram is a blank sheet except for a background grid, a package list (with the standard IEEE libraries *std_logic_1164* and *std_logic_arith*) and empty text fields with labels for *Declarations*, *Ports* and *Diagram Signals*.

The diagram also shows page boundaries for the default printer and a copy of the default title block.

You can choose to show or hide a side Diagram Browser

Edit the Title Block Template

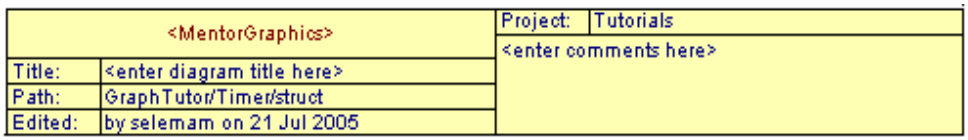
A title block is automatically added to all new diagrams if the **Add Title Blocks in new diagrams** option is set in your diagram preferences.

The default title block is a template with default locations for your company name, diagram title and comments.

The template incorporates internal variables which automatically enter the current project name, your login name and the current date. Internal variables are also used to enter the logical pathname for the design. This path is initially shown as *<TBD>/<TBD>/<TBD>* but the internal variables are converted to show the library, design unit and view name when you save the diagram.

3. Click twice on *<company name>* in the default title block to display the Comment Text page of the Object properties dialog box and replace the default text by the name of your company.

4. Select the title block by clicking with the mouse so that the selection handles are displayed and choose **Save Title Block** from the **File** menu.



5. A dialog box is displayed with a warning that saving the title block cannot be undone. Click the Yes button to proceed.



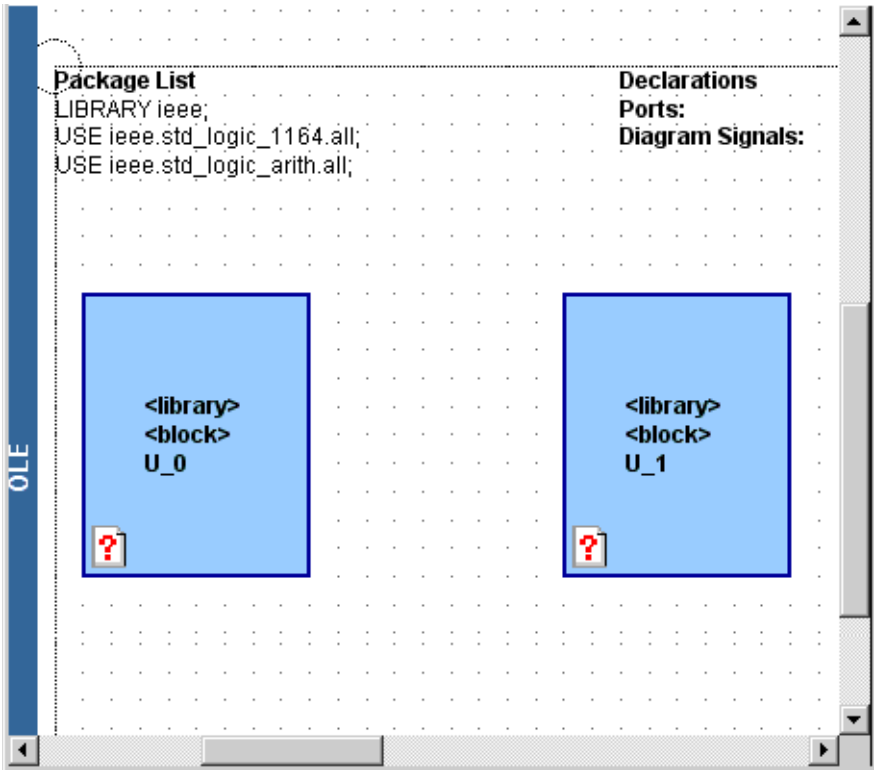
The title block is saved as a team resource at the location specified in your preferences and will be used as the default template in new diagrams.

If shared team preferences have been set up on your system, there may already be a read-only team template stored at this location.

Refer to the *HDL Designer Series User Manual* for information about how you can add and modify title blocks.

Add Blocks

6. Use the button (or choose **Block** from the **Add** menu) to add two blocks on the diagram as shown below:



Notice that the blocks are added with the default library <*library*>, the default name <*block*> and unique instance names (*U_0* and *U_1*).



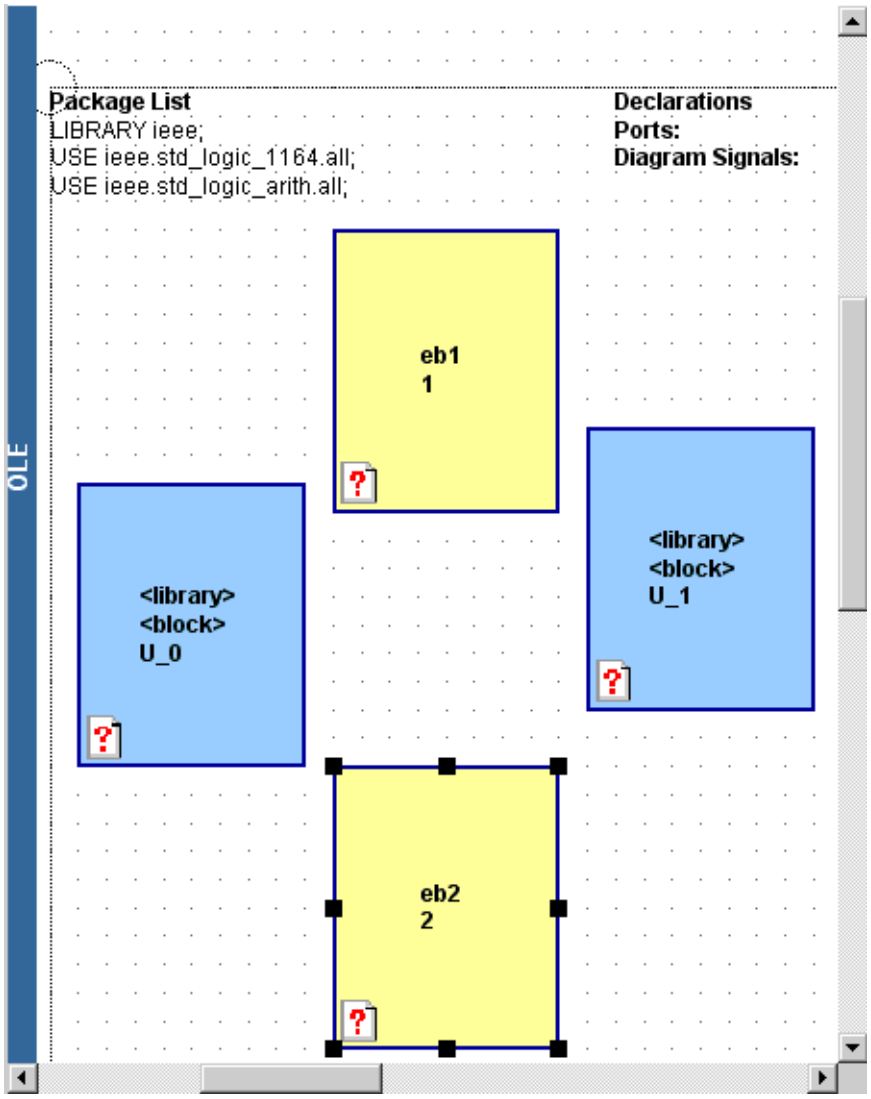
The Add Block command normally auto-repeats until you select another command or terminate the repeating command by using the right mouse button or Esc key. However, you can change the behavior of the toolbar buttons by setting the **Activate once only** preference in the **General** tab of the Main Settings dialog box.

You can also use the Ctrl key with any toolbar button to toggle the repeat mode. For example, when Remain active is set, Ctrl + adds a single block on a block diagram.

The ? indicates that you have not yet specified a view for your object.

Add Embedded Blocks

7. Use the button to add two embedded blocks on your block diagram as shown in the picture below.



Notice that the embedded blocks are added with unique default names (*eb1* and *eb2*) and numbers (*1* and *2*).

The view describing a block must be saved as a uniquely named design unit in a library directory. However, the view describing an embedded block is saved as part of the parent block diagram and does not impose hierarchy when HDL is generated for your design. The name of an embedded block must be unique on the diagram and is used as a label in the generated HDL.

The blocks (*U_0* and *U_1*) will be used to define a child state machine and block diagram view. The embedded blocks (*eb1* and *eb2*) will be defined by concurrent assignment statements on the top level block diagram.

Add Ports and Signals

You can use the following buttons to add signal and bus nets on a block diagram:


Add a signal


Add a signal with a port


Add a bus


Add a bus with a port


Add a bus with a ripper

Signals or buses can be added between any existing connectable items on the diagram or left unconnected by double-clicking to terminate the net with an open net connector. However, you can use the  pulldown on the buttons to change the default setting and terminate with a default port or ripper. Notice that the toolbar button changes to show the current setting.

When the  or  button is selected, a port is automatically added at an unconnected source or destination end point. When the  button is selected, a ripper is used if the end point is over an existing bus or bundle.

8. Choose **Signal with Port** and use the  button to connect three signals originating from the block on the left (instance *U_0* in the picture) to the block on the right (instance *U_1*) and one signal returning from *U_1* to *U_0.*

   The signals are added with unique names (*sig0*, *sig1*, *sig2* and *sig3*) and the default type *std_logic*. Notice how the full declarations for these diagram signals are automatically added to the list of Declarations on the diagram.



Allow one or more grid lines between each port or signal. You can resize objects by selecting a block or embedded block and dragging one of its resize handles.

If necessary, you can drag text elements such as the signal name using the Left mouse button.

9. Add a signal from block *U_0* to the embedded block *eb2* and another signal from a point on *sig2* terminating on the embedded block *eb2*.

10. Add a signal from the embedded block *eb2* terminating in space on the right side of your diagram. Notice that an output port is added when you double-click at the end of the last signal and its declaration is added to the list of ports on the diagram.

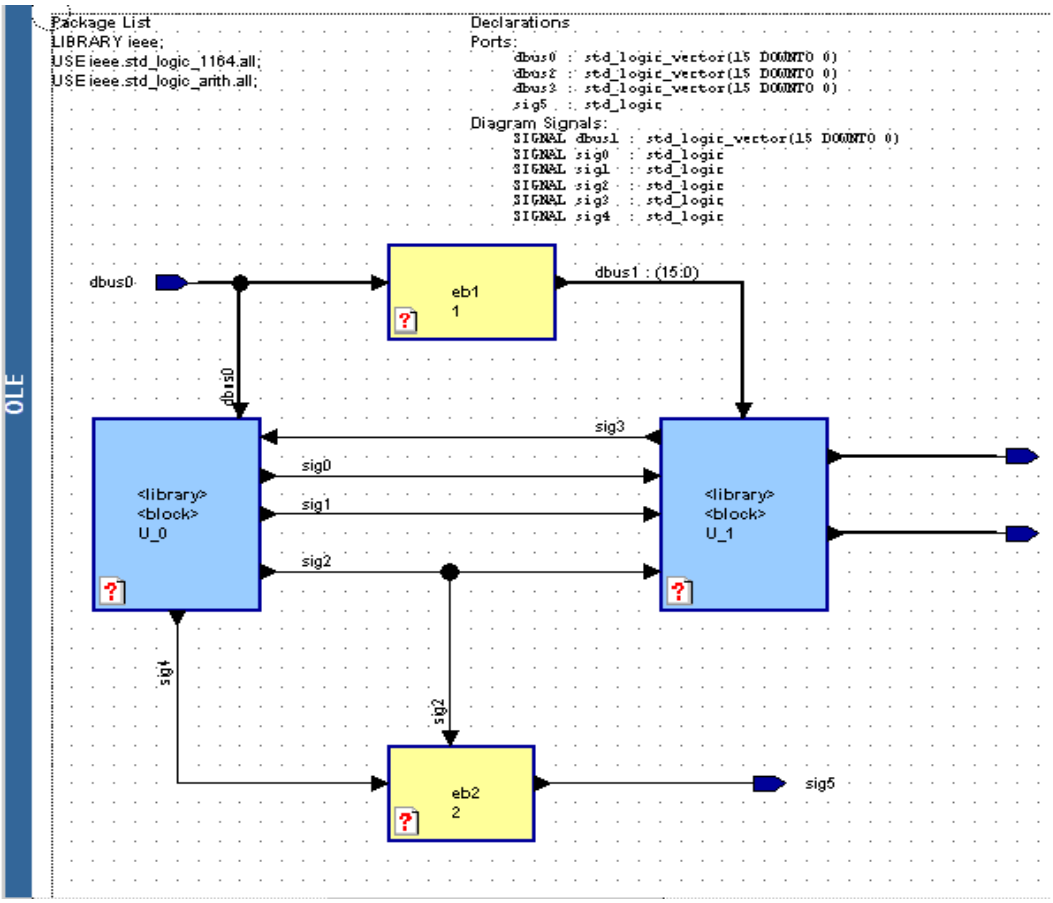
11. Choose **Bus with Port** and use the  button to add a bus from a source on the left side of your diagram with its destination on the upper embedded block *eb1*. A default input port is automatically created at the beginning of the bus.

12. Add another bus starting from this bus and terminating on instance *U_0*. Notice how both bus segments have the same default name *dbus0*. The full declaration showing the default bus type and bounds *std_logic_vector*(*15 DOWNTO 0*) is added to the list of ports.

13. Add a bus (*dbus1*) from *eb1* to *U_1*. This internal diagram signal is shown with the default bounds *(15:0)* shown (in abbreviated format) on the net.

14. Then add two buses (*dbus2* and *dbus3*) from *U_1* terminated with default output ports on the right of the diagram.

Your diagram should now look similar to the picture below:



Add a Bundle and Global Connector

15. Use the button to add three signals on the left side of your diagram. Notice that a default input port is created at the source of each signal but a dangling net connector is drawn when you double-click at the end of each signal.

16. Select the three signals (by dragging a select rectangle with the Left mouse button held down) and use the button to connect a bundle containing these signals to block instance *U_0* as shown in the picture below.

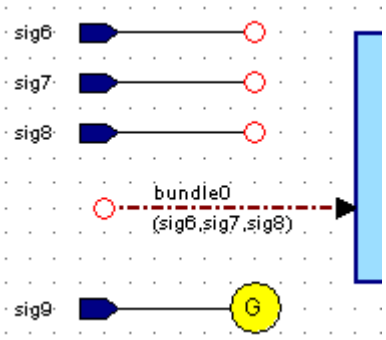
    Notice that the bundle has the default name *bundle0* and the three selected signals are automatically included in the bundle with their names listed under the bundle name.



You can use the pulldown on the button to select a button which allows you to rip one or more signals and buses from an existing bundle.

17. Use the button to add a global connector on your diagram below the bundle.

18. Use the button to add a signal between the global connector and a default input port. (This will be a clock signal which is implicitly connected to every block on the diagram.)





If you make a mistake when editing a diagram, you can use the button to undo your last edit and the button to redo an undo operation.

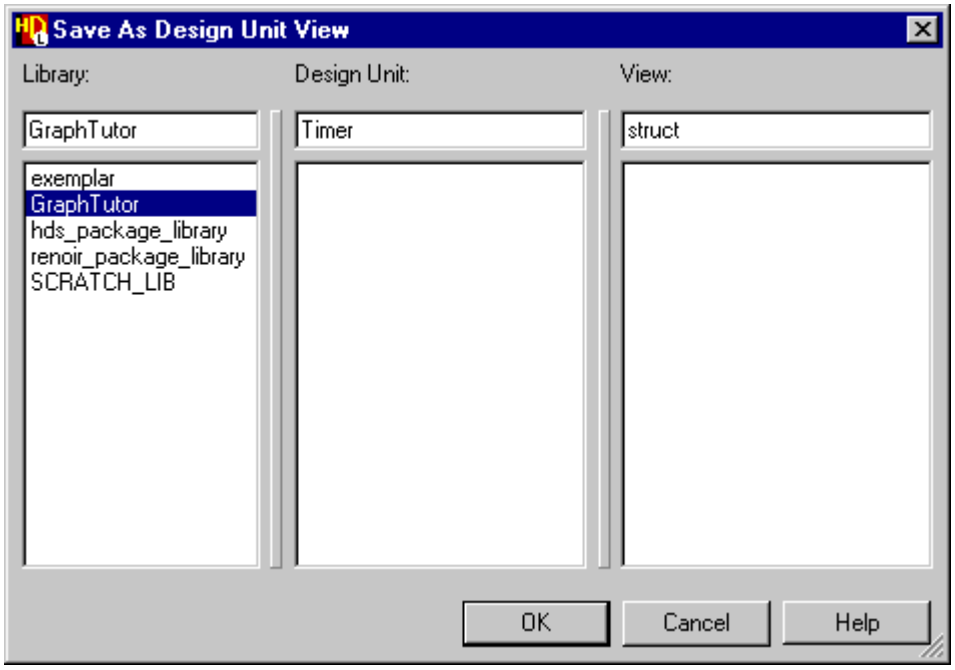
You can also use commands from the **Align** cascade of the **Edit** menu to re-align and distribute objects on the diagram.

Save the Block Diagram

Notice the asterisk (*) character in the header of the block diagram editor window. This indicates that the diagram has been edited since it was last saved.

19. Use the  button to save the block diagram. The Save As dialog box is displayed which allows you to choose from the currently mapped libraries and specify the design unit and design unit view names.

20. Choose the *GraphTutor* library and enter design unit *Timer*. The dialog box should look similar to the example below:



The view name and file extension when you save graphical diagrams defaults as follows:

struct.bd	block diagram
struct.ibd	interface-based design view
fsm.sm	state diagram
flow.fc	flow chart
tbl.tt	truth table
symbol.sb	symbol

If you omit the suffix, it is automatically added to identify the type of diagram you are saving.

The default leaf names can be changed by setting preferences. However, you should not change the extension (*.bd*, *.ibd*, *.sm*, *.fc*, *.tt* or *.sb*) or the design data file will not be recognized and cannot be reopened.

21. When you click the OK button, your diagram is saved and the window title bar is updated to show the pathname *GraphTutor/Timer/struct*.

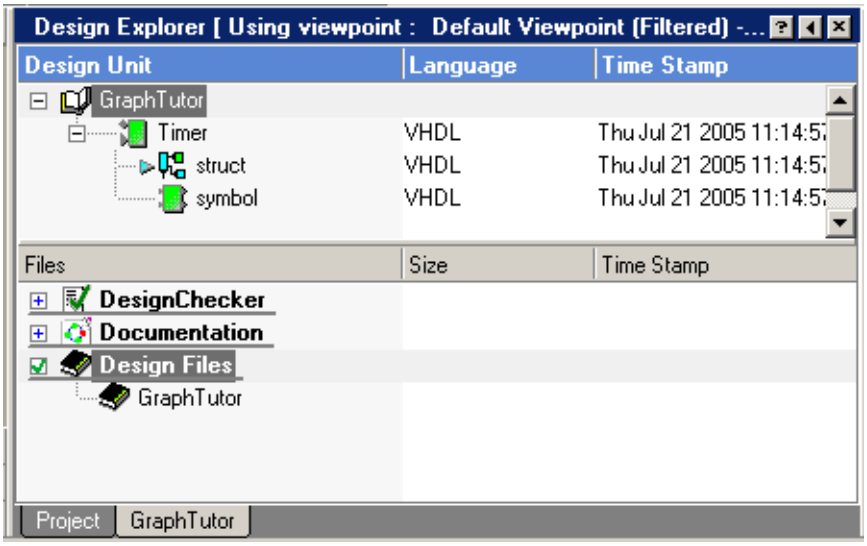
    This path is also added in the title block replacing the <*TBD*> used when the diagram was created. Notice that the asterisk (*) character has been cleared in the block diagram header and the library name used on the blocks in your diagram has been updated to *GraphTutor*.



If the design manager window is obscured, you can pop it to the front by selecting **Design Manager** from the list of open windows in the block diagram **Windows** menu.

22. Click on the ⊞ icon for the *GraphTutor* library in the design explorer and notice that the view is expanded to display the *Timer* design unit.

23. Click on the ⊞ icon for the *Timer* design unit to reveal that it contains a symbol and block diagram view.



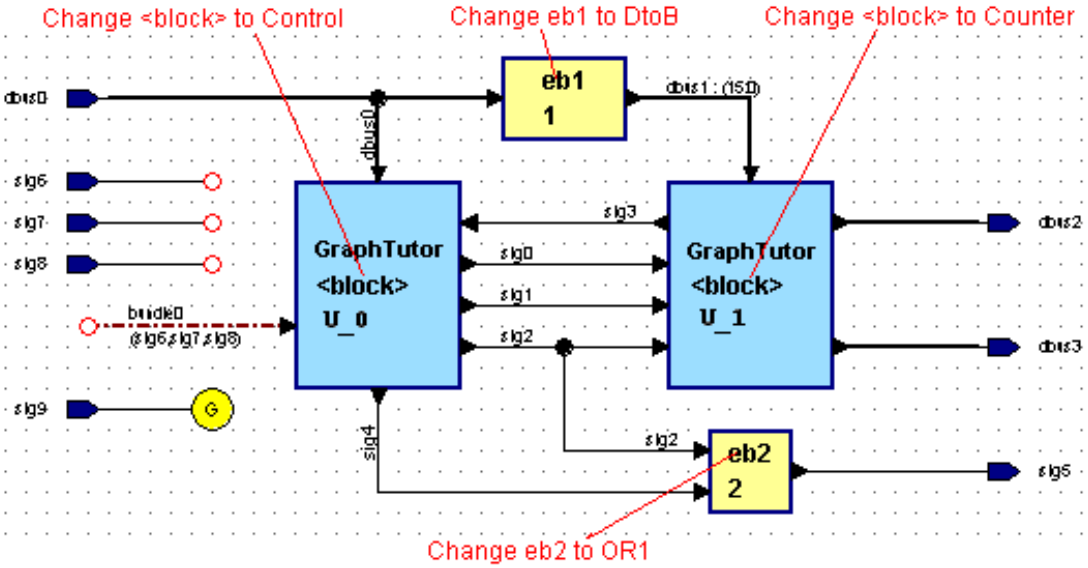
Edit the Block and Signal Names

You now have a completed top-level block diagram for the *Timer* design. However, the blocks and signals have default names.

24. Click on the text <block> in the lower block on the left (instance *U_0* in the picture) and notice the small handles which indicate that the text object is selected. Click again and notice that the text is now highlighted and can be directly overwritten.

    If you click again, the cursor changes to an I-beam which allows you to move the cursor in the text and edit individual characters. Enter the new name *Control* and click outside the text to complete the edit.

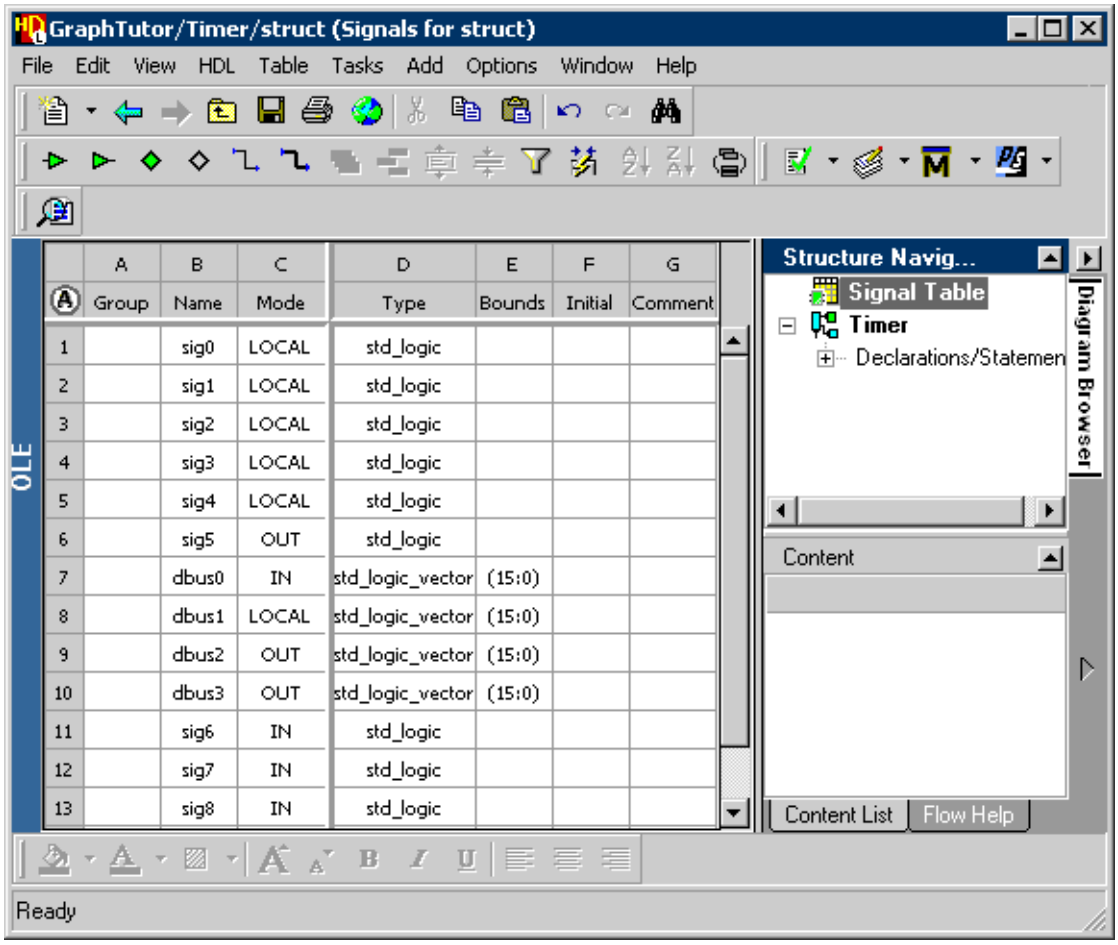
25. Repeat this procedure to change the name of block instance *U_1* to *Counter*, embedded block *eb1* to *DtoB* and embedded block *eb2* to *OR1*.



Direct text editing can also be used to edit the signal and bus names. Alternatively, you can use a dialog box which allows you to edit the properties for a selected object. By default, edits to signal and bus nets are applied only to the connected nets but you can choose to apply the changes to the entire diagram or to propagate changes to all occurrences of the net in the hierarchy of the design.

26. Choose **Entire Net in diagram** from the **Scope for Changes** cascade of the **Signals** cascade in the **Diagram** menu.

27. You can display the signals table by clicking on the **Signal Table** icon in the Structure Navigation pane.



	A	B	C	D	E	F	G
	Group	Name	Mode	Type	Bounds	Initial	Comment
1		sig0	LOCAL	std_logic			
2		sig1	LOCAL	std_logic			
3		sig2	LOCAL	std_logic			
4		sig3	LOCAL	std_logic			
5		sig4	LOCAL	std_logic			
6		sig5	OUT	std_logic			
7		dbus0	IN	std_logic_vector	(15:0)		
8		dbus1	LOCAL	std_logic_vector	(15:0)		
9		dbus2	OUT	std_logic_vector	(15:0)		
10		dbus3	OUT	std_logic_vector	(15:0)		
11		sig6	IN	std_logic			
12		sig7	IN	std_logic			
13		sig8	IN	std_logic			

28. Use the signal table to update the port and signal declarations as shown in the following tables.

Ports:

Old Name	New Name	Type	Bounds
dbus0	d	std_logic_vector	9 DOWNTO 0
sig6	start	std_logic	(none)
sig7	stop	std_logic	(none)
sig8	reset	std_logic	(none)
sig9	clk	std_logic	(none)
dbus2	low	std_logic_vector	3 DOWNTO 0
dbus3	high	std_logic_vector	3 DOWNTO 0
sig5	alarm	std_logic	(none)

Diagram Signals:

Old Name	New Name	Type	Bounds
dbus1	dat_in	std_logic_vector	3 DOWNTO 0
sig0	clear	std_logic	(none)
sig1	load	std_logic	(none)
sig2	hold	std_logic	(none)
sig3	zero	std_logic	(none)
sig4	beep	std_logic	(none)



All occurrences of each signal name (including the bundle contents) should be automatically updated on the diagram when you confirm the dialog box.

If any nets are not updated, check that you have set the scope for changes to **Entire Net in Diagram** as described on page 1-15.

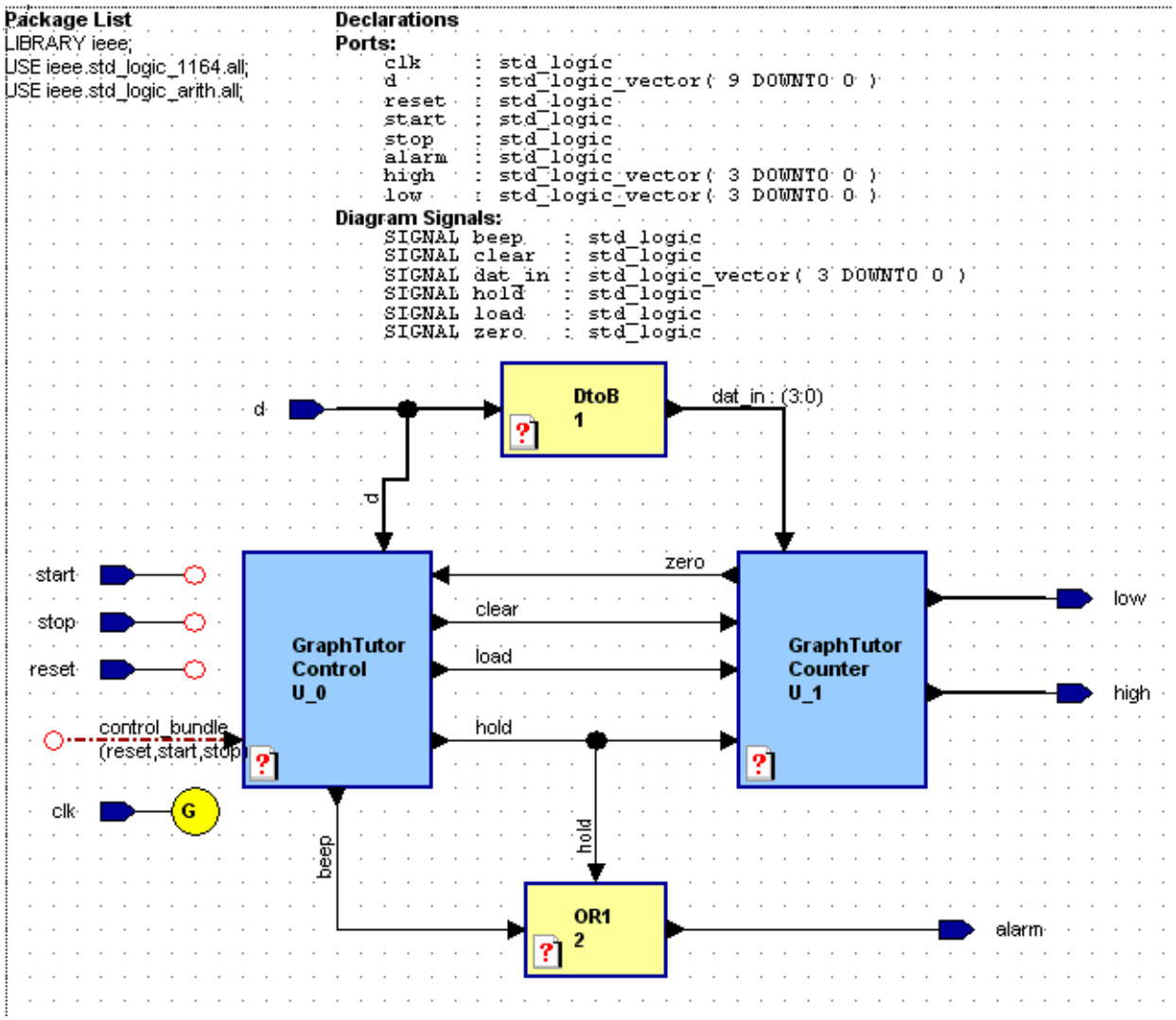
29. If you choose to update the signal and bus declarations through the signal table, save your changes and click the **Timer** object icon in the Structure navigation pane to display your block diagram.

30. Select the bundle name and use direct text editing or the **Bundles** page of the Object Properties dialog box to change the bundle name to *control_bundle*.



If you have difficulty selecting text objects, you can change the selection mode by using the pulldown on the button to select text or object shapes only.

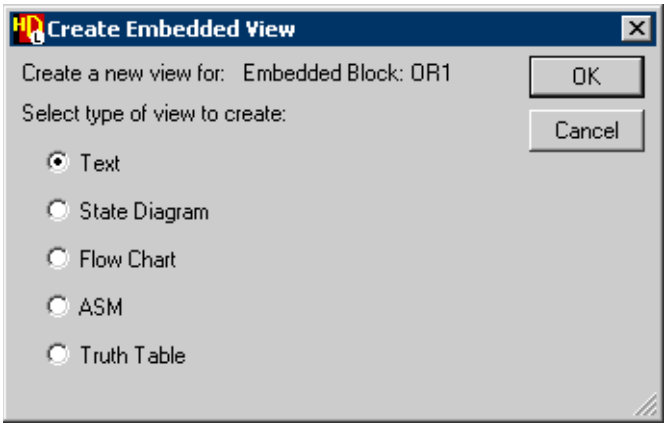
Your block diagram should now look similar to the following picture:



31. Use the button to save the block diagram.

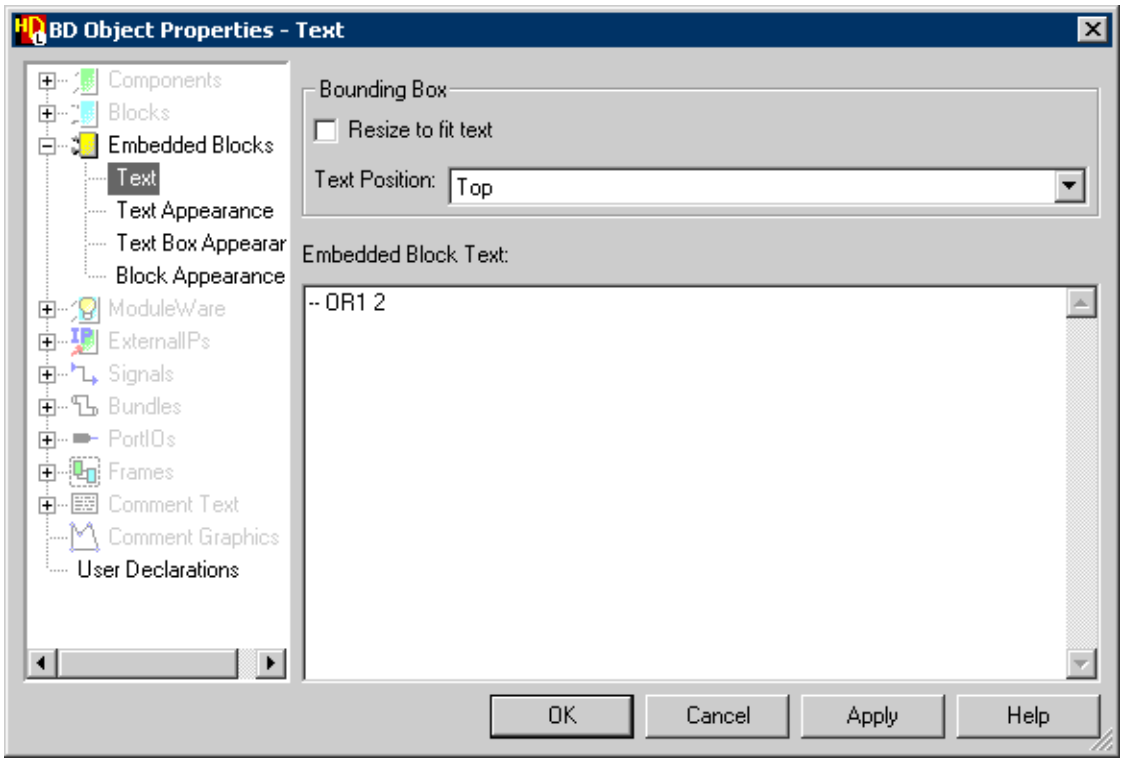
Add an Embedded HDL Text View

32. Select the *OR1* embedded block and display the popup menu by clicking the Right mouse button. Choose **New View** from the **Open** cascade in the popup menu to display the Create Embedded View dialog box. Choose Text in the dialog box.



When you confirm the dialog box, an embedded HDL text view containing default text is displayed on the block diagram adjacent to the embedded block.

33. Select the text, re-display the Object Properties dialog box if necessary (using the button) and choose the **Text** page.



34. Check the bounding box **Resize to fit text** option and enter the following VHDL statement under the default comment text in the dialog box:

```
-- OR1 2
alarm <= hold OR beep;
```

    The modified HDL text is checked for syntax errors and applied to the diagram when you click the OK or Apply button on the dialog box.

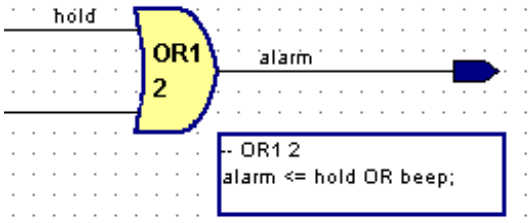
The functional blocks on the diagram are shown by default as simple rectangular shapes. However, it is sometimes useful to use logic notation when a block has a specific logical function. For example, in this block diagram, the *OR1* embedded block represents a logical OR function.

You can apply a logical OR shape by choosing **Autoshapes** from the **Shape** cascade of the popup menu or from the **Embedded Blocks** Block Appearance page of the Object Properties dialog box.

35. Select the embedded block and choose Autoshapes from the Shape cascade of the popup menu to display the Choose Shape dialog box. Select Or from the dialog to apply a logical OR shape to the embedded block.

36. Hide the port arrow heads by clearing the **Show Ports when connected** check box in the **Embedded Blocks** page of the Object Properties dialog box.

The *OR1* embedded block should now look similar to the following picture:





It is also possible to indicate an active low (Not) or edge triggered clock signal. This feature can be used with autoshapes to represent extra functions such as an invertor, NAND, NOR or flip-flop.

The logical OR function could also be implemented using a ModuleWare part similar to that described in “Add ModuleWare Components” on page 1-53.

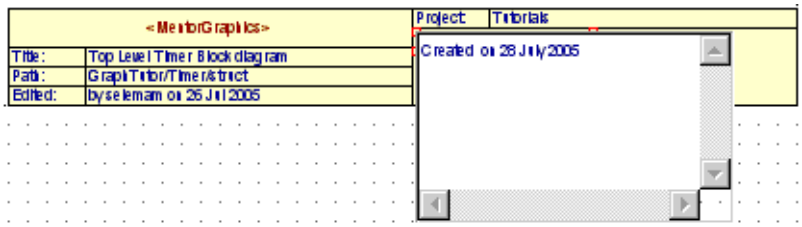
Add a Panel and Edit the Title Block

A panel can be useful to outline areas of a diagram. For example, you can use a panel to outline a view used for simulation or animation.

37. Use the button to add a panel and hold down the Left mouse button to drag the panel and enclose the graphical objects on your diagram. The panel is added with the default name *Panel0*.

38. Complete the block diagram by editing the title and comments in the title block on the diagram. For example, enter the title *Top Level Timer Block Diagram* and a comment of the form: *Created by <your name> on <date>*.

The title block comprises a number of grouped comment text objects. Each comment text object can be edited directly by clicking twice on the text to display a text entry box.



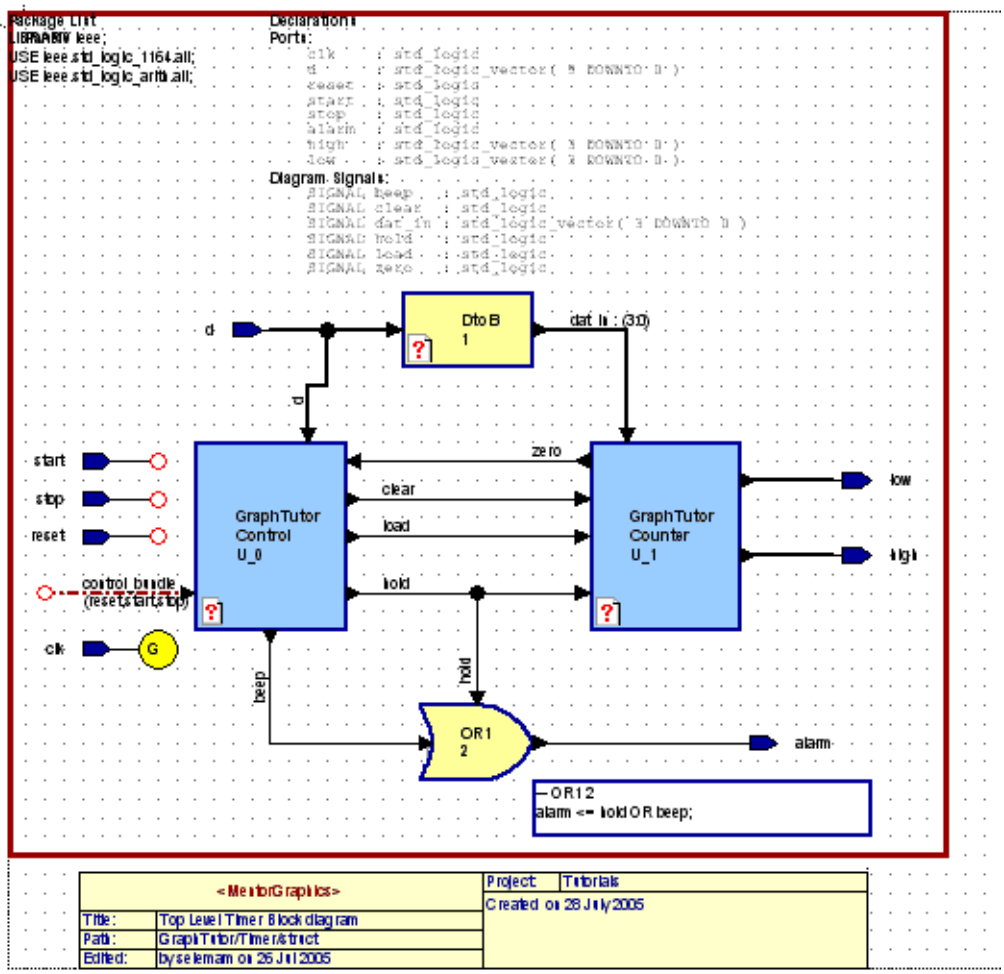
You can enter free-format text including line breaks and spaces which will be preserved on the diagram...



You can also edit an existing comment text object by double-clicking to display the **Text** page in the Object Properties dialog box.

When comments are edited in the dialog box, it is possible to enter any special characters (for example accents or Kanji characters) which are supported on your system.

The completed block diagram should look similar to the following picture:



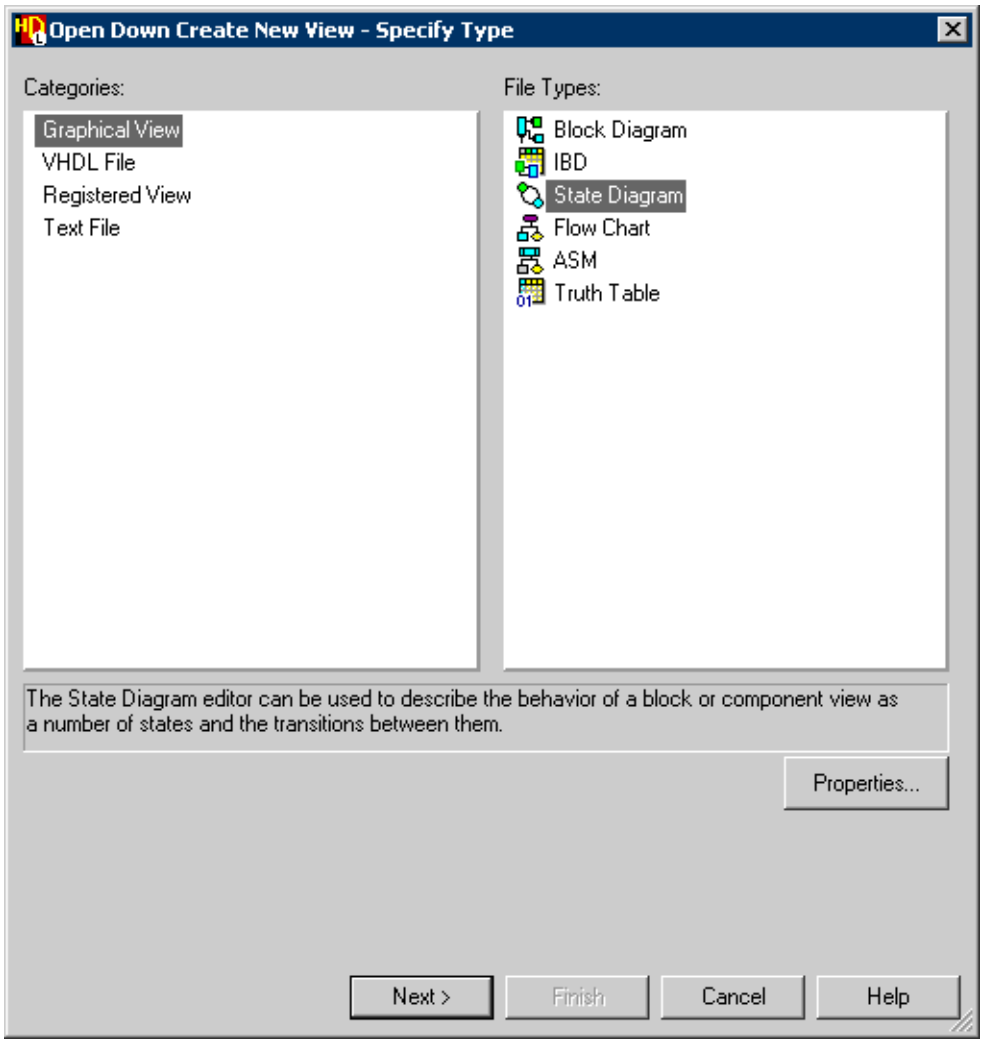
39. Use the button to save your work.

Create a Child State Diagram

The procedures in this section create a graphical state machine to describe the *Control* block.

1. Move the cursor over the body of the *Control* block on the *Timer* block diagram, then press and release the Right mouse button to select the block and display the popup menu.

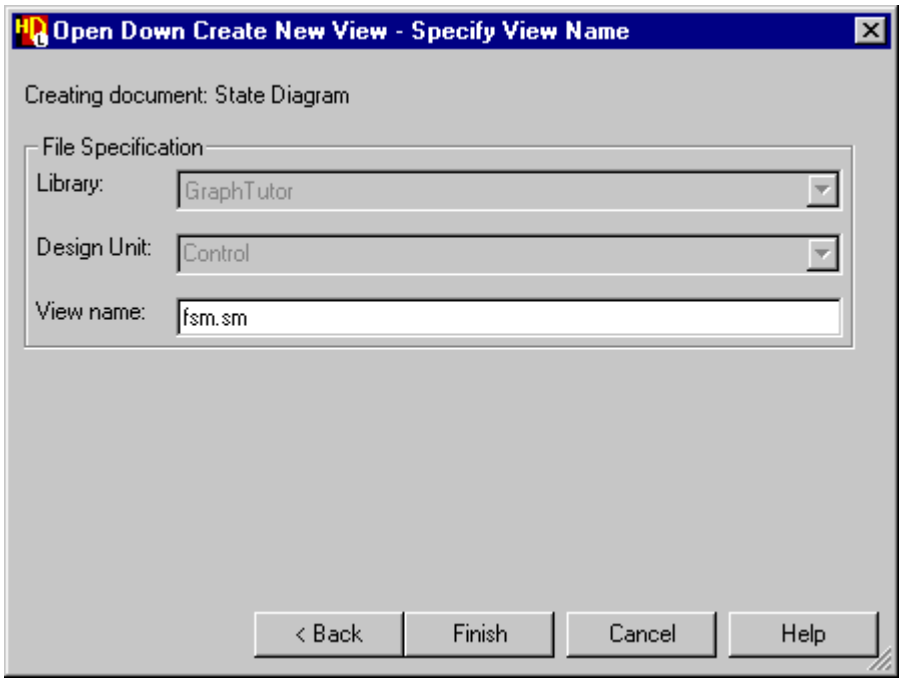
2. Choose **New View** from the **Open As** cascade menu. The Open Down Create New View wizard is displayed:





Solid handles are displayed when the body of a block (or other re-sizable object) is selected. You can display the wizard directly by double-clicking on the body of a block which has no views defined.

3. Select *State Diagram* from the list of file types and use the Next > button to display the Specify View Name page of the wizard:



The library and design unit fields are shown dimmed because they are copied automatically from the library and design unit of the parent diagram. The view name defaults to *fsm.sm* for a state machine.



You do not need to enter the file extension (*.bd*, *.fc*, *ibd*, *.sm* or *.tt*) for graphical views as the correct extension is automatically added. However, if you do enter any other extension you are warned that the file will not be recognized.

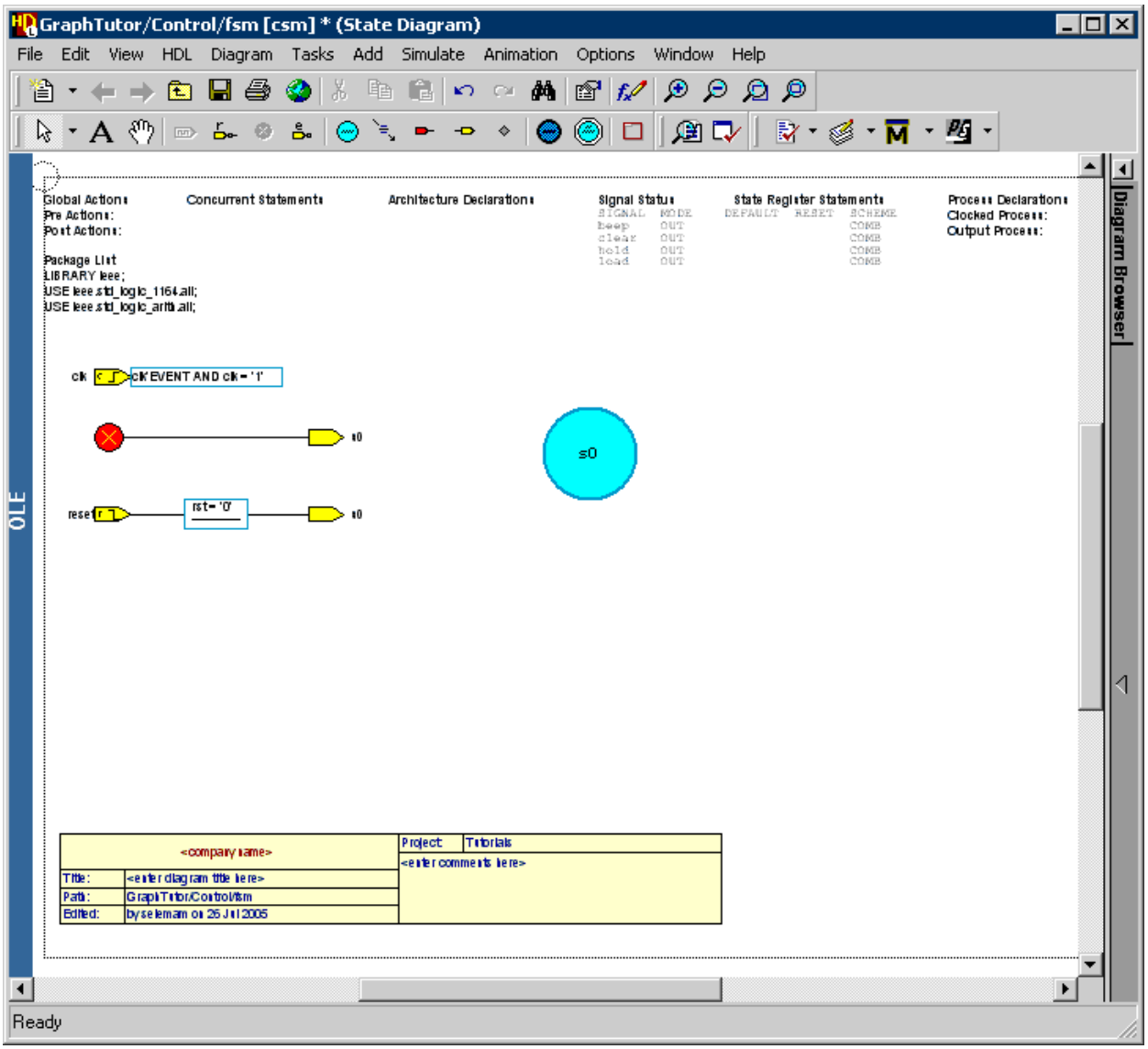
4. Use the Finish button to confirm the wizard with the default view name *fsm.sm.*

A new state diagram *GraphTutor/Control/fsm [csm]* is created as a child view of the *Control* block.

Note that a default state machine name *csm* is appended to the design unit and view names in the diagram title. This name can be changed by choosing **Rename State Machine** from the **Diagram** menu in the state diagram.

The state diagram is initialized with a default clock point, recovery state point, reset point and simple state with page boundaries set for the default printer.

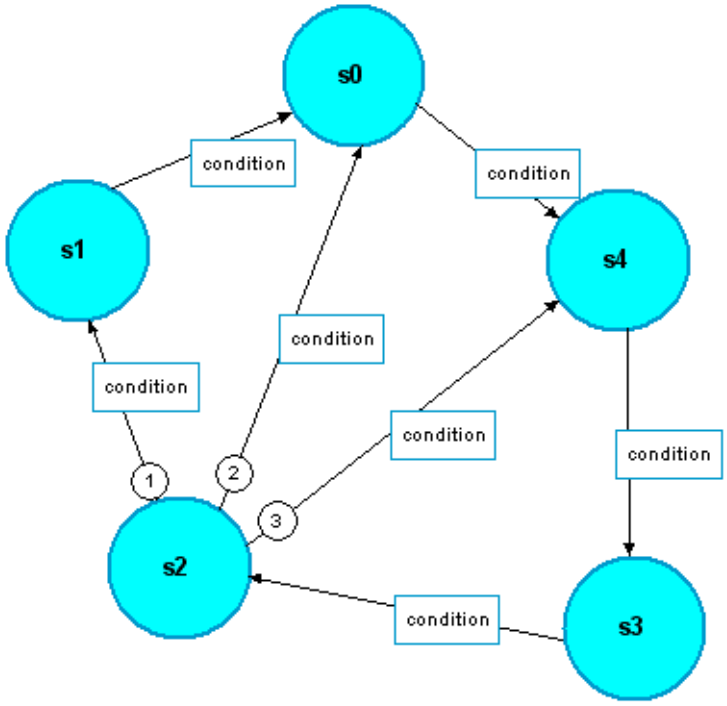
The diagram includes text objects for the default VHDL package list and labels for global actions, concurrent statements, architecture declarations, signals status, process declarations and state register statements:



Add States and Transitions

5. Use the button to add four states on your state diagram. The states are added with default names *s1*, *s2*, *s3* and *s4*.

6. Use the button to add transitions between the states as shown in the picture below.



The transitions are added with default conditions which will be edited later in this tutorial.

Notice that when you add more than one transition leaving a state, the transition priority is indicated by a priority number associated with the transition arc. The priorities are initially assigned in the order that you add the transitions but will be re-assigned later in this tutorial if necessary.

If you add a transition in the wrong direction, you can easily change its direction by choosing **Reverse Direction** from the popup or **Diagram** menu.

Note that a popup description (known as an object tip) is displayed when the cursor is stationary over an object. In particular, when the cursor is over a transition, the source state and the destination state are named even if the states are outside the current window.

Save the State Diagram

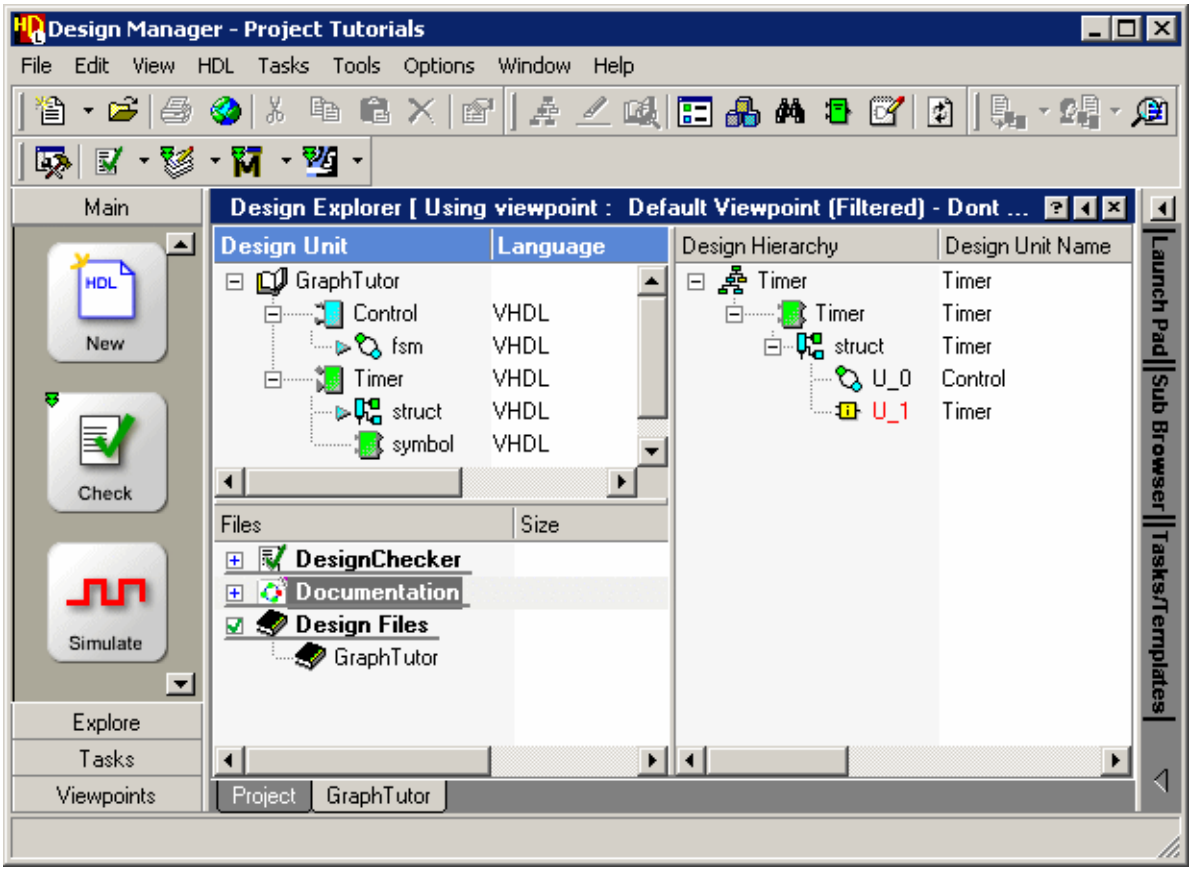
7. Use the button to save the state diagram. The state diagram was created as a child view from its parent block diagram and is saved using the library, design unit and view names specified when it was created. The design explorer is updated to display the *Control* design unit.

   The *Control* design unit is shown as a block in the design explorer because its interface is defined by the connections on its parent block diagram. The *Timer* design unit is shown as a component because it has no parent block diagram and its interface is defined by a symbol.

8. Click on the ⊞ icon for the *Control* design unit in the design explorer to expand the design unit revealing that it contains a state diagram view .

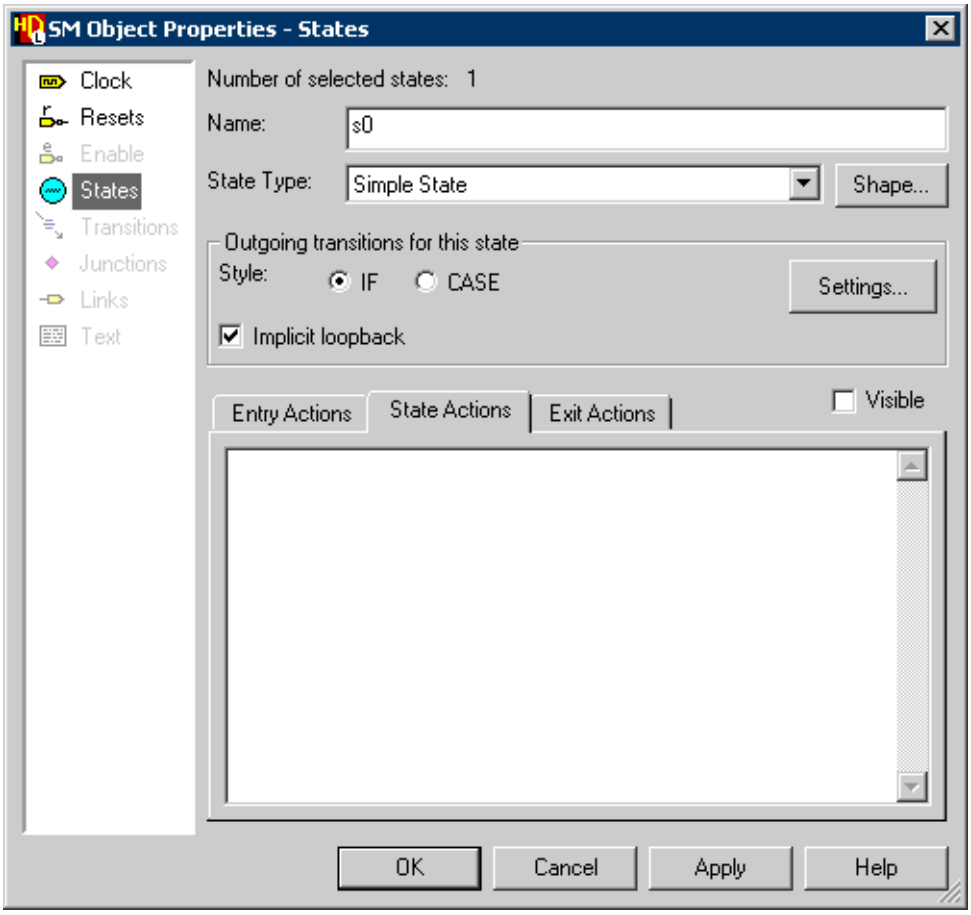
9. Select the *Timer* design unit and choose **Show Hierarchy** from the popup menu.

10. Select the Timer hierarchy node in the *Design Hierarchy* pane and choose **Expand All** from the popup menu to reveal the full hierarchy:



Edit the States

11. Select the start state (*s0*) and use the  button to display the **States** page of the State Machine Object Properties dialog box:





You can also display the dialog box by choosing **Object Properties** from the **Edit** menu or by double-clicking on a state.

The **States** page allows you to enter a name and actions text for one or more selected states on a state diagram. You can also change the visibility of state actions and (when a single state is selected) change the state to a simple state, a hierarchical state or a wait state.

12. Use the dialog box to enter the following state names and actions replacing the default state names *s0* to *s4* :

Old Name	New Name	Style	Actions
s0	flush	IF	hold<='1'; clear<='1'; beep<='0';
s1	count	IF	(no actions)
s2	getkey	IF	hold<='1';
s3	load_t	IF	hold<='1';
s4	load_u	IF	hold<='1'; load<='1';



The Expression Builder dialog box can be explicitly displayed at any time by choosing **Expression Builder** from the **Edit** menu.

The expression builder can be used to choose from lists of the available port or local signal names, operators and example values.

For example, choose the *hold* signal, <= button, value *'1'* and ; button to enter the action *hold <= '1';*.

The syntax for state actions is automatically checked and any errors reported on entry. VHDL statements must be terminated by a semicolon (;) character. Line breaks and spaces can be used for clarity and will be preserved on the diagram.



You can also edit the state name or actions by direct text editing on the diagram or copy a state and paste its state actions into another state by choosing the **Paste State Actions** option from the **Paste Special** cascade in the popup menu.

If the name is larger than the state, it auto-resizes to fit the new name. You can also resize a state by selecting the state and dragging one of its resize handles but you cannot make it smaller than the enclosed name.

Edit the Transitions

13. Hold down the Left mouse button and select the two transitions entering the state *flush* by dragging the cursor across them (or Shift + Left mouse button). Use the button to display the **Transitions** page of the State Machine Object Properties dialog box.

    Replace the default condition by the new condition *stop='1'* in the dialog box.

    Repeat this procedure for the *start='1'* and *d/=NOUGHTS* conditions.

14. Edit the remaining transition conditions to match the following table:

Origin	Destination	Priority	Condition	Actions
count	flush	1	stop='1'	(none)
getkey	flush	3	stop='1'	(none)
getkey	count	1	start='1'	(none)
getkey	load_u	2	d/=NOUGHTS	(none)
flush	load_u	1	d/=NOUGHTS	(none)
load_u	load_t	1	(none)	(none)
load_t	getkey	1	d/=NOUGHTS	(none)



The *NOUGHTS* text string is the name of a constant which will be declared as a state machine property later in this tutorial.

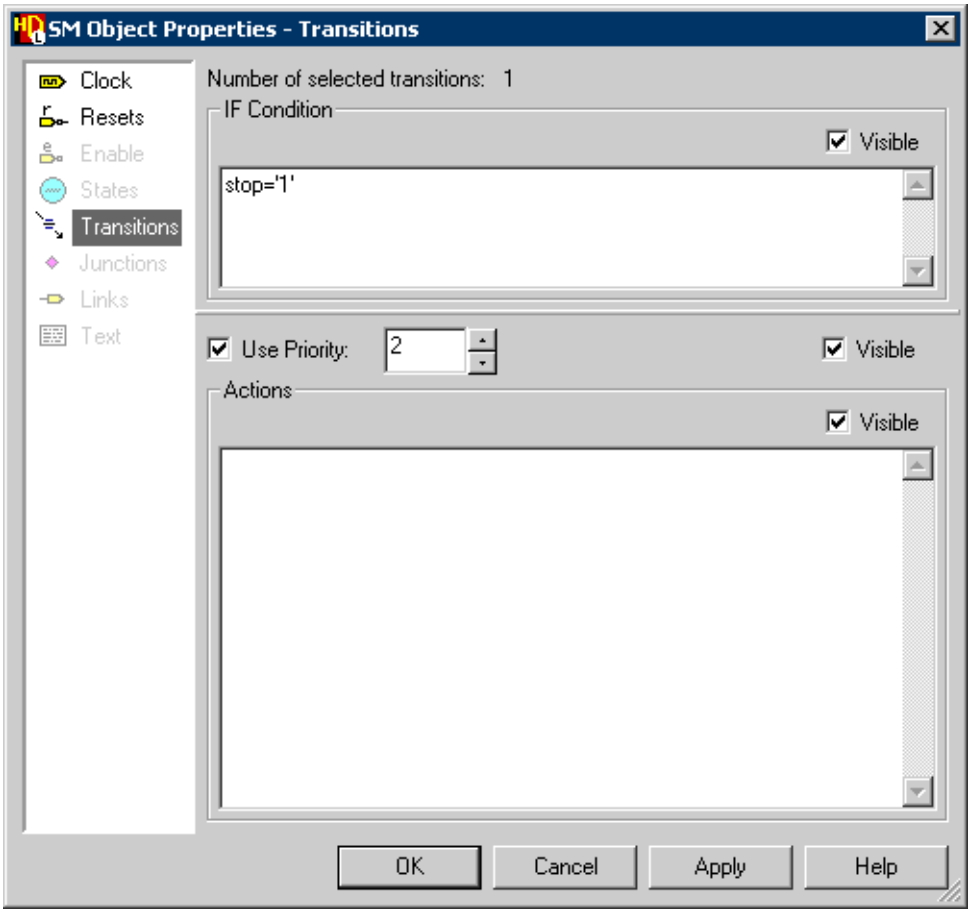
Where conditions or actions should be left blank this is indicated by ***(none).***

The condition syntax is checked on entry. Any valid VHDL fragment can be entered using line breaks and indentation to improve the legibility on the diagram if required.

Ensure that the transition priorities leaving the state *getkey* are the same as those shown in the table.

You can change a transition priority in the **Transitions** page of the State Machine Object Properties dialog box. Alternatively, you can use the button to zoom in or the button to view an area and use direct text editing to change the priority.

When you change a transition priority, the priority of the other transitions leaving the same state are automatically adjusted.

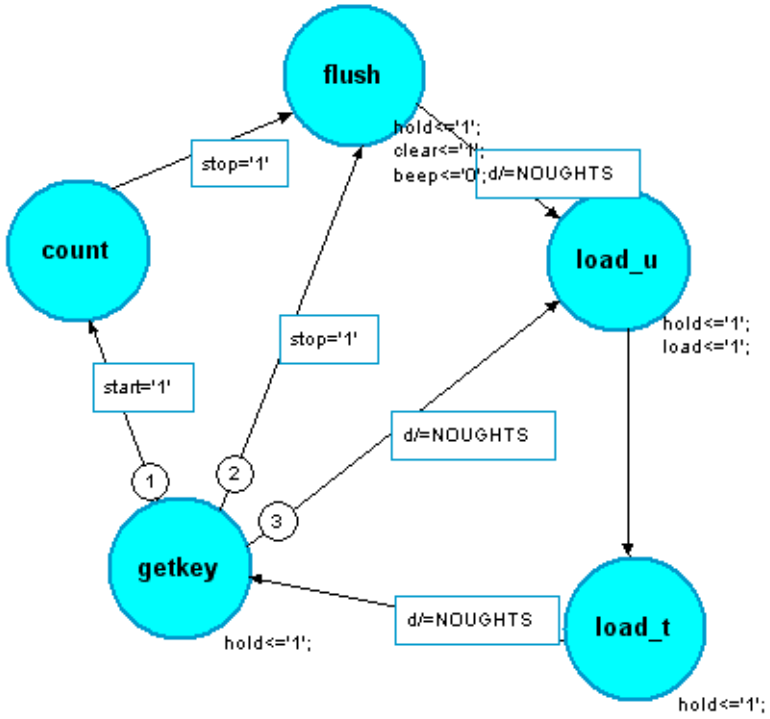


If you have zoomed in, use the button to view all of the state diagram.

15. Use the button to save your changes to the state diagram.

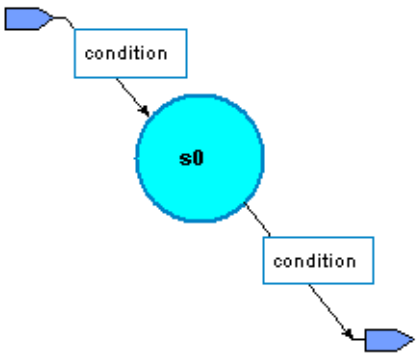
Create a Hierarchical State Diagram

The state diagram should look similar to the following picture.



16. Select the *count* state and choose **Hierarchical State** in the **Change To** cascade from the **Diagram** menu. Alternatively, choose the Hierarchical State pulldown option for the State Type in the **States** page of the State Machine Object Properties dialog box. The *count* state is redrawn as a hierarchical state with a triple outline and darker fill color.

17. Double-click on the *count* hierarchical state (or use the **Open Down** option from the popup menu) to create the new hierarchical child state diagram. A new child state diagram is opened as a window initialized with a default state *s0* connected to an entry point and exit point by transitions with default conditions.



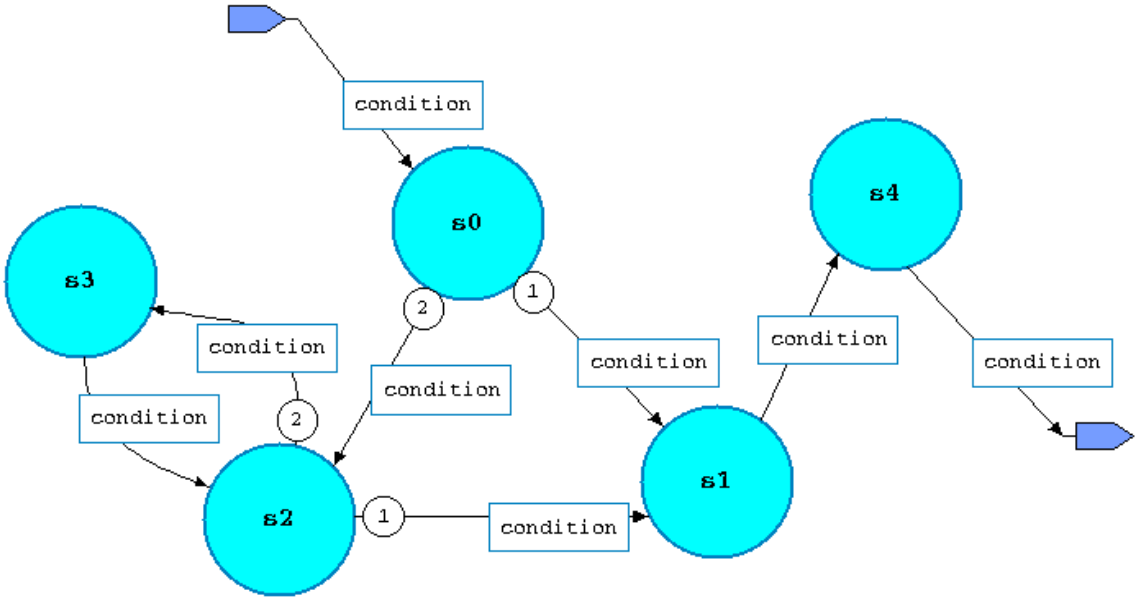
Notice that the name of the hierarchical state *count* is included in the window title: *GraphTutor/Control/fsm [csm/count].* This convention shows that the child diagram is a partial view of the parent diagram.

18. Select the exit point and choose **State** in the **Change to** cascade of the **Diagram** menu.

19. While the new state is selected, drag with the Right mouse button and release the mouse button with the ghosted state to the left and below the first state. Use the **Copy Here** option from the popup menu to make a copy of the state at the cursor position.

20. Repeat this procedure to add two more states on the diagram. This method for adding objects can be useful when you want to add an object with the same or similar properties and attributes to an existing object.

21. Use the button to add a new exit point and the button to connect transitions between the states as shown in the following picture:



You can add route points by clicking at several points between states to create a smooth arc as shown in the picture between states *s2* and *s3*.

You are not limited by the page boundaries when you add objects or edit a graphical diagram.

If necessary, you can drag a rectangle around the objects to select them and move them inside the page boundary or choose **Refresh Page Boundaries** from the **File** menu to show how the edited diagram will be fitted onto printer pages.

Complete the Hierarchical State Diagram

22. Use the **States** page of the State Machine Object Properties dialog box to rename the states and add state actions as shown below:

Old Name	New Name	Style	Actions
s0	standby	IF	hold<='1';
s1	alarm	IF	hold<='1'; clear<='1'; beep<='1';
s2	counting	IF	(none)
s3	suspended	IF	hold<='1';
s4	end_count	IF	hold<='1'; clear<='1';

23. Use the **Transitions** page to edit the conditions:

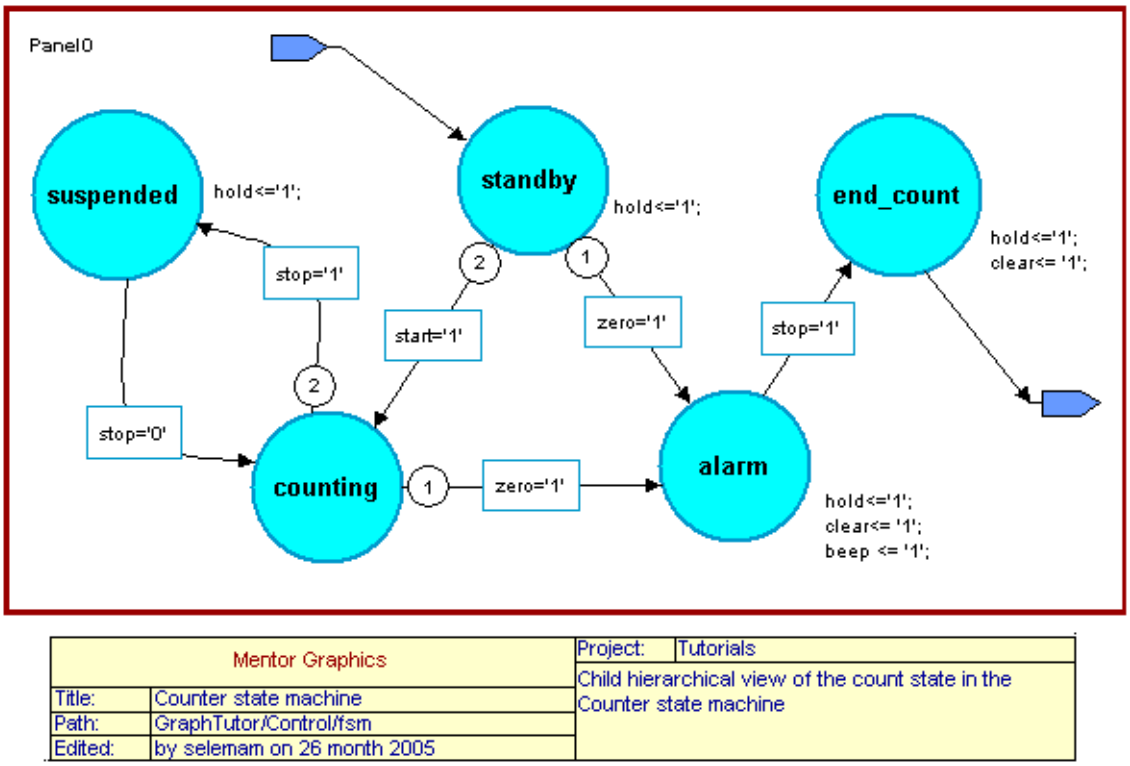
Origin State	Destination State	Priority	Condition	Actions
suspended	counting	1	stop='0'	(none)
counting	suspended	2	stop='1'	(none)
counting	alarm	1	zero='1'	(none)
standby	counting	2	start='1'	(none)
standby	alarm	1	zero='1'	(none)
alarm	end_count	1	stop='1'	(none)
entry point	standby	1	(none)	(none)
end_count	exit point	1	(none)	(none)



You can remove the default conditions on the transitions connected to the entry and exit points by selecting the transitions and using the Del key.

24. Complete the hierarchical state diagram by editing the title and comments in the title block and using the button to add a panel around the graphical objects on your diagram. This panel can be used to set the diagram view when you animate the state diagram later in this tutorial.

    The state diagram should look similar to the picture below:



Mentor Graphics		Project:	Tutorials
Title:	Counter state machine	Child hierarchical view of the count state in the Counter state machine	
Path:	GraphTutor/Control/fsm		
Edited:	by selemam on 26 month 2005		

Ensure that the transition priorities are as shown and that you include the terminating semi-colon when you enter the state actions.

25. Use the button to save the state diagram.

    Although it is displayed in a separate tab, a child hierarchical diagram is a partial view of a state machine and the parent and child diagrams are saved as a single design unit view (*GraphTutor/Control/fsm.sm*).

Editing State Machine Properties

There are a number of properties associated with a state machine which can be edited using the State Machine Properties dialog box.

The dialog box can be accessed by choosing **State Machine Properties** from the **Diagram** or popup menu in the parent or child state diagram. It can also be accessed by double-clicking over one of the labels which are displayed in the parent diagram of a hierarchical state machine.

26. Use the  button (or choose the **Open Up** option from the **File** menu) to re-display the parent state diagram if it is not already displayed.

27. Double-click the 

mouse button over the Global Actions, Concurrent Statements or State Registers label on the diagram to display the **Statement Blocks** page of the State Machine Properties dialog box.

    There are no global actions, concurrent statements or state register statements required in this design. Hide the labels for these objects on the state diagram by clearing the **Visible** check box for Global Actions, Concurrent Statements and State Register Statements in the **Statement Blocks** page.

28. Choose the **Declarations Blocks** page. As there are no process declarations required in this design, hide this label by clearing the **Visible** check box for the **Process Declarations** .

29. Enter the following in the Architecture Declarations edit box on the **Declarations Blocks** page for the 10-bit constant *NOUGHTS* in the free-format entry box:

```
Constant NOUGHTS : std_logic_vector := "0000000000";
```

30. Click the 

button to confirm the dialog box and update the diagram.

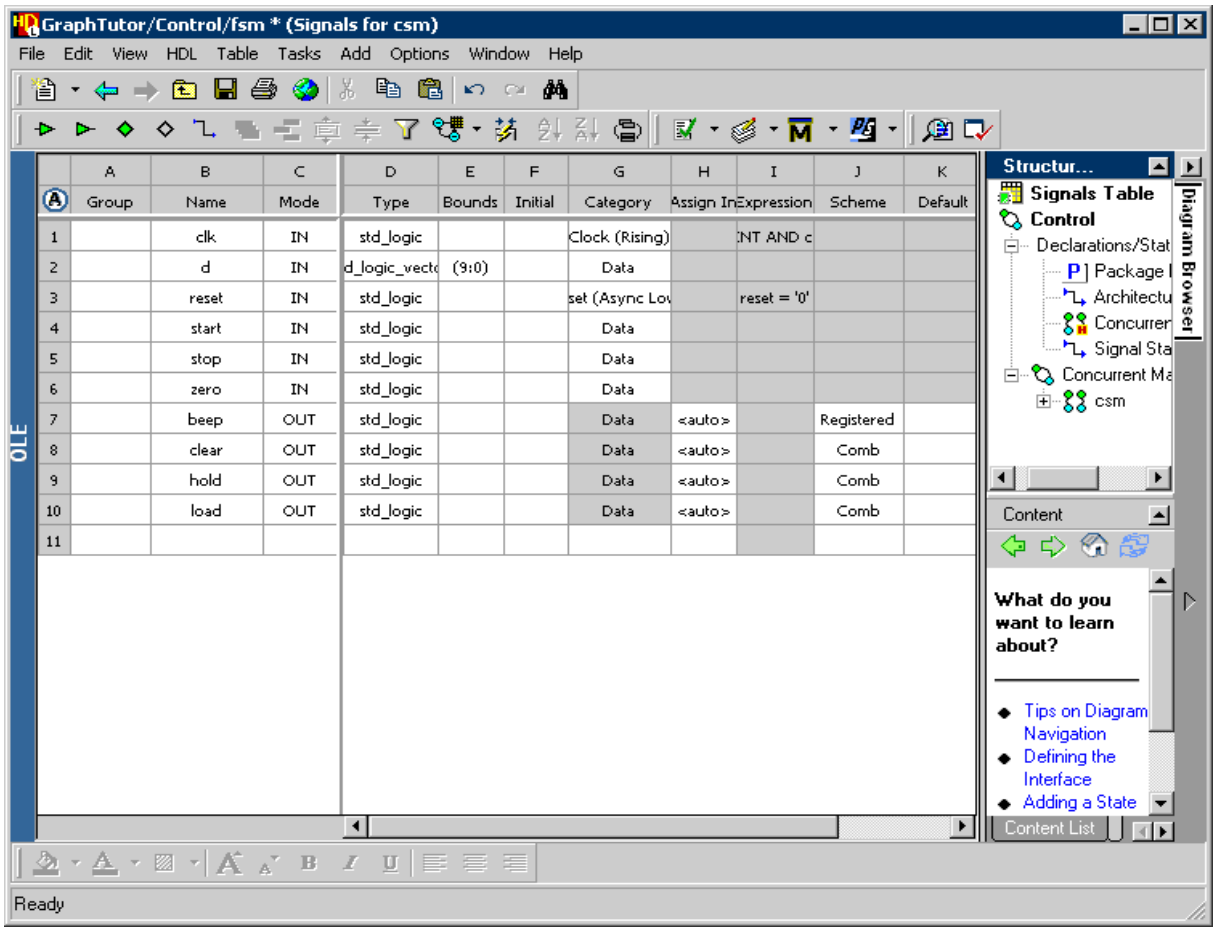
    The constant declaration is added below the Architecture Declarations label on the diagram and will be included as architecture declarations when HDL is generated for the state machine. All other labels other than the default signals status should now be hidden on the diagram.

31. Display the Signals Table by double-clicking over the Signals Status label on the diagram or by clicking the Signals Table icon in the Structure Navigator pane of the Diagram Browser sub-window.

    Notice how the output signals are listed in the dialog box with the default value set to *'0'* and the scheme *COMBINATORIAL*.

32. Click on the Scheme field for the `beep` signal and choose *REGISTERED* from the drop down box.

    Change Reset value field to *'0'*.





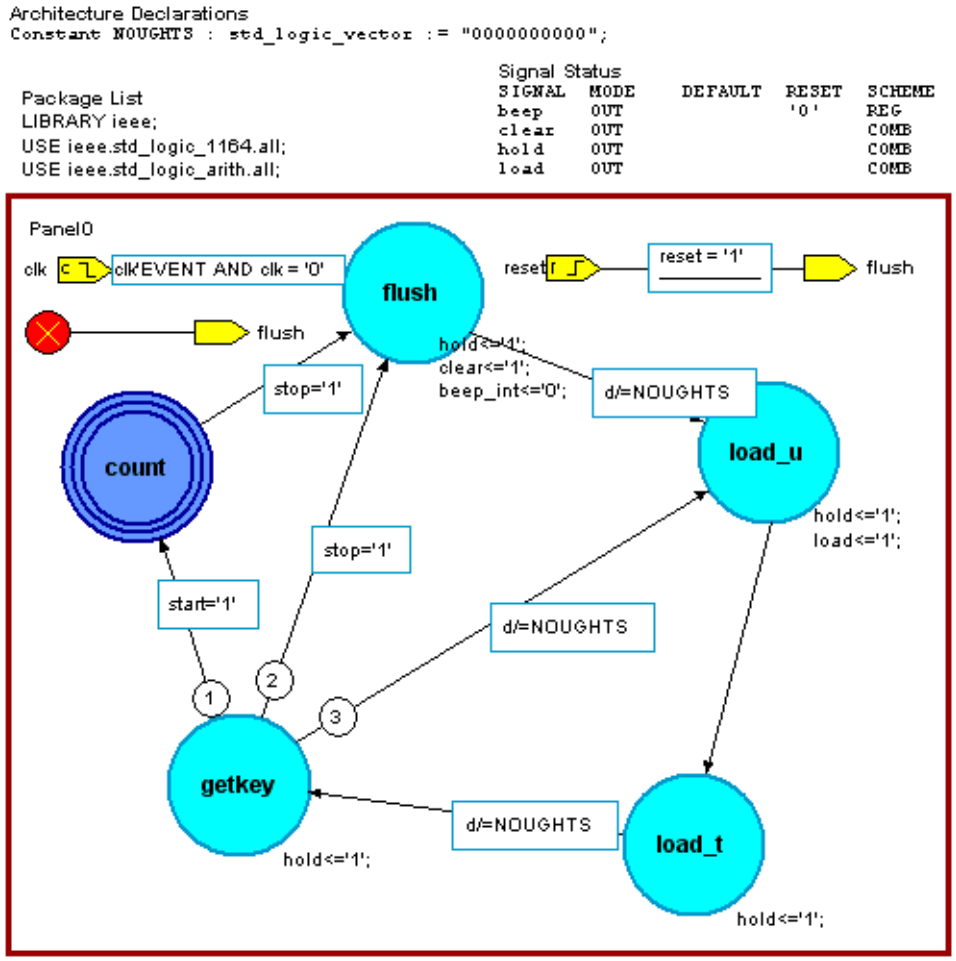
Any locally declared internal signals would be shown with *REGISTERED* status. Bidirectional or buffer signals are treated as output signals.

33. Click the **Save** button.

34. Complete the state diagram by editing the title and comments in the title block and using the  button to add a panel around the graphical objects on your diagram.

    This panel can be used to set the diagram view when you animate the state diagram later in this tutorial.

The final state diagram should look similar to the picture below:



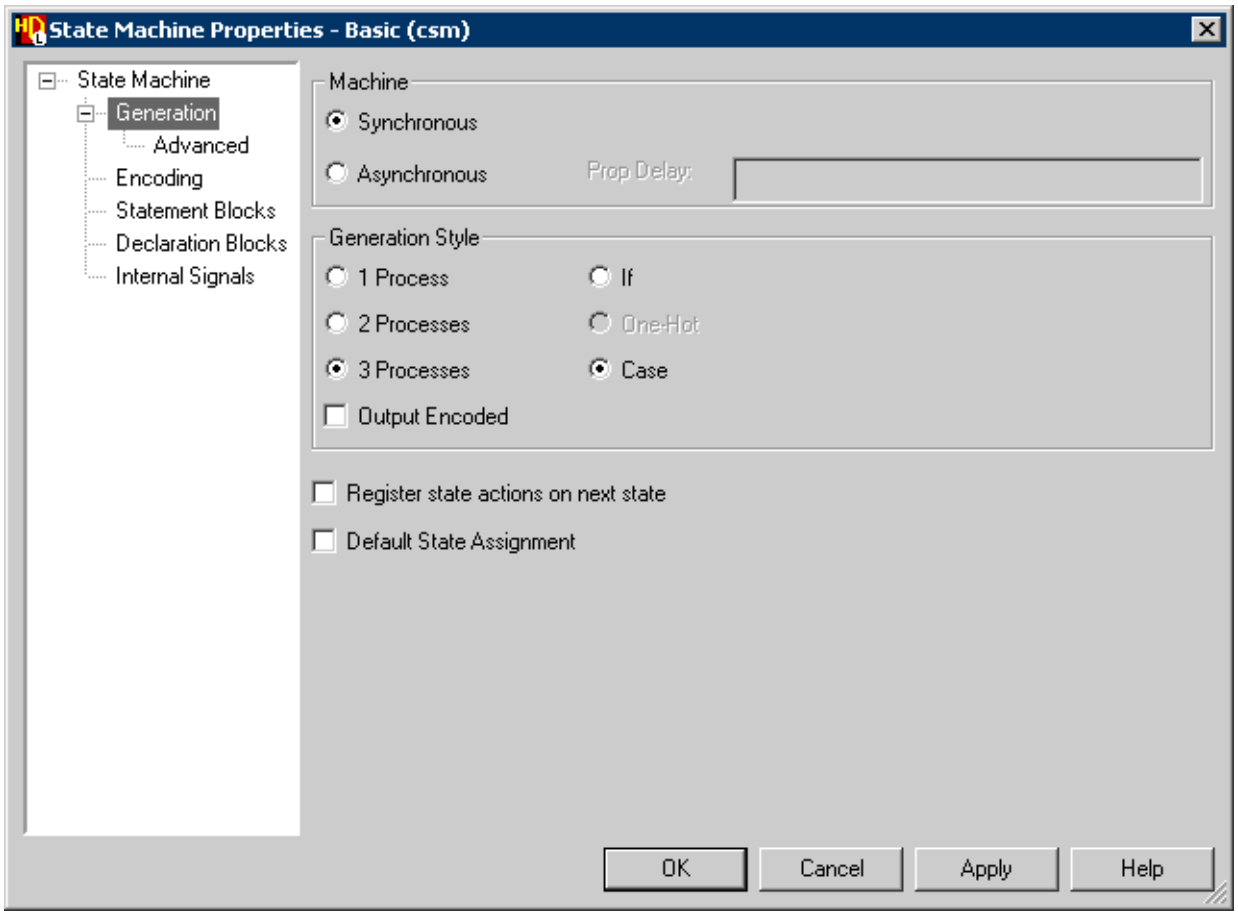
Notice that all occurrences of the *beep* signal (in the *flush* state and in the *alarm* state on the child hierarchical state diagram) have been replaced by the internal signal name *beep_int* using the default suffix *_int*.

The State machine Properties dialog box provides pages for setting HDL generation characteristics and state machine encoding. State machine encoding is not used in this tutorial and the encoding scheme should be set to **None**.

You can use the Help buttons for more information about the State Machine Properties dialog box.

Set Generation Properties

35. Display the State Machine Properties dialog box by choosing **State Machine Properties** from the **Diagram menu** and choose the **Generation** page This page sets properties used for HDL generation.



36. Use the default **Synchronous** option in the Machine box. Use the default options **Case** and **3 Processes** for Generation Style. Leave the **Register state actions on next state** and **Default State Assignment** options unset.

37. From the Advanced sub-page select **Auto Generate Type** for the State Variable and if a Model*Sim* or NC-Sim simulator is available, set the **Instrument HDL for animation** option.This option adds additional code to the generated HDL which is used for state machine animation later in this tutorial.



This additional code is surrounded by translation control pragmas which ensure that it is ignored by downstream synthesis tools.

38. Use the OK buttonto apply the generation properties to your state machine and dismiss the dialog box.

39. Select **Object properties** from the popup menu of the clock object to display the Clock Object properties page and choose *Falling* from the Edge pulldown list.

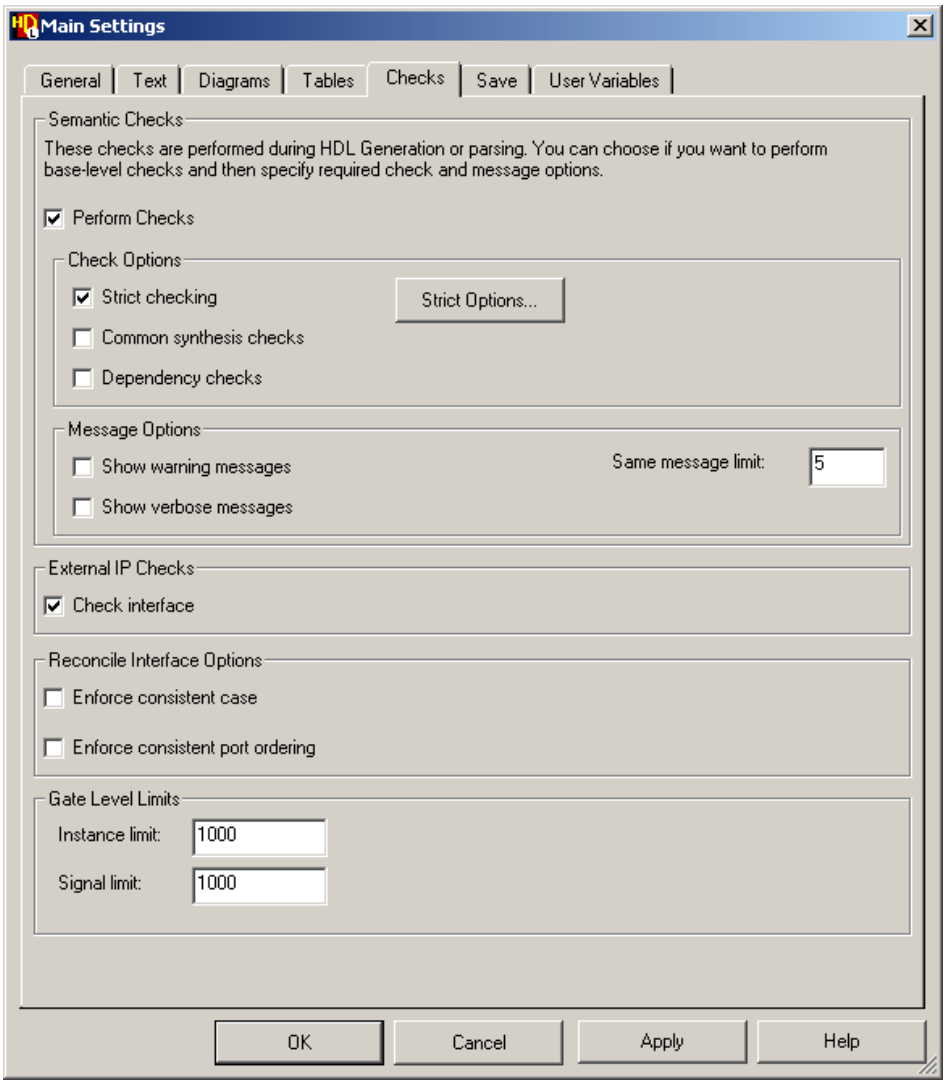
40. Select **Object properties** from the popup menu of the reset object to display the Reset Object properties page and to set the rest mode as **Asynchronous** reset and choose the signal name *reset* with a *High* level from the pulldown lists.

41. Use the  button to save the state diagram.

Set Checks for HDL Generation

You can set the level of checks performed when HDL is generated by using the **Checks** tab in the Main Settings dialog box.

42. Open the Main Settings dialog box by choosing **Main** from the **Options** menu.



43. For this tutorial, the **Strict checking** option should be selected.



The Strict Options... button can be used to display a dialog box which allows you to modify the strict checking options. The default strict check options should be used for this tutorial.

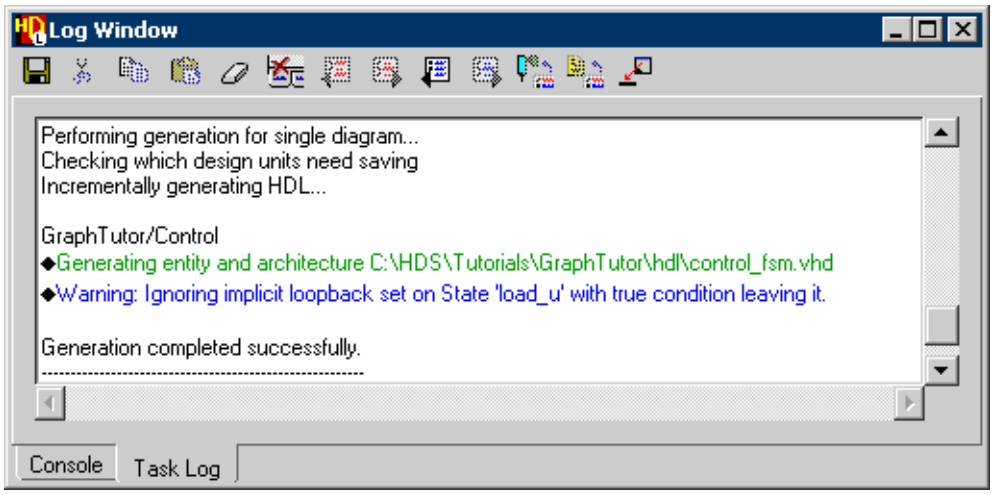
44. Use the OK button (or Apply followed by Cancel) to apply these checks when the HDL is generated and dismiss the dialog box.

Generate HDL for the State Machine

Although the *Timer* design is not yet complete, you can now generate HDL for the *Control* state machine.

45. Choose the **Run Single** option from the **Generate** cascade in the **Tasks** menu (or use the  button) in the state diagram window.

    The progress of the HDL generation is monitored in a log window which includes any errors and warnings issued during generation.





This example shows a combined file is generated for the VHDL entity and VHDL architecture body. Separate files may be generated if you have changed the preferences for VHDL file options.

If there are any errors, you can move to the next error message using the  button or the previous error using the  button.

You can display the source graphics corresponding to the error by double-clicking on the error message (or by using the  button when the error is selected). Alternatively, you can use the  button when the error is selected to view the generated HDL. If your text editor supports a line number argument, the HDL line corresponding to the error is selected automatically.

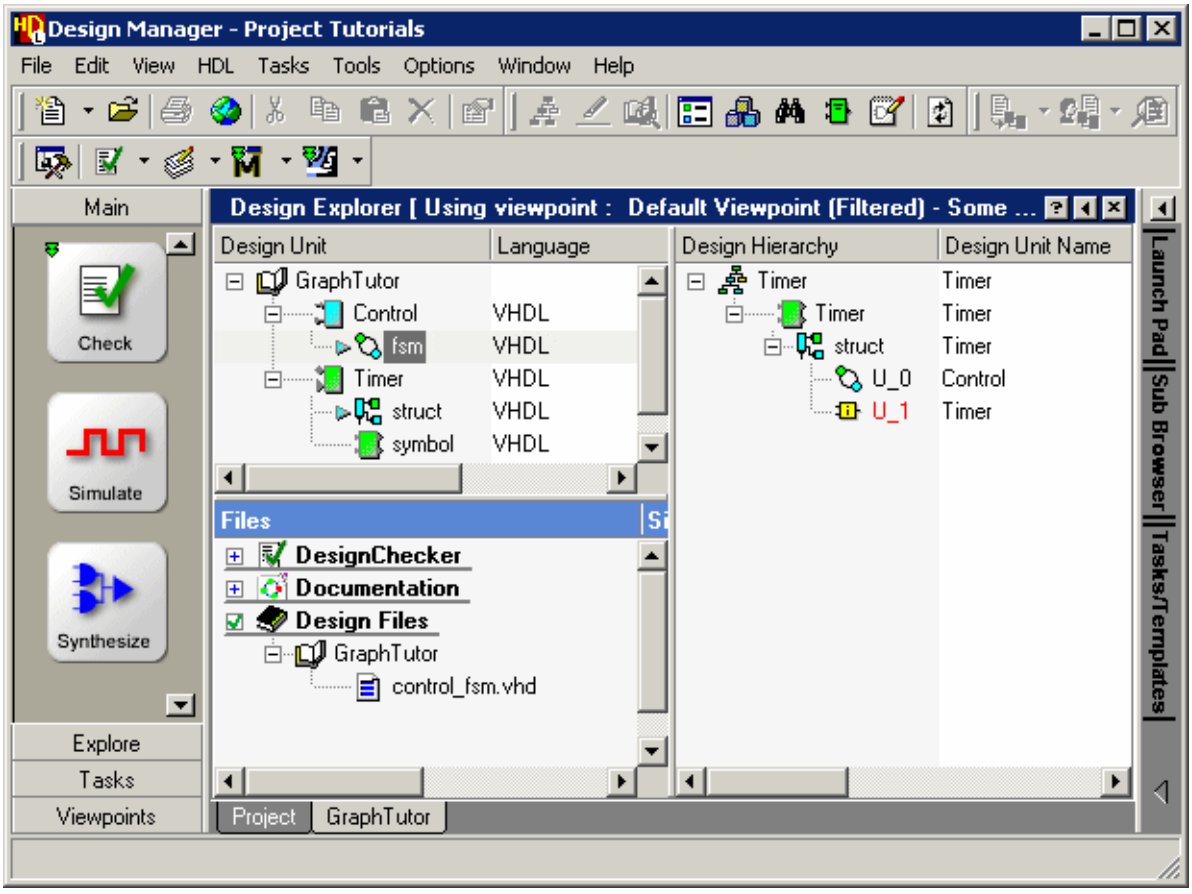
If there are no errors, you can use the toolbar buttons  and  to view the source graphics or generated HDL corresponding to any message which is marked by the ◆ icon in the log window. For example, the “Generating entity and architecture ...” message shown above.

Generated HDL files can also be opened by using the  button in any graphics editor window or when the diagram view is selected in the design explorer.

HDL Files in the Design Explorer

46. Expand the Design Files node in the Files pane of the Design Explorer window

47. Click on the ⊞ icon for the *GraphTutor* library and notice that the view is expanded to reveal the generated VHDL architecture body file for the *Control* design unit.





If you have changed your preferences to create separate VHDL entity and architecture body files, then both files are shown.

48. Open the HDL file by double-clicking on the generated file name or by choosing **Open File** from the popup menu.

If you are using the DesignPad HDL text editor, you can use the  button or the **Trace To Graphics** command from the **Document** menu in DesignPad to cross-reference from a line in the generated HDL to the corresponding graphics object.

A similar option can be set up for other user customizable text editors Refer to the “Text Editors” chapter in the *HDL Designer Series User Manual* for more information.

49. Close the text editor after you have viewed the generated HDL.

50. Use the **Close Window** option from the **File** menu to close both the parent *GraphTutor/Control/fsm [csm]* and child *GraphTutor/Control/fsm [csm/count]* views of the state machine.

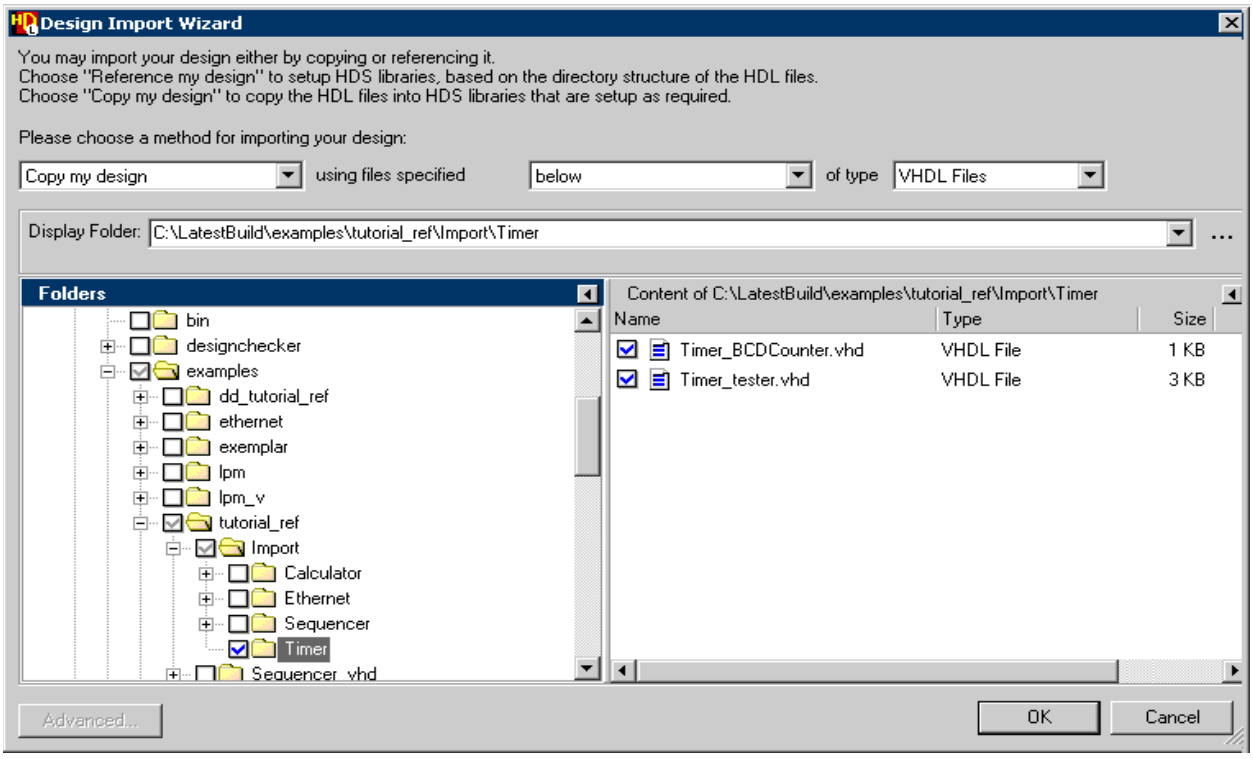
Import the BCDCounter Design Unit

You have now defined the *Control* block by drawing a graphical state machine. The *Counter* block will be defined by instantiating two instances of a HDL text component in a child block diagram.

1. Choose **Design** from the **Import** cascade of the **File** menu to display the Design Import wizard.

2. Specify the method to import your design by selecting **Copy my design** from the left most dropdown list.

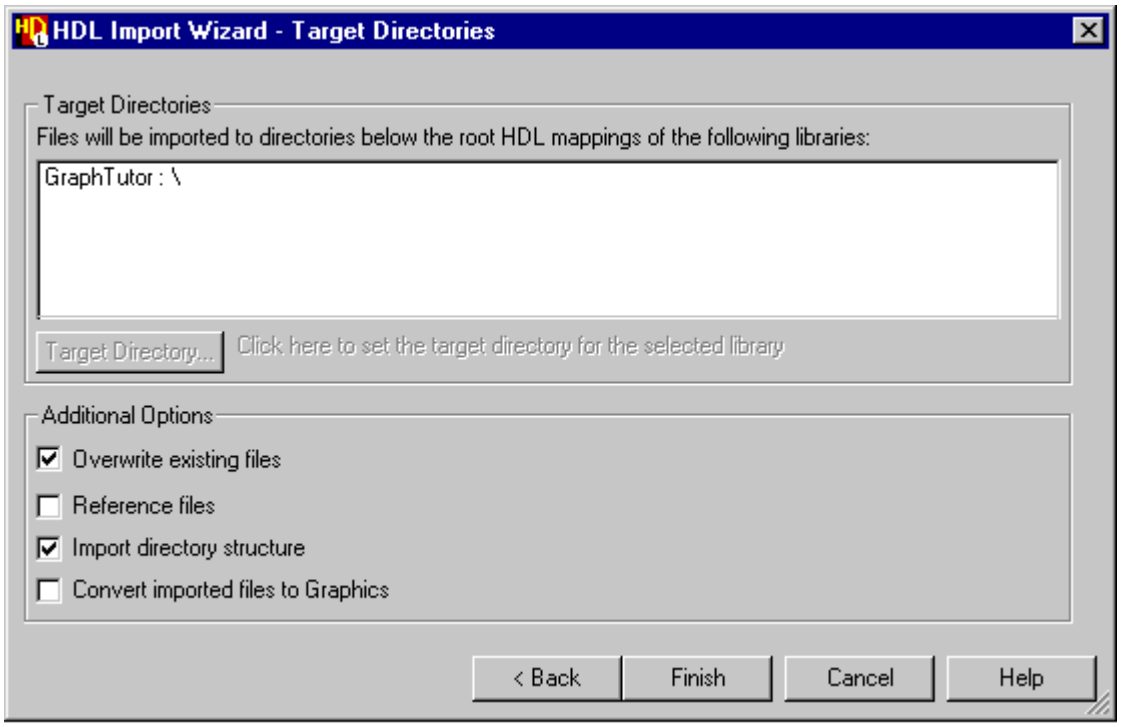
3. Choose **Below** from dropdownlist**.**.



4. Choose *VHDL Files* from the pulldown filter for **Files of type** and locate the *<install_dir>\examples\tutorial_ref\Import\Timer* installation subdirectory. Uncheck the *Timer_tester.vhd* file in the content pane.

5. Click the **OK** button on the wizard to display the Target Libraries page of the HDL Import wizard. This page allows you to select the default target library and add libraries if there are any instantiations for black box components in the source HDL code.

6. Ensure that your *GraphTutor* library is selected as the target library and click the **Next** button to display the Target Directories page. This page allows you to control the imported directory structure and set additional HDL import options:



7. Click the Finish button in this page to complete the HDL import using the default target directory and options specified in the wizard.

   The following completion message should be displayed in the HDL log window:

```
HDL Import complete

-------------------------------------------

1 file imported to 1 library

-------------------------------------------
```

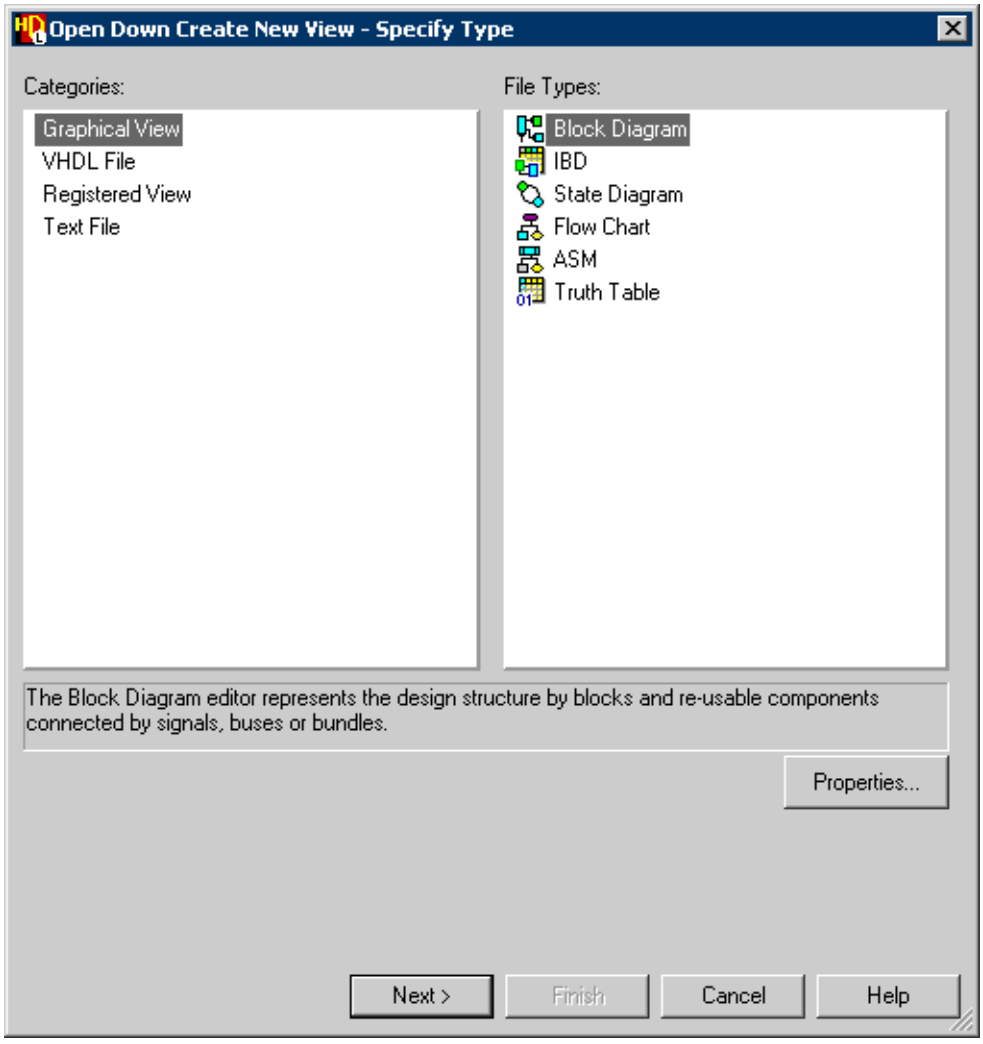
Create a Child Block Diagram

1. Display the *Timer* block diagram.



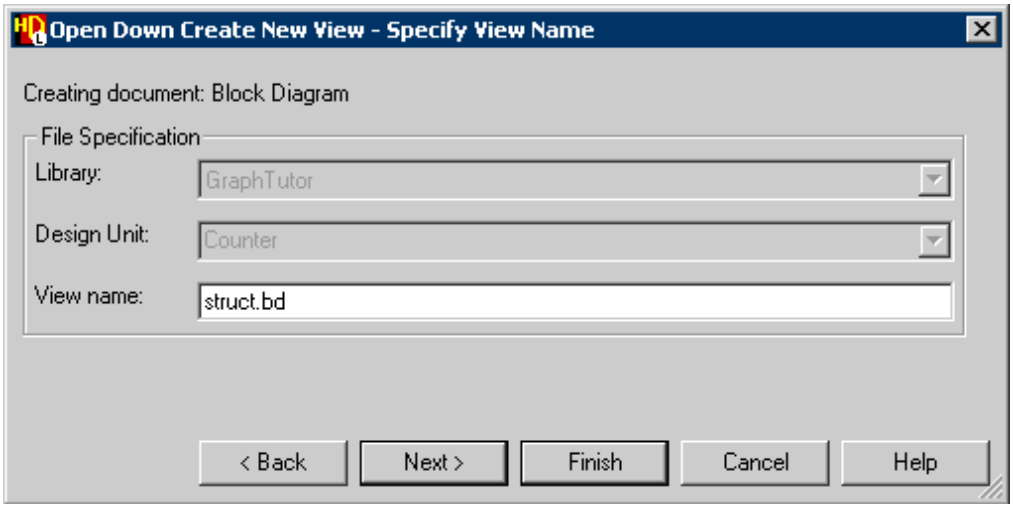
If the window is obscured, you can move it to the front by selecting the window name from the **Windows** menu list in any other window.

2. Double-click on the body of the *Counter* block (or choose **New View** from the **Open As** cascade of the popup menu) to display the Open Down Create New View dialog box.



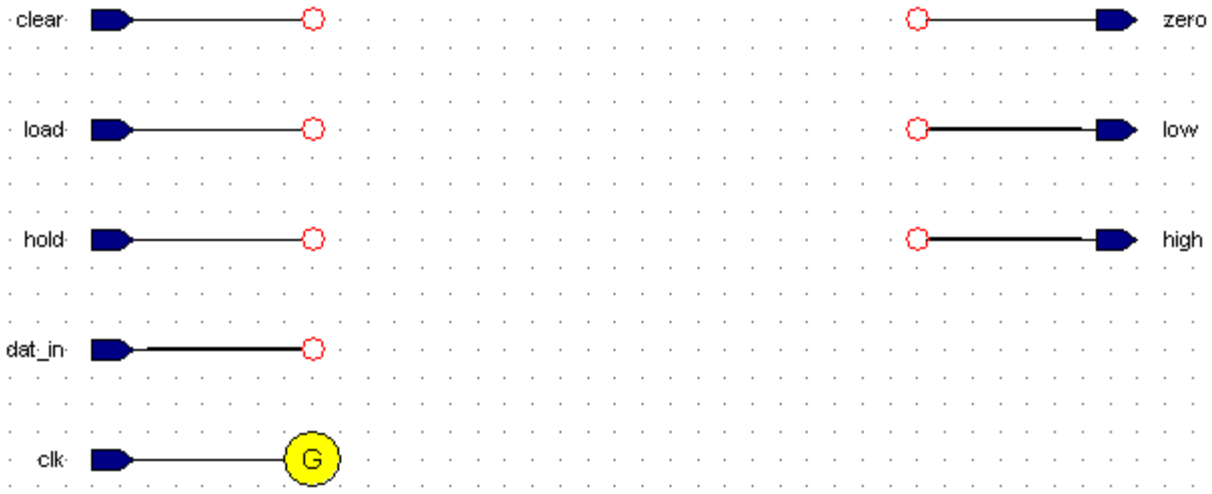
3. Select *Block Diagram* from the list of file types and use the **Next** button to display the Specify View Name page of the wizard.

4. Use the **Finish** button to confirm the wizard with the default view name *struct.bd.*

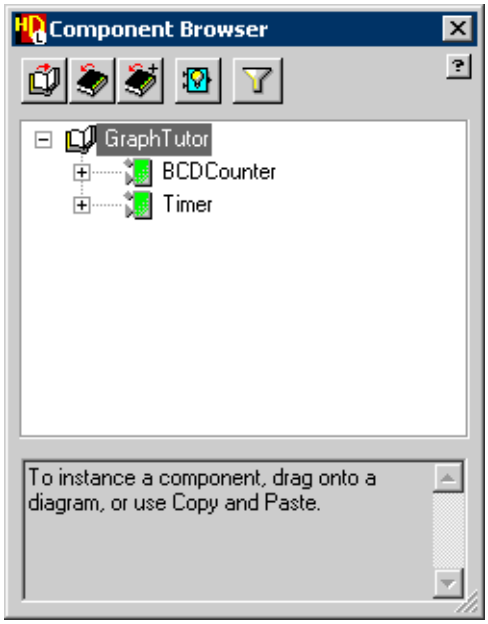


A new block diagram *GraphTutor/Counter/struct* is created as a child view of the *Counter* block. The block diagram is initialized with declarations and ports corresponding to the interface signals and buses on the parent diagram (including a global connector for the *clk* signal).

Notice how inputs are initialized on the left and outputs on the right.

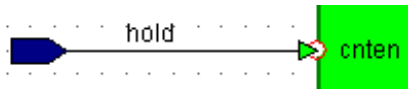


5. Use the button (or choose **Component** from the **Add** menu) to display the Component Browser.



6. Drag two instantiations (*U_0* and *U_1*) of the *BCDCounter* component from the Component Browser placing one below the other on the block diagram but allowing several grid lines for connections between them.

7. Move the *hold* signal so that it overlaps the *cnten* port on instance *U_0*.



8. Similarly, move the *low* signal over the *count* output port on instance *U_0* and the *high* signal over the *count* output port on instance *U_1*.

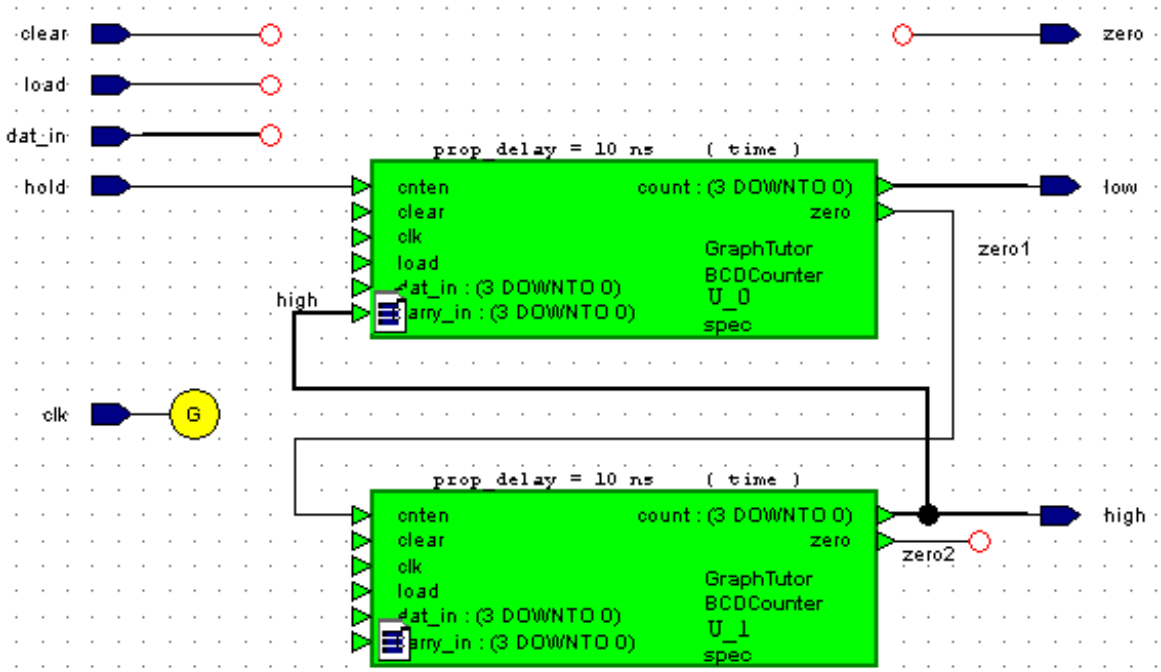
9. Select both instances and choose **Connect** from the popup menu to make the connections.

10. Use the button to connect a signal between the *zero* output port on *U_0* and the *cnten* port on *U_1*. This signal is automatically named *zero1* since the output signal *zero* already exists on the diagram.

11. Use the button to connect a bus between a point on the *high* output bus and the *carry_in* port on *U_0*.

12. Use the  button to connect a stub signal to the zero output port on U_1.

    This signal is automatically named *zero2* since the signals *zero* and *zero1* already exist on the diagram. The block diagram should now look similar to the following picture:



13. Select both instances *U_0* and *U_1 of the BCDCounter* and choose **Add Signal Stubs** from the popup menu to display the Add Signal Stubs dialog box.

14. Click the OK button to confirm that you want to add signal stubs to all port types. You are warned that the nets *clear*, *clk*, *dat_in* and *load* already exist on the diagram.

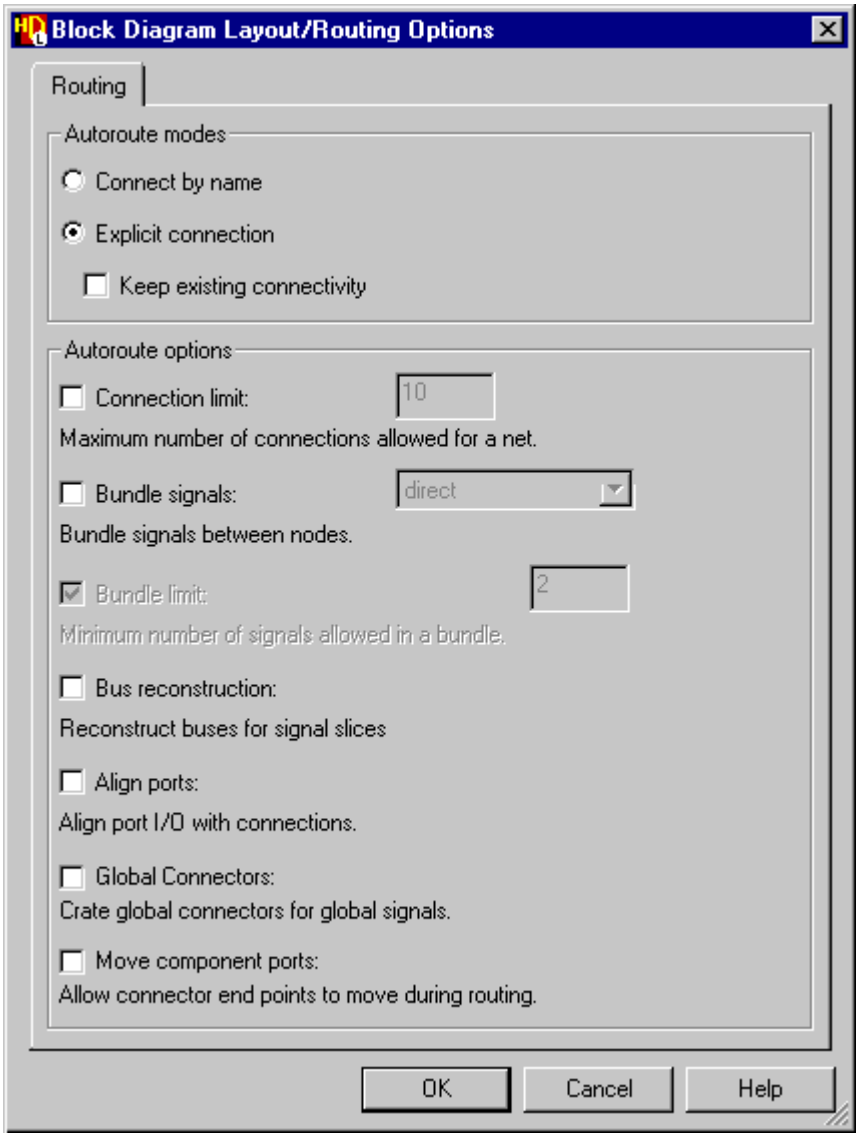


Do not set the **Create unique net names** option. This option could be used when you do not want to automatically connect nets with the same net but is not required in this tutorial.

15. Click the OK button to acknowledge the warning. The signal stubs are added when you accept the warning and the matching signals are implicitly connected by name.

It is not necessary to connect the remaining nets on the diagram although making implicit connections can make the diagram clearer. You can automatically connect the matching nets on a diagram by using commands in the **Autoroute** cascade from the **Diagram** or popup menu.

16. Check the options for automatic routing by choosing **Autoroute Options** from the **Autoroute** cascade to display the Block Diagram Layout/Routing Options dialog box.



17. Check that **Explicit connection** is set. Unset **Align ports**, **Global Connectors** and **Move component ports** and confirm the dialog box.

18. Select the *clear*, *load* and *dat_in* nets (or the ports with these names on instances *U_0* and *U_1*). Choose **Autoconnect** from the **Autoroute** cascade of the popup menu.

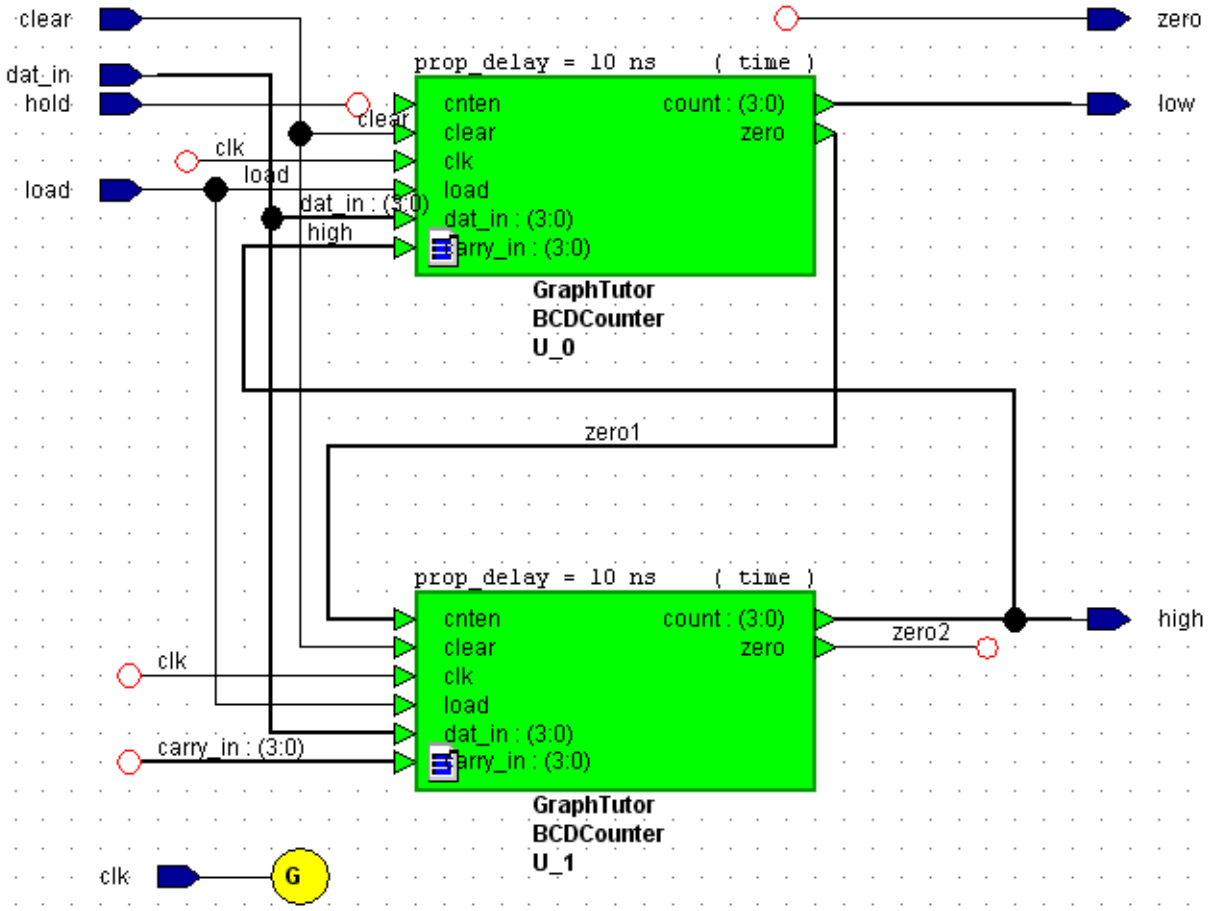
The signals are automatically connected to the matching stub signals on instances *U_0* and *U_1*.



If nothing is selected on the diagram, autoconnect attempts to re-route all connections on the diagram. Ensure that only the required signals are selected if you do not want to change all the connections on a diagram.

Note that the *clk* port (with its global connector) is connected by name to both *clk* ports on the instances.

The block diagram should now look similar to the following picture:



19. Close the Component Browser.

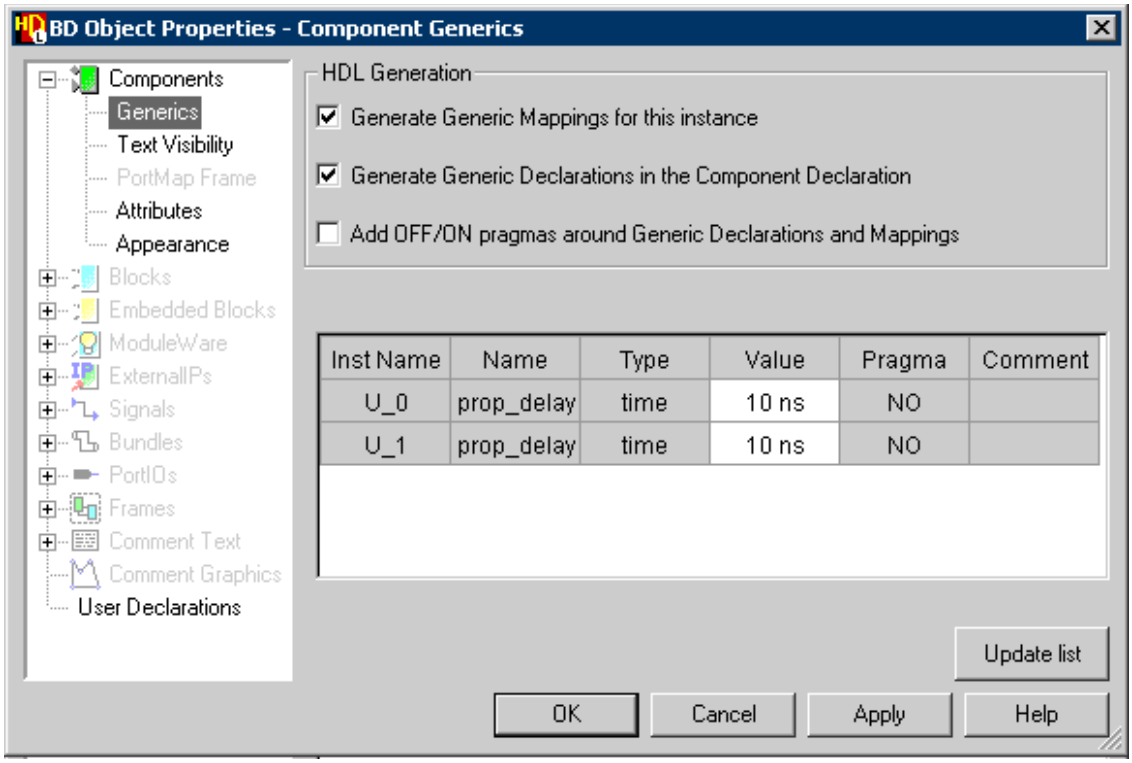
Edit the Generic Mapping

When you instantiated the *BCDCounter* components, a VHDL generic mapping for the propagation delay (*prop_delay*) was placed above the component.

The generic is declared in the symbol defining the component and can be mapped to different values for each instance of a component.

The generic mapping is a text object which can be moved independently by dragging it with the Left mouse button. For this tutorial, both values should be changed to *5 ns.*

20. Use the Shift key with the Left mouse button to select both instances of the *BCDCounter* component. Then use the button to display the Block Diagram **Generics** page of the Object Properties dialog box .



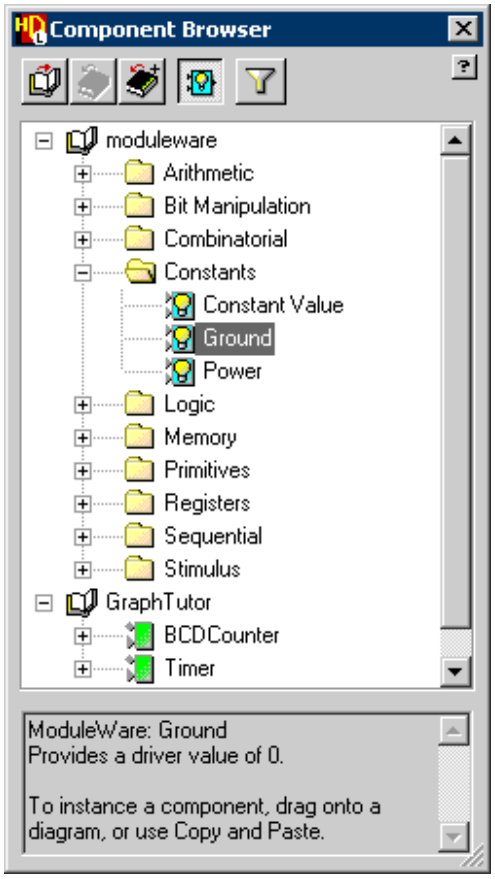
21. Select the Value entry for instance *U_0 and* overwrite it with the new value 5 ns. Repeat for the value on instance *U_1*. Then use the **OK** button to update both values on the block diagram.

Add ModuleWare Components

Although you have routed the *Counter* block diagram, several signals have been left unconnected. These will be connected by using ModuleWare components.

22. Display the Component Browser by using the button (or by choosing **ModuleWare** from the **Add** menu). The browser displays the contents of the *moduleware* library which is divided into a number of folders.

23. Click on the ⊞ icon to expand the folder for the *Constants* category:

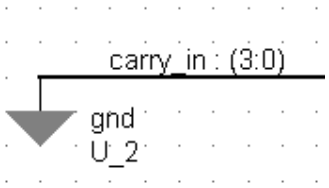


24. Use the Left mouse button to drag an instance of the *Ground* component from the Component Browser over the *Counter* block diagram and release the button to place it near the *carry_in* input to instance *U_1.*

    A *gnd* ModuleWare instance is added with a default instance name (*U_2*).

25. Use the mouse to position the stub signal on the dangling connector of the *carry_in* net over the *gnd* component. These should automatically connect. If this does not occur select the *carry_in* net and and choose **Connect** from the popup menu.

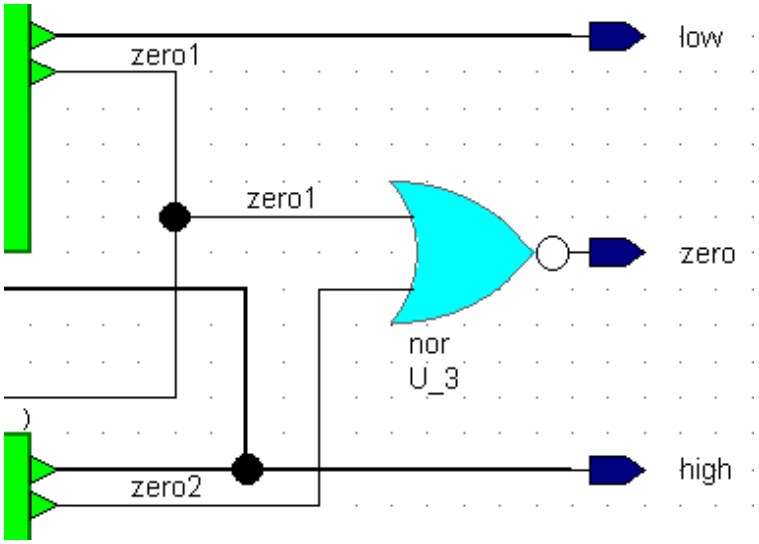
The *U_2* ModuleWare instance should now look similar to the following picture.



Notice that the *carry_in* net is a 4-bit bus. The ModuleWare port width is automatically set to match the width of the connected net.

26. Repeat this procedure to add an instance of the *N Input NOR* component from the *Logic* folder of the *moduleware* library near the *zero1* output from instance *U_0* on your block diagram.

27. Connect this *nor* instance to the *zero1*, *zero2* and *zero* nets as shown in the following picture:

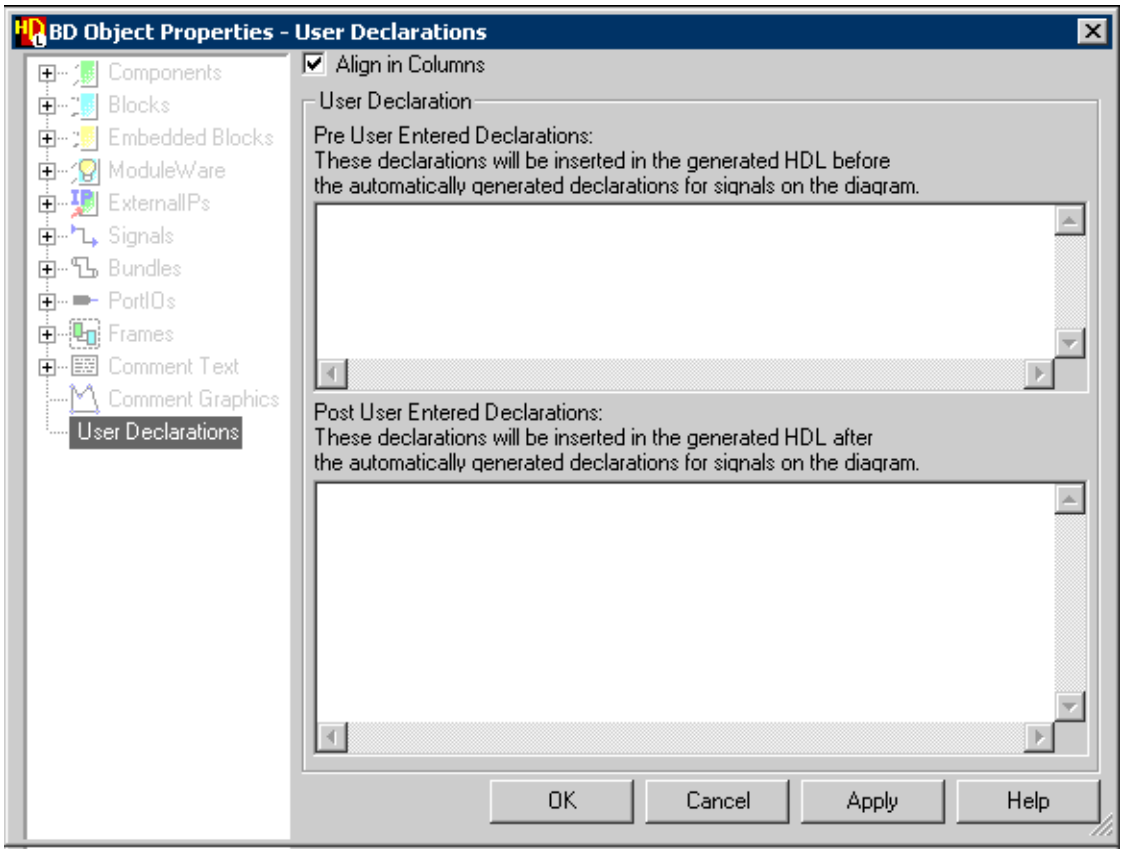


Add a User Declaration

28. Double-click over the Declarations label on the diagram (or use the  button) to display the **Declarations** tab of the block diagram Object Properties dialog box.

29. Define the *gnd* signal to be a 4-bit binary zero vector constant by entering the following declaration in the Pre User Entered Declarations section of the dialog box:

```
CONSTANT gnd : std_logic_vector(3 DOWNTO 0) := "0000";
```





The syntax is automatically checked and any errors reported on entry.

You can enter any valid VHDL statements which must be terminated by a semicolon (;) character. Line breaks and spaces can be used for clarity and will be preserved on the diagram.

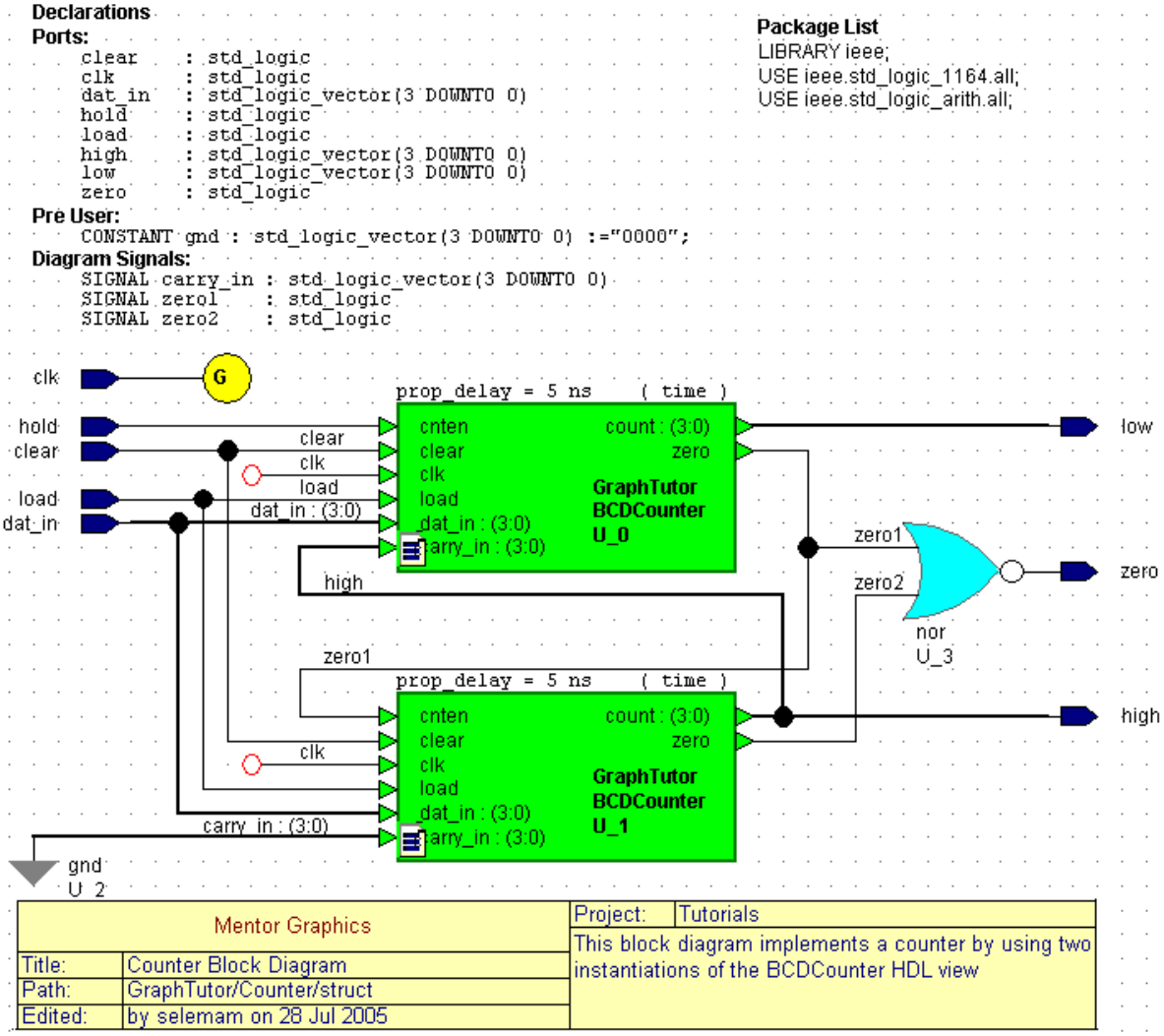
30. Click on the the  button in the Block Diagram Object Properties dialog box to add the new user declaration on the diagram.

    The pre user entered declarations are added to the declarations list on the diagram between the port and signal declarations.

31. Complete the block diagram by editing the title block.

    You may also want to drag objects or groups of objects to re-arrange the diagram within the page boundaries for your default printer.

    For example, the following picture shows the *Counter* block diagram re-arranged for portrait page orientation.



32. Use the  button to save the block diagram.

Create a Truth Table

You have now defined a state diagram view for the *Control* block and a block diagram view for the *Counter* block.

The *Timer* design is completed by using a truth table to describe the *DtoB* embedded block.

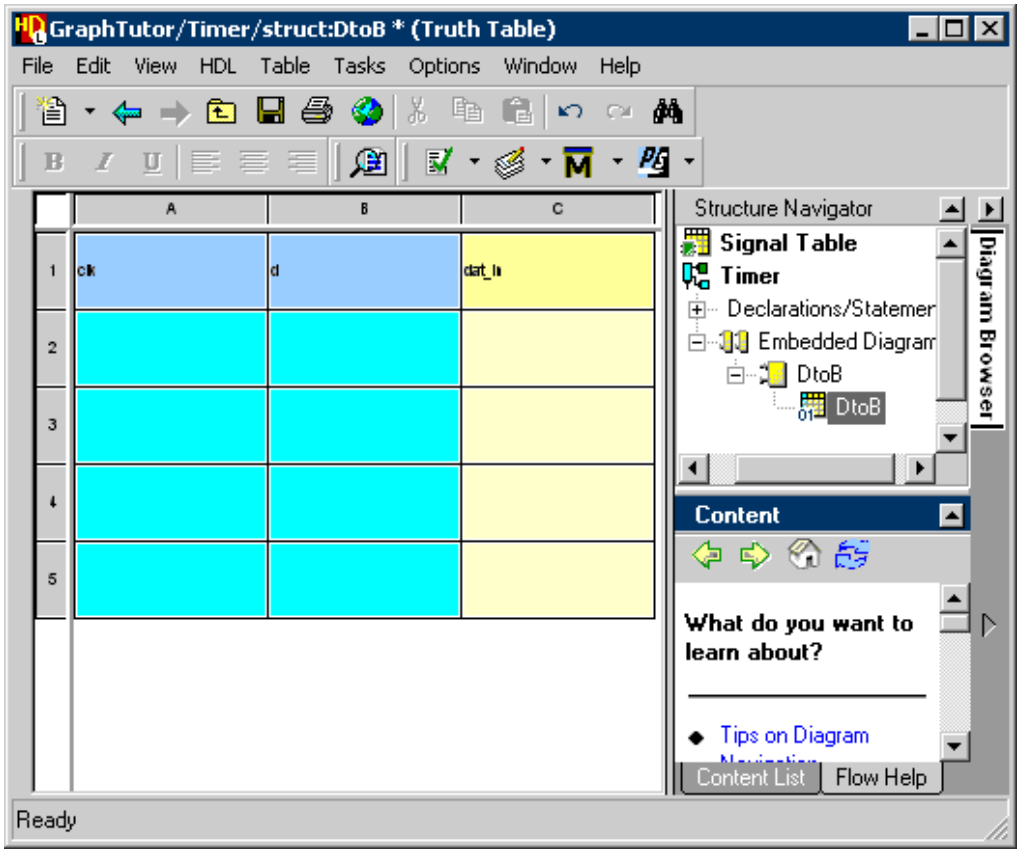
1. Use the  button (or choose the **Open Up** option from the **File** menu) in the *Counter* block diagram to display its parent diagram (the *Timer* block diagram).

2. Double-click over the *DtoB* embedded block to display the Create Embedded View dialog box.

3. Select **Truth Table** and confirm the dialog box.

   A new truth table (*GraphTutor/Timer/struct:DtoB*) embedded view is created and displayed in a tab window.

   The table is initialized as a matrix of four rows with two input columns (for the *clk* and *d* inputs) and one output column (for the *dat_in* output).



Edit the Truth Table

4. Click the mouse in either of the blue input columns (A or B) and use the **Add Column** option from the popup menu to add another eight input columns (C to J).

5. Click the mouse in one of the blue input rows (rows 2, 3, 4 or 5) and use the **Add Row** option from the popup menu to add another seven rows (6 to 12) to the truth table.



You can add multiple rows or columns by selecting more than one cell to add the corresponding number of rows or columns.

6. Rename the input columns to represent each bit (*d(9)* down to *d(0)*) of the *d* input bus and enter the value *'1'* in one row for each column. (The *clk* signal is not used and can be renamed.)

7. Enter the following values in the *dat_in* output column:

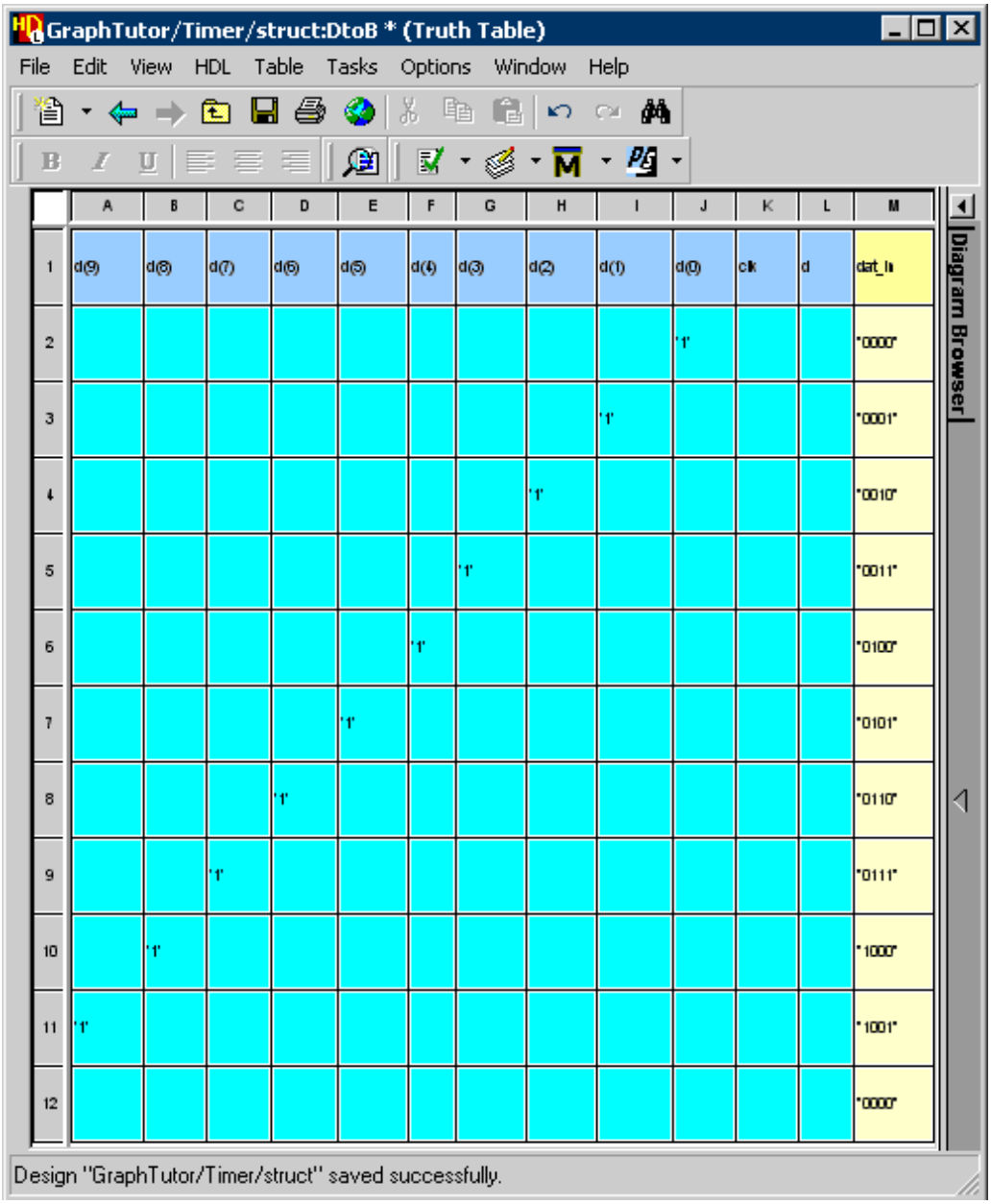
   "0000"
   "0001"
   "0010"
   "0011"
   "0100"
   "0101"
   "0110"
   "0111"
   "1000"
   "1001"
   "0000"

   To add text to a cell, click in the cell, enter the text and click in any other cell to confirm the entry. You can edit the text in a cell by double-clicking the cell and overwriting its contents or click again to edit individual text characters.

   You can also use the  button to copy and the  button to paste text between cells and re-size the rows or columns by dragging the control sash in the non-scrolling area.

   You can format text in single or multiple selected cells by using the  (bold),  (italic) or  (underline) buttons and align text using the  (align center),  (align left) or  (align right) buttons.

The completed truth table should look similar to the picture below.



	A	B	C	D	E	F	G	H	I	J	K	L	M
1	d(9)	d(8)	d(7)	d(6)	d(5)	d(4)	d(3)	d(2)	d(1)	d(0)	clk	d	dat_b
2										'1'			"0000"
3									'1'				"0001"
4								'1'					"0010"
5							'1'						"0011"
6						'1'							"0100"
7					'1'								"0101"
8				'1'									"0110"
9			'1'										"0111"
10		'1'											"1000"
11	'1'												"1001"
12													"0000"



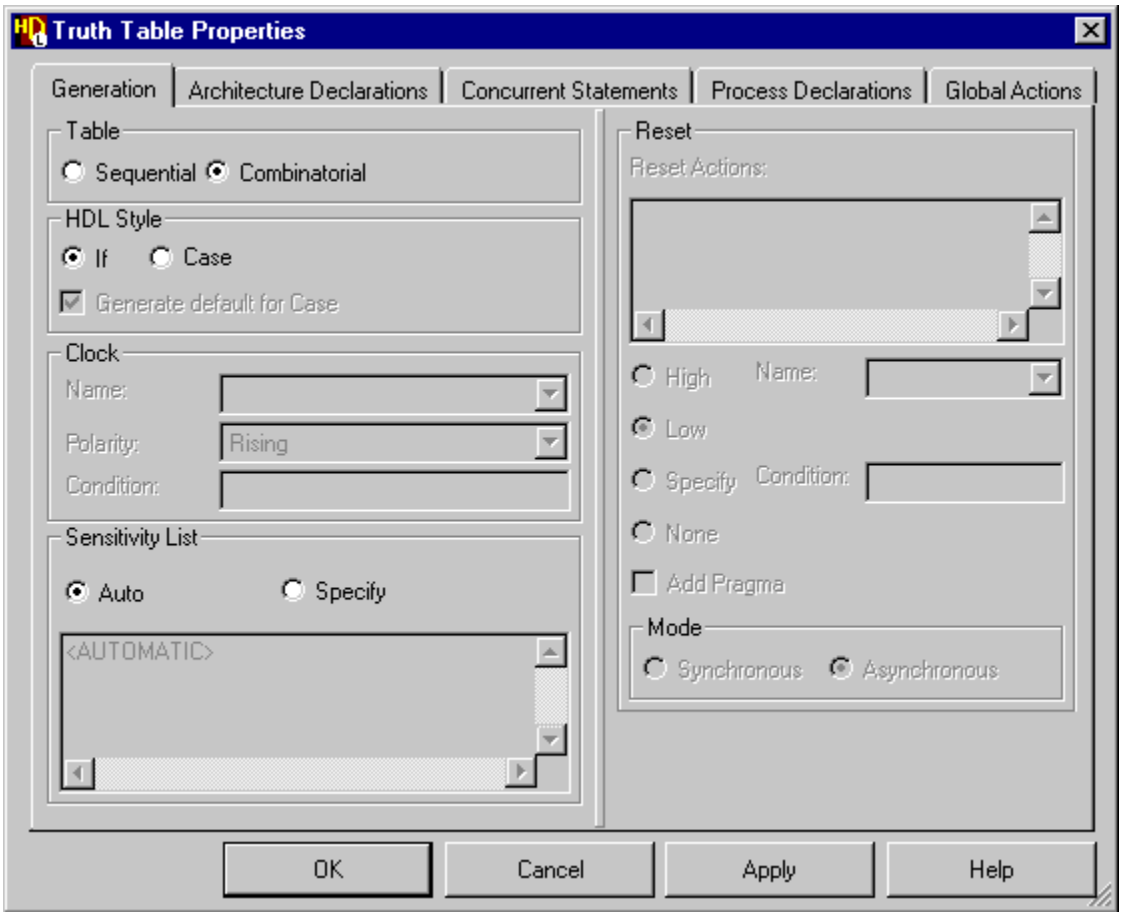
The last (empty) row represents an ELSE condition and assigns the output expression a known value when none of the input expressions are true.

Set Truth Table Properties

8. Choose **Truth Table Properties** from the **Table** menu to display the Truth Table Properties dialog box. Select the **Generation** tab to display the HDL generation characteristics for the truth table.

Notice that the sensitivity list is set to **Auto**. When this option is set, the list is automatically created by HDL generation.

Check that the default options for **Combinatorial** and **If** style are set and use the OK button to confirm the dialog box.



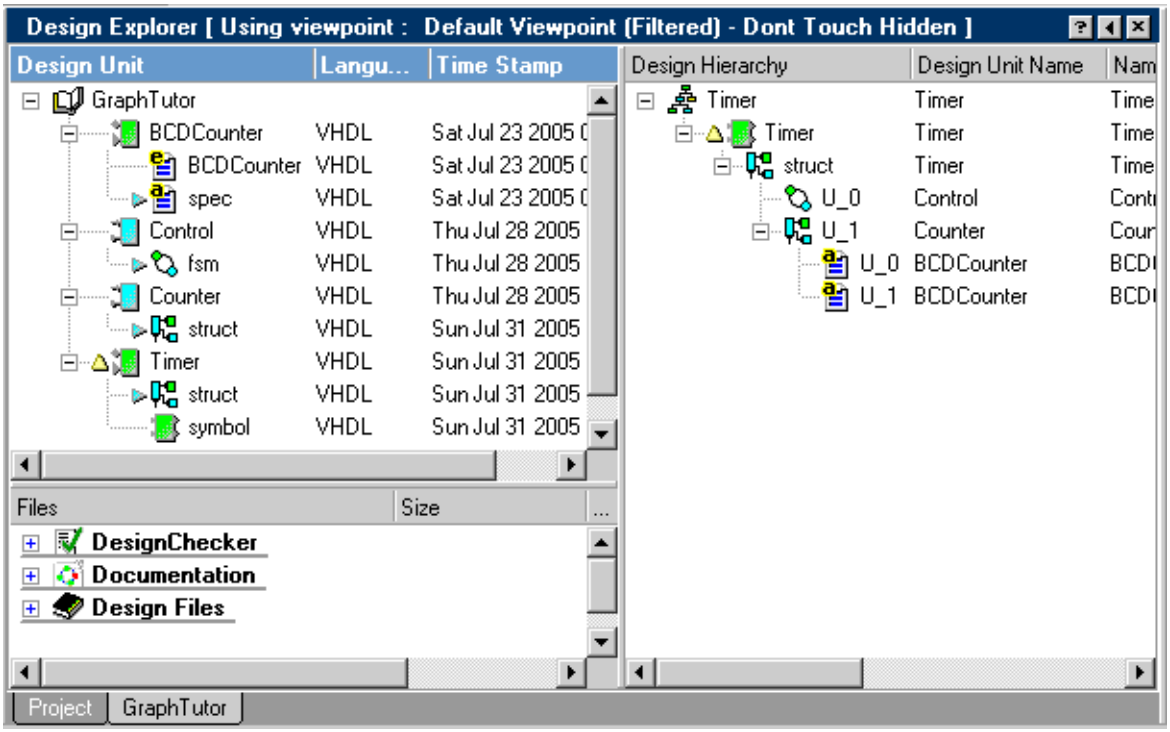
The tabs for Architecture Declarations, Concurrent Statements, Process Declarations and Global Actions can be left empty for this tutorial.

9. Save the truth table. This also saves the *Timer* block diagram since the truth table is an embedded view within the same design unit.

Browse the Timer Design

1. Return to the design manager and examine the completed design in the *Design Units* and *Design Hierarchy* panes using the ⊞ icons to expand each design unit.

The fully expanded design should look similar to the following picture:



The *BCDCounter* component contains a VHDL entity and architecture (*spec*) defined in the *Timer_BCDCounter.vhd* file.

The expanded *Control* block design unit shows that it contains a state machine (*fsm*).

The *Counter* block contains a block diagram (*struct*).

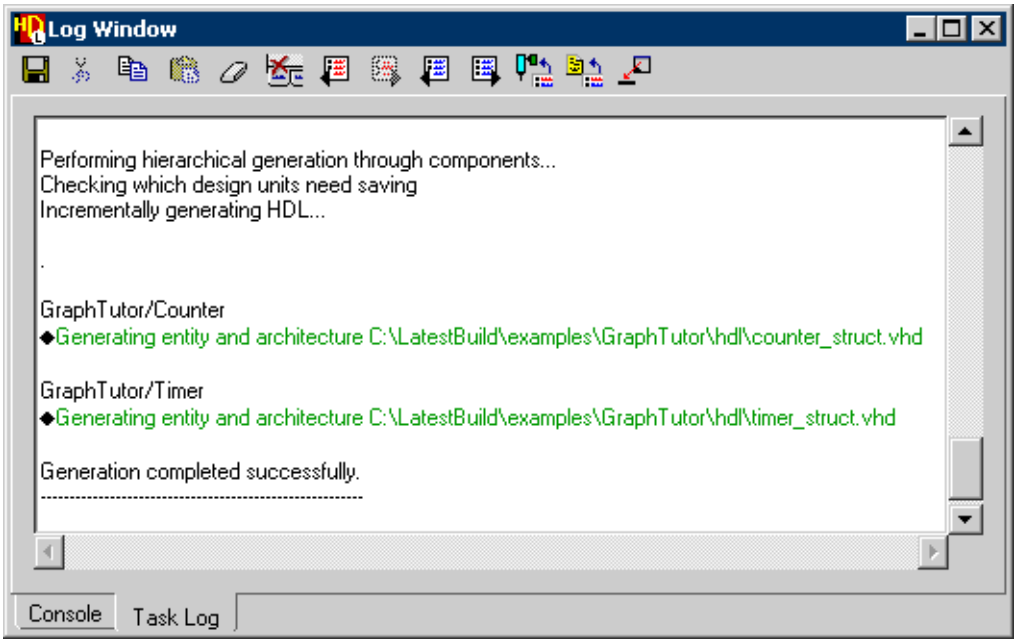
The Timer component contains a block diagram (*struct*) and a symbol describing its external interface.

Notice that you can expand the *Timer* component in the *Design Hierarchy* pane to display the complete hierarchy including the *BCDCounter* and the *Control* state machine.

Generate HDL for the Hierarchy

2. Select the *Timer* design unit in the *Design Units* or *Design Hierarchy* pane of the design explorer window and choose the  button (or choose Run **Through Components** from the **Generate** cascade of the **Tasks** menu).

   The progress of the HDL generation is monitored in a log window which includes any errors and warnings issued during generation.





This example shows a combined file generated for the VHDL entity and VHDL architecture body. Separate files may be generated if you have changed the preferences for VHDL file options.

The *Control* design unit generated earlier in this tutorial is not regenerated unless it has been changed. HDL is generated for all other design units. You are prompted to save any design units which have not been saved.

If there are any errors, you can move to the next error message using the  button or the previous error using the  button.

You can display the source graphics corresponding to the error by double-clicking on the error message (or using the  button when the error message is selected).

Alternatively, you can use the  button when the error is selected to view the generated HDL. If your text editor supports a line number argument, the HDL line corresponding to the error is selected automatically.

The embedded truth table view is generated as part of the *Timer* design unit.

If any syntax errors are issued for the truth table, the corresponding cells are highlighted. For example, if HDL generation encounters an expression which is not defined in the interface to the diagram, the corresponding input cells are highlighted.

If there are no errors, you can use the toolbar buttons to view the Graphics or HDL corresponding to the "*Generating*..." message in the HDL Log window.

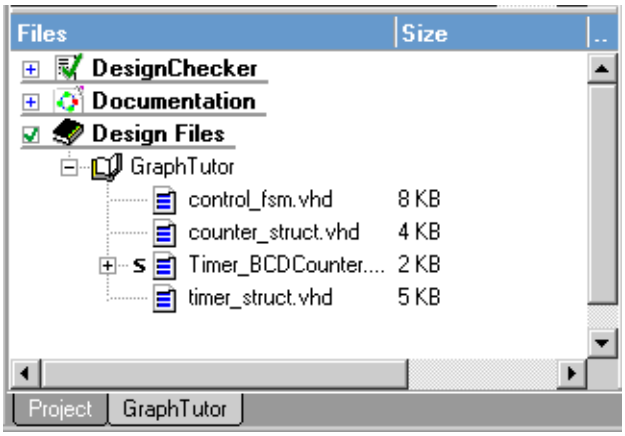


You can cross-reference to the graphics or HDL from any message which is marked by the ◆ icon in the log window.

Any files in the generated HDL directory can also be opened from the HDL browser.

3. Expand the Design Files node in the Files pane of the Design Explorer window.

4. Click on the ⊞ icon for the *GraphTutor* library and notice that the view is expanded to reveal the source file for the *BCDCounter* and the generated files for the *Control*, *Counter* and *Timer* design units.



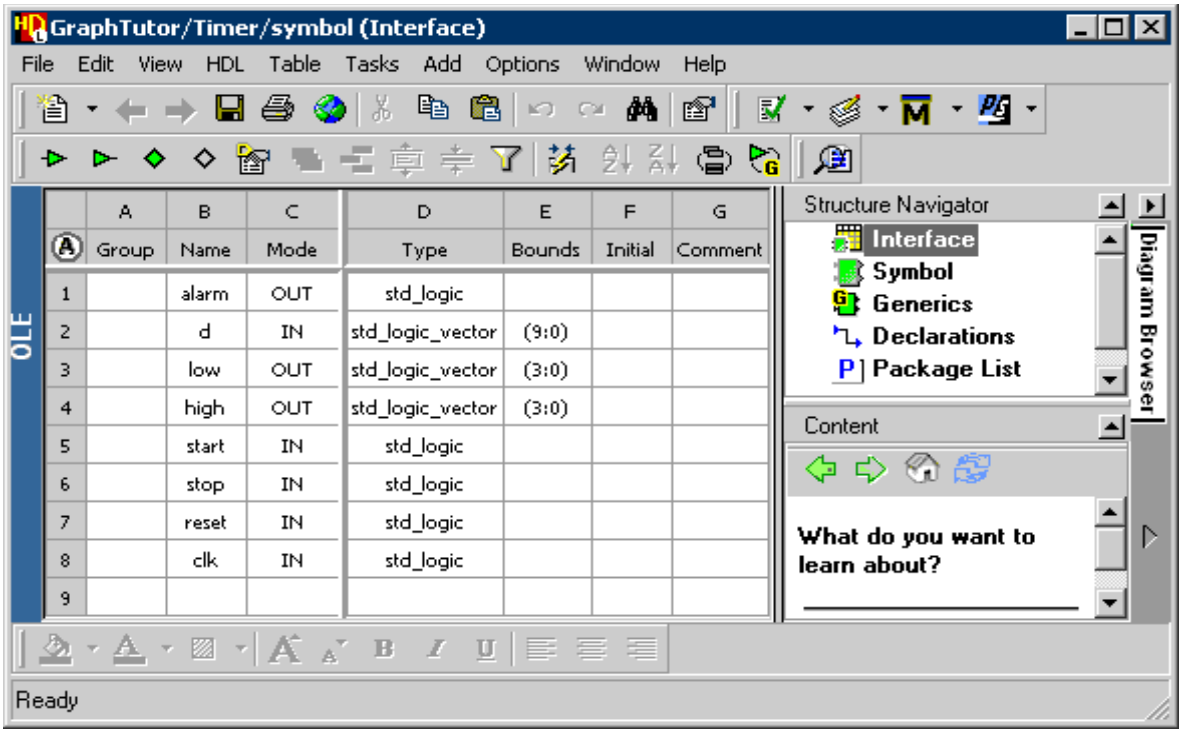
You can open any of these files by double-clicking on the generated file name or by choosing **Open** from the popup menu.



You can also view the generated HDL files for the active diagram by using the  button in any editor window or when the diagram view is selected in the design explorer.

Edit the Timer Symbol

1. Use the button (or choose the **Interface** option from the **Open** cascade of the **File** menu) in the *Timer* block diagram.



An interface editor window is opened which shows the external interface for the *Timer* design. Click on the symbol icon in the Structure Navigator pane to display the symbol editor. This symbol is used when the *Timer* design is instantiated as a component in a higher level design.

Ports representing the interface connections are shown in default locations but you can select and drag the ports to any location around the body of the symbol. You may also want to resize the symbol body.



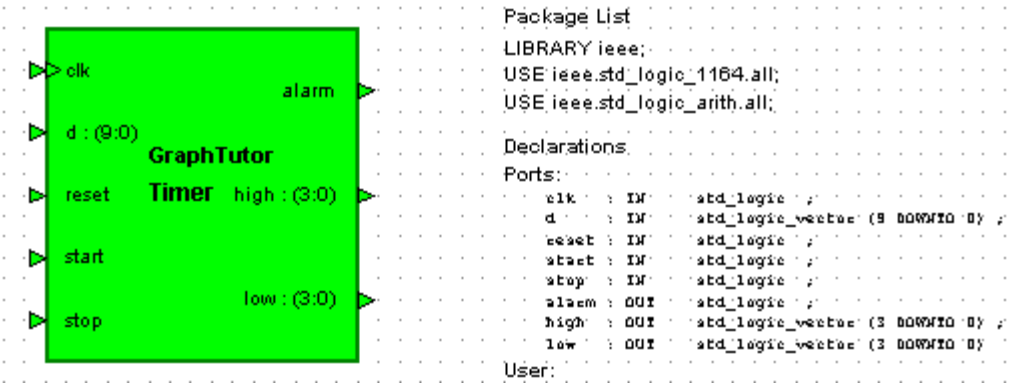
You can redistribute the ports evenly by selecting the symbol body and choosing **Equidistant Ports** from the popup menu.

The full port declarations are shown as a list in the symbol editor and are not shown on the individual ports.

Refer to “Changing the Visibility of Symbol Port Properties” in the *HDL Designer Series User Manual* for information about setting preferences to control whether type and bounds information is displayed or the properties are visible for an individual port.

2. Select the *clk* port and choose **On** from the **Clock** cascade to apply an edge triggered clock indicator to the port.

   Your edited symbol should look similar to the picture below:



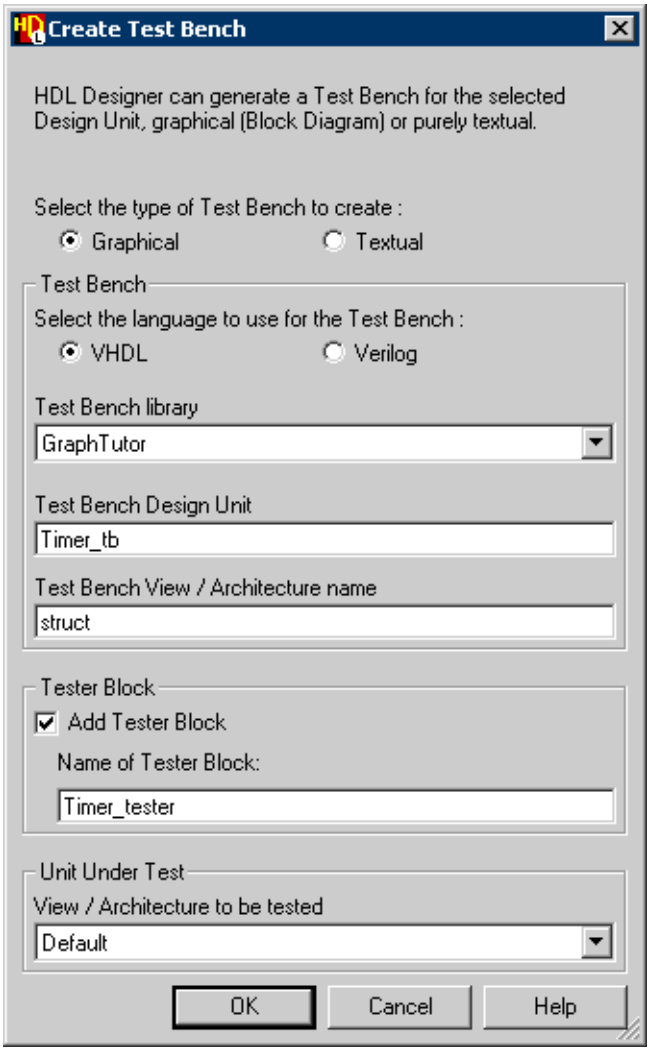
3. Save and close the symbol view.

Create a Test Bench

1. Select the *Timer* design unit in the *Design Units* view of the design explorer and choose **Test Bench** from the **New** cascade in the **File** menu to display the Create Test Bench dialog box.

   Notice that the dialog box defaults to the *GraphTutor* library, with the test bench name derived by adding *_tb* to the *Timer* design unit and the default tester block by adding *_tester*.

2. Confirm the dialog box by clicking the OK button.

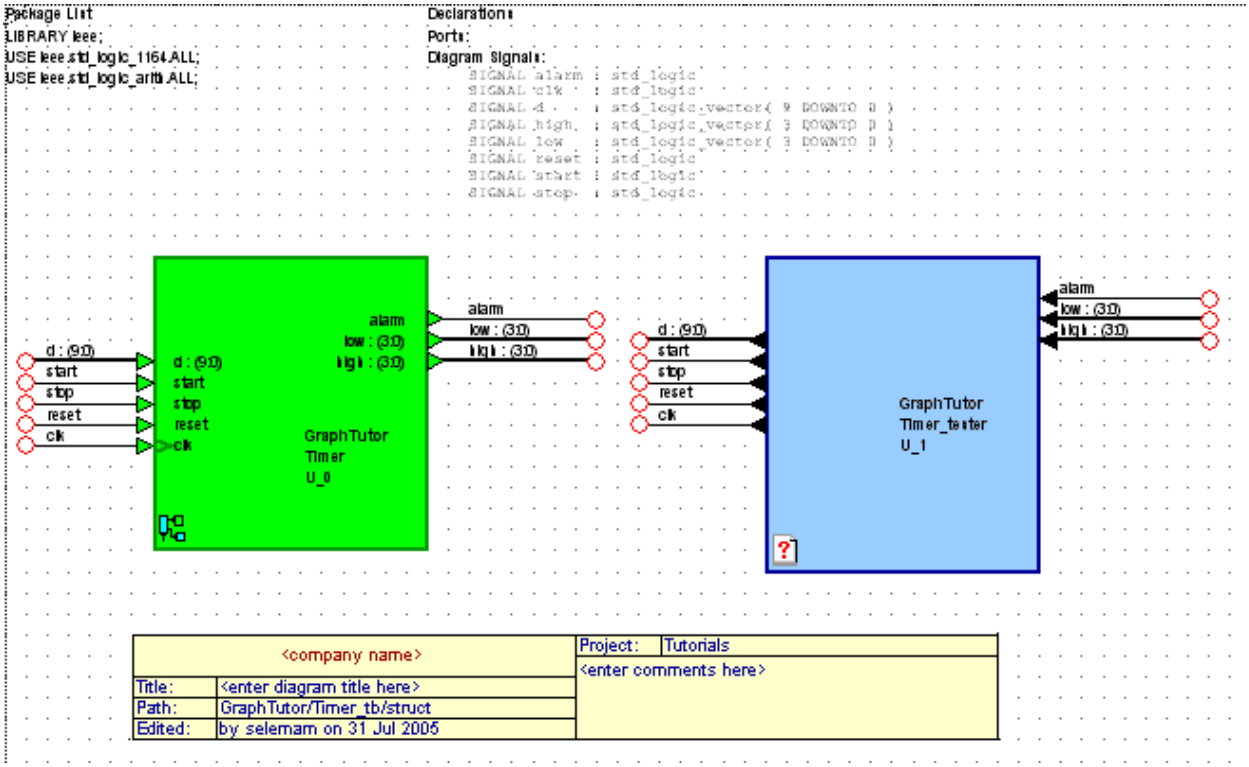


   Notice that a *Timer_tb* design unit is added to the *Design Units* displayed in the design explorer window.

3. Click on the ⊞ icon for the *Timer_tb* design unit in the design explorer to expand the view and reveal that a symbol and block diagram view (with the default name *struct*) have been created.

4. Double click on the *struct* view to open the block diagram.

Notice that the *Timer* design has been instantiated as a component with stub signals connected to the input and output ports. The adjacent *Timer_tester* block has corresponding output and input ports which are implicitly connected by name to the stub signals on the component.



All the signals and buses are declared as diagram signals since a test bench typically has no external ports.

The test bench block diagram should look similar to the following picture.

It is not necessary to explicitly connect signals and buses between each port on the component and the tester block as these are implicitly connected by name.

Import the Tester Design Unit

The *Timer_tester* block should provide test stimulus to the *Timer* design unit and monitor its output.

For this tutorial, the *Timer_tester* design unit can be imported from the *examples* installation subdirectory using a similar procedure to that used to "Import the BCDCounter Design Unit" on page 1-44.

The *Timer_tester* design unit can optionally be created as a flow chart. Refer to the alternative procedures for Creating a VHDL Flow Chart provided in Chapter 2.

1. Choose **Design** from the **Import** cascade of the **File** menu to display the Design Import wizard.

2. Specify the method to import your design by selecting **Copy my design** from the left most dropdown list.

3. Choose **Below** from the second dropdownlist and choose *VHDL Files* from the pulldown filter for **Files of type** and locate the *<install_dir>\examples\tutorial_ref\Import\Timer* installation subdirectory. Uncheck the *Timer_BCDCounter.vhd* file in the content pane.

4. Click the **OK** button on the wizard to display the Target Libraries page of the HDL Import wizard.

5. Ensure that your *GraphTutor* library is selected as the target library and click the **Next** button to display the Target Directories page. This page allows you to control the imported directory structure and set additional HDL import options:**..**

6. Use the **Next** button to page through the HDL Import wizard and use the **Finish** button on the last page to import the *Timer_tester* design unit as a HDL text view.

   The following completion message should be displayed in the HDL log window:

```
HDL Import complete
-------------------------------------------

1 file imported to 1 library

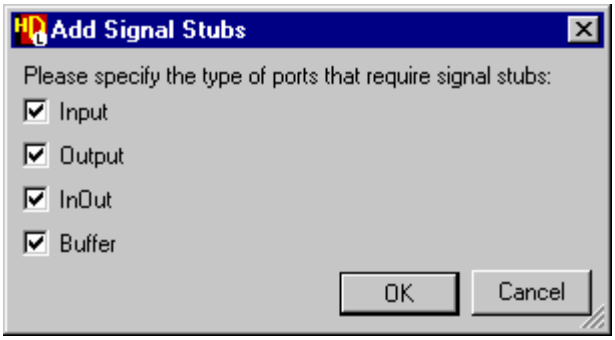
-------------------------------------------
```

Instantiate the Imported Tester

7. Re-open the *Timer_tb* block diagram if it is not still open. Select the *Timer_tester* block including all the connected signal stubs and delete them using the Del key.

8. Use the button to display the Component Browser. Select the *GraphTutor* library and instantiate the imported *Timer_tester* component in the block diagram.

9. Select the *Timer_tester* component and choose **Add Signal Stubs** from the popup menu to display the Add Signal Stubs dialog box:



10. Click the OK button to confirm that you want to add signal stubs to all port types. You are warned that the nets *alarm*, *clk*, *d, high, low, reset, start_in* and *stop* already exist on the diagram.



11. Click the OK button to acknowledge the warning. The signal stubs are added when you accept the warning and the matching signals are implicitly connected by name.

12. Complete the test bench by editing the title and comments in the title block and using the  button to add a panel around the components. This panel can be used to set the diagram view when you simulate your test bench later in this tutorial.

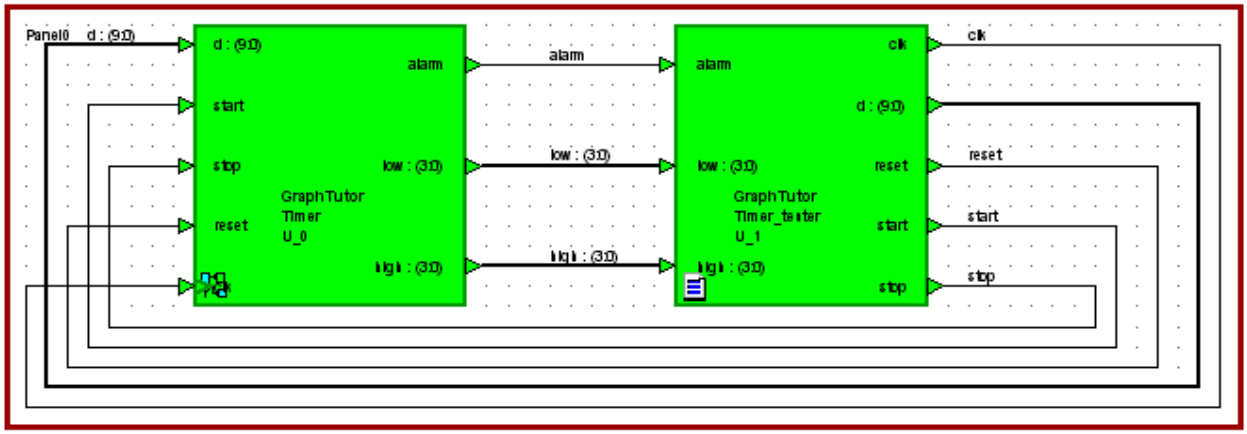
    The matching nets on the two components are implicitly connected by name. However, you may like to connect them explicitly by selecting the *Timer* and *Timer_tester* blocks and choosing **Autoconnect** from the **Autoroute** cascade of the **Diagram** menu.



Using this command with nothing selected should automatically route all nets on the diagram. If any nets are selected, then only the matching nets in the selection are routed.

It may also be necessary to move some of the component ports in order to avoid cross-overs.

The following example shows a test bench which has been explicitly connected:



13. Use the  button to save your work.

Generate HDL for the Test Bench

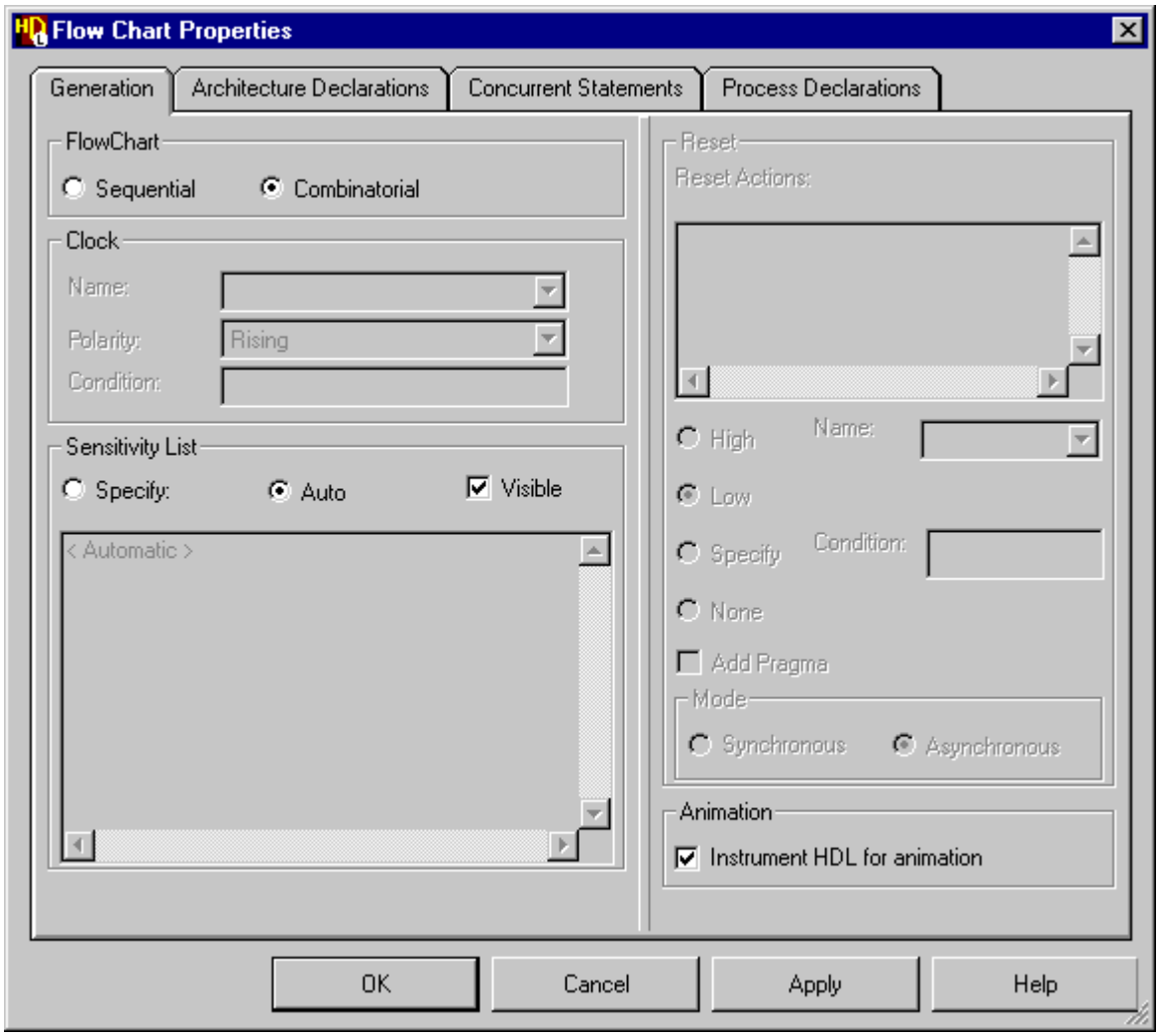
If the *Timer_tester* component is described by a flow chart follow steps 14 on page 1-71 to 17 on page 1-72. If it is described by a block diagram, proceed to step 18 on page 72.

14. Double-click on its instance in the test bench to open the view and choose **Flow Chart Properties** from the **Diagram** menu to display the Flow Chart Properties dialog box.



If you have recovered the *Timer_tester* component as a HDL text view, all properties for the view are defined in the HDL text and do not need to be set using a dialog box. You can proceed directly to step 18 on page 1-72

15. Choose the **Generation** tab and check that the **Combinatorial** option is set.



16. If a Model*Sim* or NC-Sim simulator is available, set the **Instrument HDL for animation** option. This option adds additional code to the generated HDL which is used for flow chart animation later in this tutorial.



The additional animation code is surrounded by translation control pragmas which ensure that it is ignored by downstream synthesis tools.

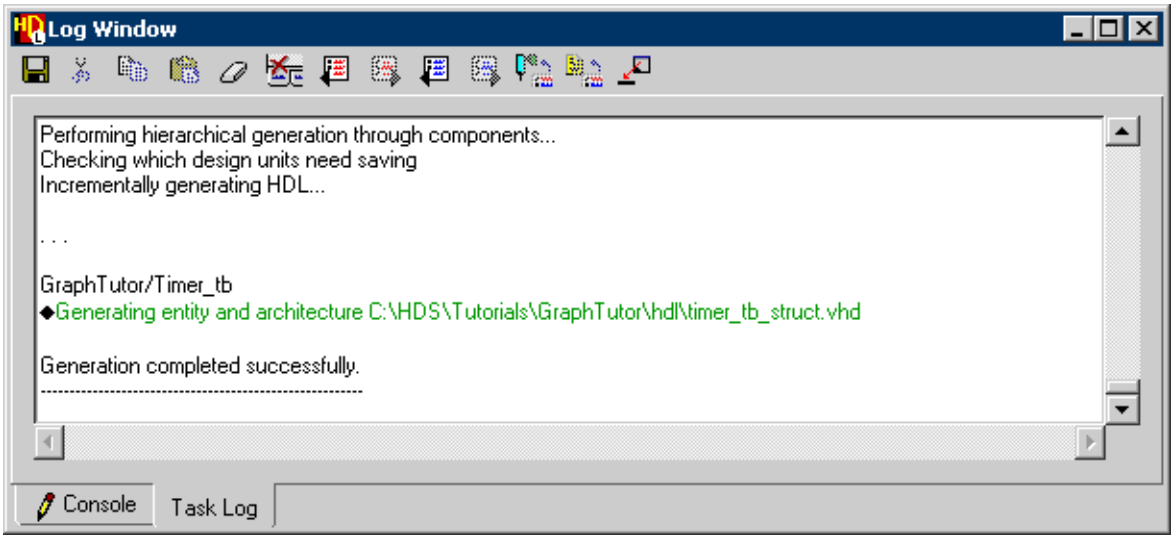
17. Set the **Auto** option for the Sensitivity List and use the OK button to confirm the dialog box.

18. Select the *Timer_tb* design unit in the design explorer and choose the  button (or choose **Run Through Components** from the **Generate** cascade of the **Tasks** menu).

    All views to be generated must have been saved and if you have not saved the test bench you are prompted whether to continue. Confirm the dialog box by clicking the Yes button.

    VHDL is generated for the test bench and tester (if it is described by a flow chart) views but not the *Timer* design hierarchy since these views were generated earlier in this tutorial and do not need to be regenerated unless they have been modified.

The progress of the HDL generation is monitored in a log window which includes any errors and warnings issued during generation.



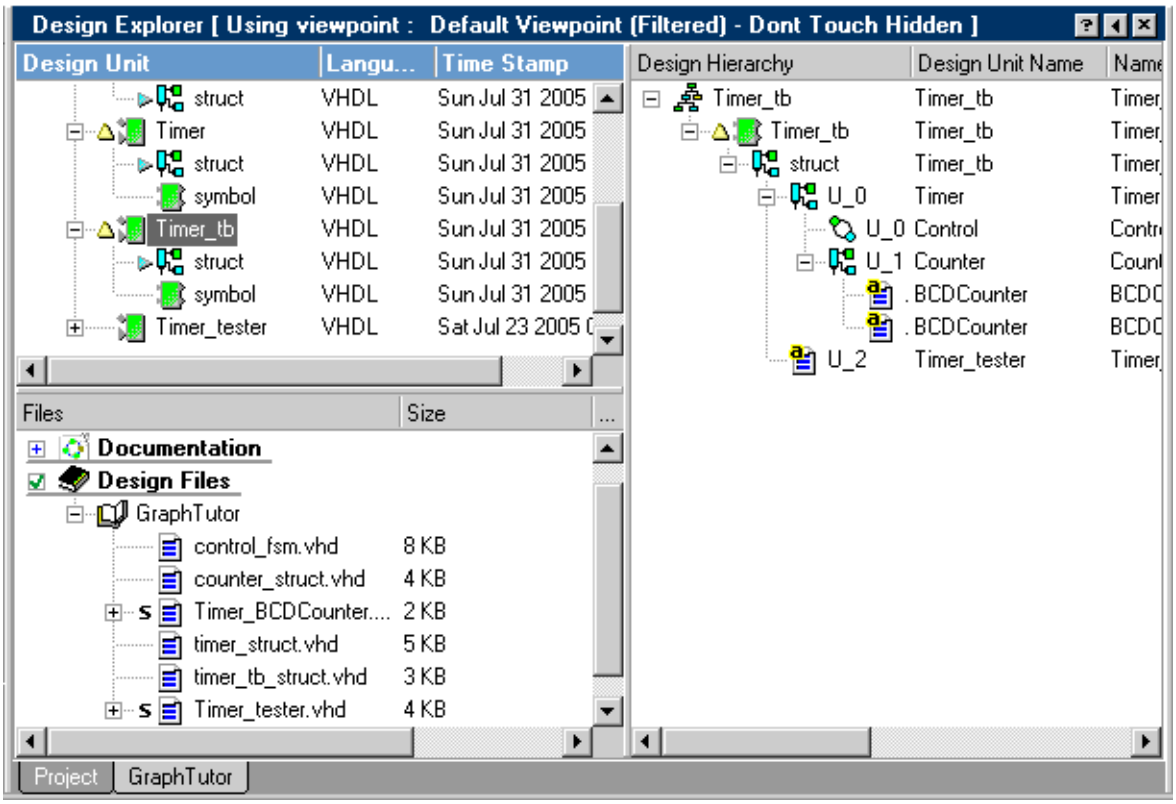
If there are any errors, you can display the corresponding graphics by double-clicking on the error message as described in the section "Generate HDL for the Hierarchy" on page 1-62.

Browse the Completed Design

1. In the design manager ensure you are using the design units view.Select the *Timer_tb* design unit and choose *Show Hierarchy* from the popup menu to display the *Design Hierarchy* pane of the design explorer.

2. Select the *Timer_tb* design unit in the *Design Hierarchy* pane and choose **Expand All** from the popup menu or use the ⊞ icons to expand the full hierarchy which also includes the hierarchy of the *Timer* design.

3. The *Files* design explorer pane should now show the *Timer_BCDCounter.vhd* and *Timer_tester.vhd* source files plus the generated HDL files for *control_fsm.vhd*, *counter_struct.vhd*, *timer_struct.vhd* and *timer_tb_struct.vhd*.



You can use the Logical Objects pane to explore the design in detail. Refer to the *HDL Designer Series User Manual* for more information.

Setup the Downstream Tools

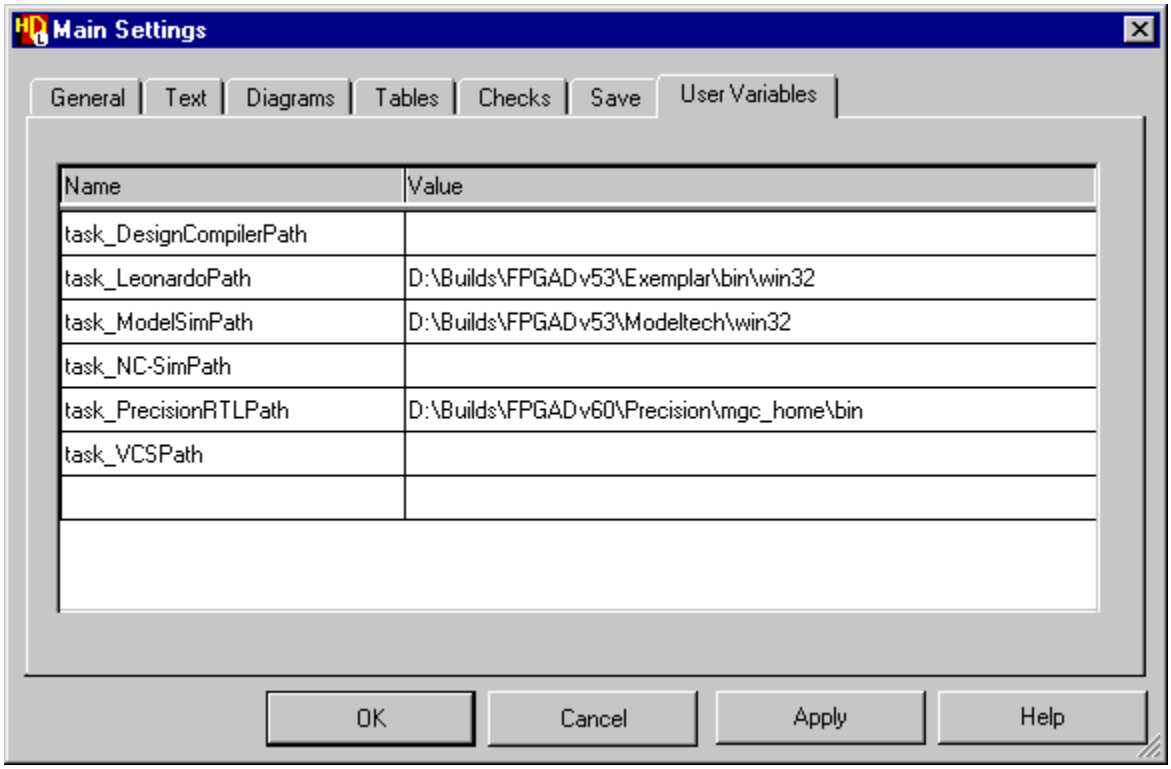
For this tutorial, it is assumed that Model*Sim* simulator and LeonardoSpectrum synthesis tools are available.

You can alternatively prepare data for use with other downstream tools.



If the HDL Designer Series tool is installed as part of the FPGA Advantage product, the Model*Sim* simulator and LeonardoSpectrum synthesis tools will already have been set up automatically.

1. Choose **Main** from the **Options** menu to display the Main Settings dialog box and choose the **User Variables** tab:



   User variables are provided for the Design Compiler, LeonardoSpectrum, ModelSim, NC-Sim, Precision Synthesis and VCS (or VCSi) tools.

2. Enter the pathname to the executables for any of the downstream tools which are available on your system and confirm the dialog box.

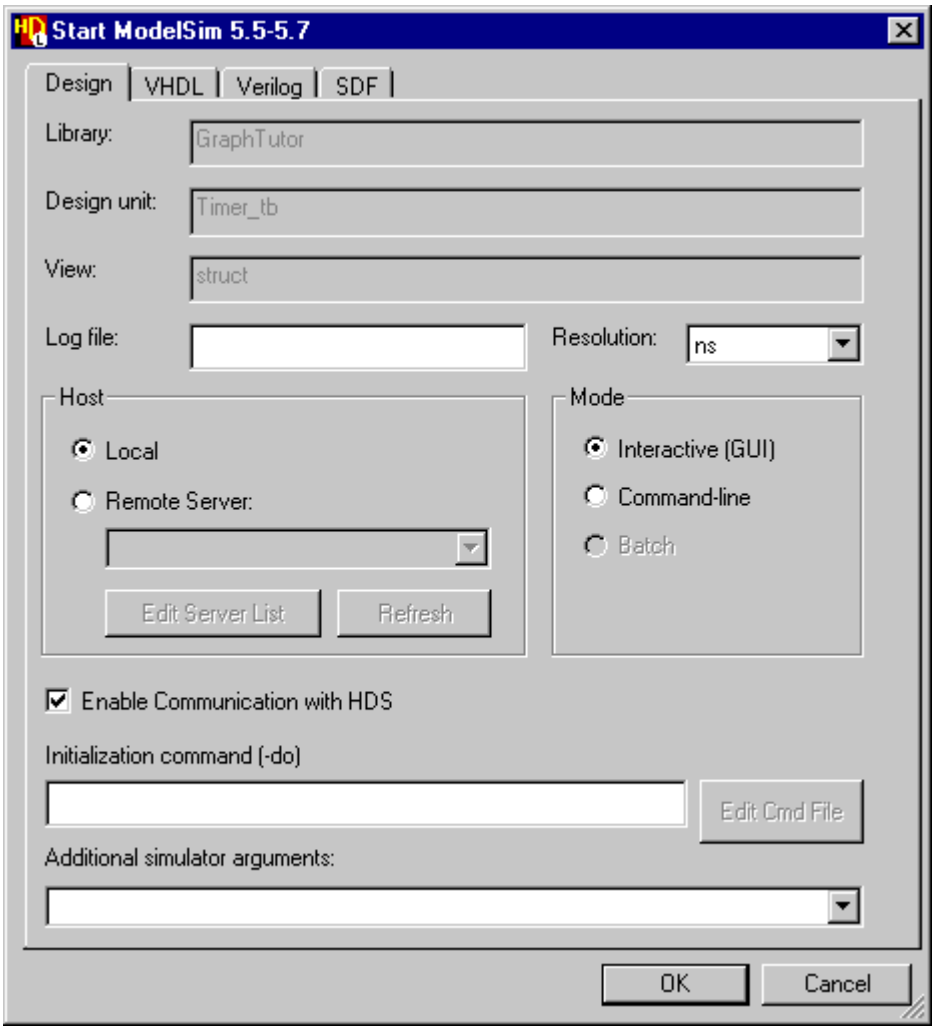
Run the ModelSim Flow

1. Select the *Timer_tb* design unit in the design explorer and use the  button to run the ModelSim flow through components.

   HDL is regenerated for any modified graphical views and the entire design is compiled. The progress of the compilation is shown in the HDL Log Window.

   If there are any errors, you can display the corresponding source diagram by double-clicking on the error message as described in the section "Generate HDL for the Hierarchy" on page 1-62.

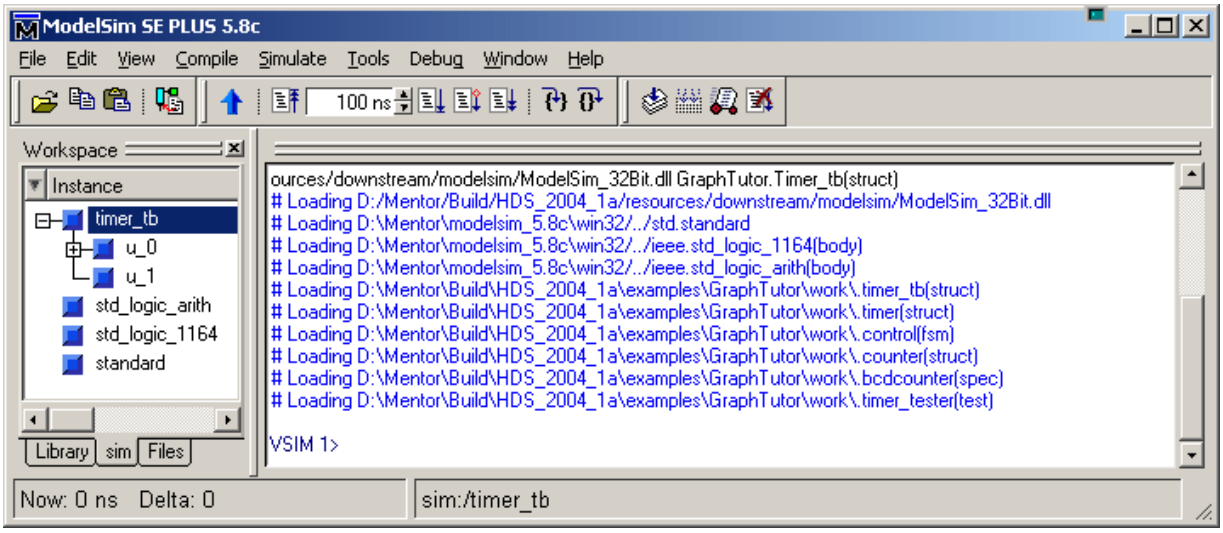
   If there are no errors, the Start ModelSim dialog box is displayed:



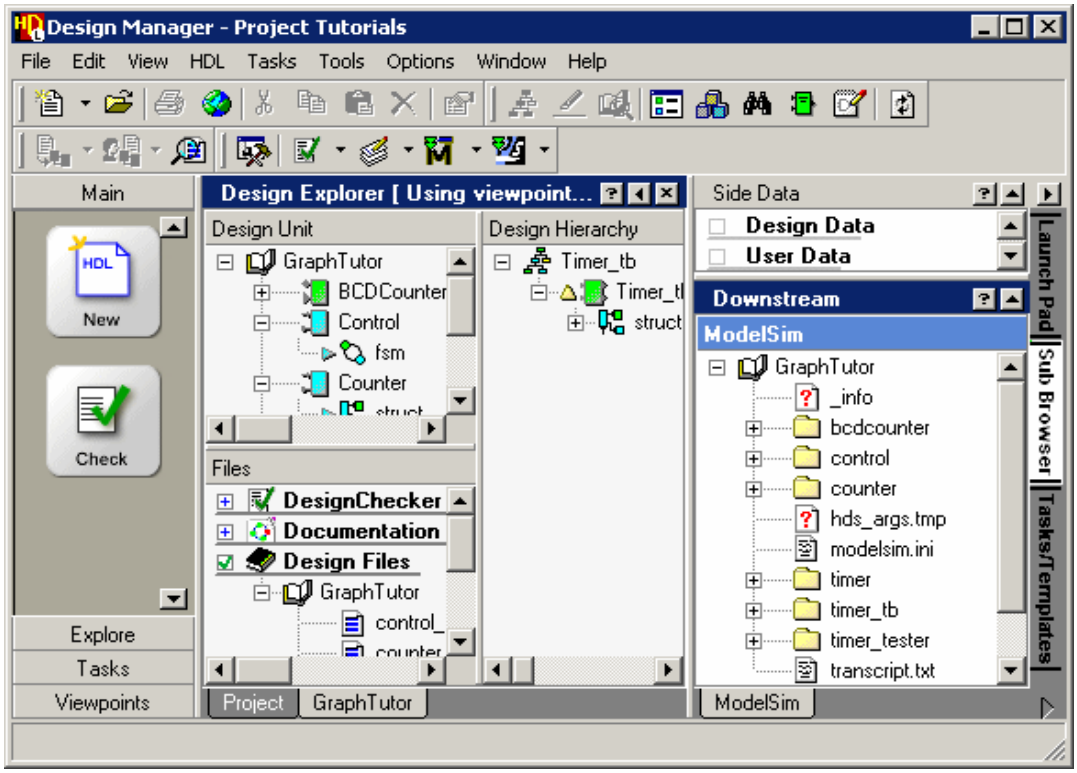
2. Accept the default resolution and check that the **Interactive (GUI)** and **Enable Communication with HDS** options are selected The other entry fields can be left in their default state.

3. Use the OK button to confirm the Invoke Settings dialog box.

   The simulator is invoked and issues a number of loading messages..



Note that library mapping is automatically created for your downstream data and the compiled objects can be displayed in the downstream browser by selecting **Side data/Downstream** from the **SubWindows** cascade of the design explorer **View** menu and choosing the ModelSim tab:



Setup the Simulator Windows

An additional Simulation toolbar is available in the block diagram, flow chart and state diagram when a simulator is invoked to support cross-probing between the simulator and source design objects in the HDL Designer Series tool.

This toolbar is normally displayed automatically at the bottom of the diagram window but can be undocked, moved, docked or hidden in the same way as the other toolbars.

For example, the following picture shows a toolbar that has been undocked from the editor window and re-arranged as two rows of tool buttons:



4. Display the *Timer_tb* block diagram and choose **View Panel** from the **View** menu to display the panel you added earlier in this tutorial.



If you added more than one panel, a dialog box is displayed for you to choose the panel to display.

5. Select the signals *alarm*, *clk*, *d*, *reset*, *start*, *stop, high* and *low*. Use the  button from the Simulation toolbar or choose **Add Wave** from the **Display** cascade in the **Simulation** menu and notice that the simulator Wave window is automatically opened with these signals loaded.

6. Use the  button from the Simulation toolbar or choose **Add List** from the **Display** cascade in the **Simulation** menu to add the selected signals to the simulator list window.



You can optionally add signals to the simulation log without displaying them in the Wave or List window by choosing **Add Log** from the **Display** cascade of the **Simulation** menu.

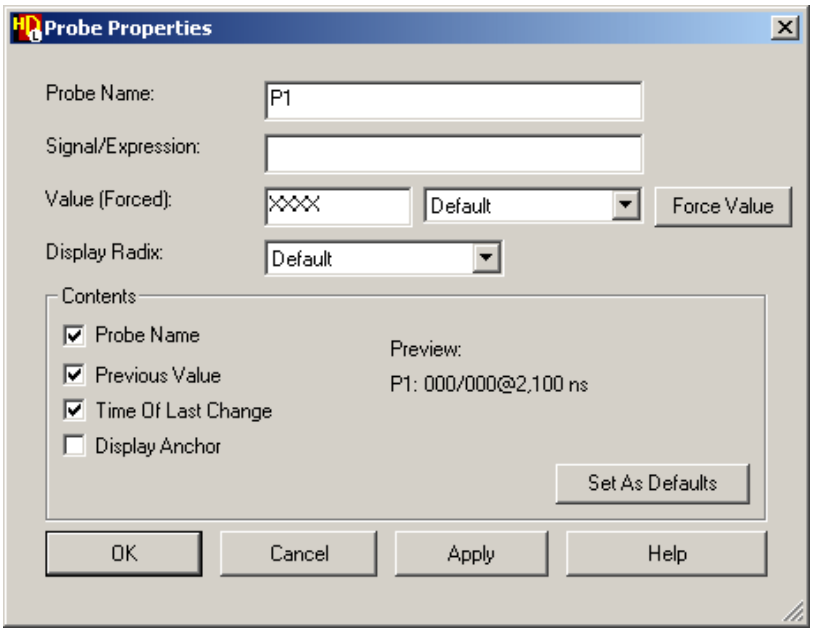
You can also open any other simulator window by choosing options from the **View** cascade of the **Simulation** menu. For example, you may wish to use this menu to open the Model*Sim* **Source** and **Structure** windows.

Add Simulation Probes

7. Set simulation probes on the *low* and *high* signals by using the button or by choosing **Add** from the **Probes** cascade of the **Simulation** menu. The probes are shown by icons with text displaying the current values.

8. Select the probe on the *high* signal and select **Probe Properties** from the popup menu to display the Probe Properties dialog box.

9. Change the probe name from the default net name to *P1* and set the Contents options for **Probe Name**, **Previous Value** and **Time of Last Change**. Use the Apply to update the probe display on the block diagram.



10. Repeat this procedure for the probe on the *low* signal renaming this probe as *P2*.

    Both probes should now display the probe names, current and previous values (initially unknown) and the current simulation time (initially 0 nanoseconds):



    The probes are connected to the signal nets by an anchor and can optionally be moved to improved diagram clarity.

Enable Animation

11. Open the *Control* state diagram and its child hierarchical state diagram. Use the **View Panel** command from the **View** menu in each tab to adjust the window view to the panel areas you defined earlier in this tutorial.



If you have created a flow chart view for the *Timer_tester* view, it can be animated in a similar way to that described below for the state diagrams.

Notice that an additional Animation toolbar is available in the state diagram (and flow chart) windows when the simulator is invoked.

This toolbar is normally docked at the bottom of the diagram window next to the Simulation toolbar but can be moved independently.



The editing toolbars are not usually required during simulation and animation. You can hide or show toolbars using the **Toolbars** cascade of the **View** menu.

12. For example, use the dialog box to hide the Format Text, Appearance, Comment Graphics and Arrange Object toolbars.



Individual toolbars can also be hidden by unsetting the corresponding options in the **Toolbars** cascade of the **View** menu.

13. Use the  buttons from the Animation toolbars (or choose **Data Capture** from the **Animation** menus) in each state diagram or flow chart you want to animate. (It is not necessary to repeat this command for each hierarchical or concurrent view since these are considered part of the same diagram.)

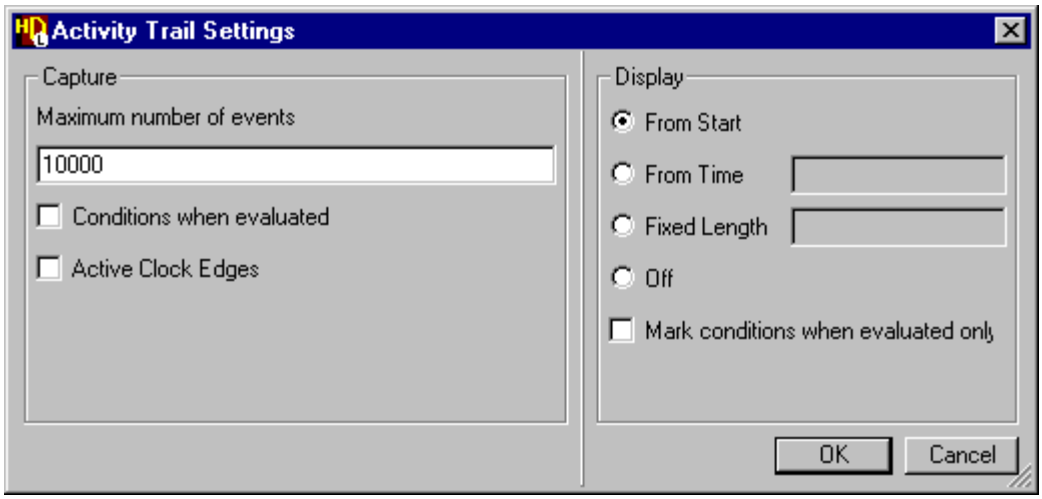
Notice that all objects (except the start state, exit and entry points in the state diagram) are redrawn with white fill. The start state and start point are drawn with red fill to indicate that they represent the current animation state.



This animation view is automatically displayed when you set data capture but can be toggled at any time by using the  button (or by toggling the **Show Animation** option in the **Animation** menu).

You can also set data capture for all instances in the current simulation hierarchy by choosing **Global Capture On** from the **Animation** menu.

14. Use the button from the Animation toolbar (or choose **Activity Trails** from the **Animation** menu) in the flow chart or state diagram window and choose the **From Start** option in the Activity Trail Settings dialog box.



15. Use the OK button to confirm the Activity Trail Settings dialog box.

The activity trail is applied to all flow chart or state diagram windows in the current simulation.

16. Use the button or choose **Add State Variable to Wave Window** from the **Display** cascade of the **Simulation** menu in the state diagram.

    This command monitors the state machine variable by a signal corresponding to the current state of the state machine.

17. Use the button or the **Add State Variable to List Window** command to add the same signal in the List window.

Simulate the Design

18. Use the Timer_Tb block diagram window to run the simulator for the default timestep (100 nano-seconds) by using the button or by choosing **For Time** from the **Run** cascade of the **Simulation** menu in the block diagram, flow chart or state diagram



The default timestep can be changed by using the button to select from a list of alternative timesteps or choose a user specified timestep.

The simulation probes remain yellow on the block diagram and in the simulator Wave and List windows because their value is unchanged.

If you have created an animated flow chart notice that the *Initialization* action box is highlighted red for the *Timer_tester*.

19. Run the simulator for another default timestep and notice that both simulation probes have changed to red because their value has changed since the last simulator timestep.

20. Select the *stop* signal in the *Timer_tb* block diagram and set a breakpoint by using the button or choosing **Add** from the **Breakpoints** cascade of the **Simulation** menu. The breakpoint is shown by a icon.



You can optionally add a simulation probe to the *stop* signal using the procedure described in "Add Simulation Probes" on page 1-78.

21. Run the simulator until there are no more events scheduled by using the button from the Simulation toolbar or by choosing **Forever** from the **Run** cascade of the **Simulation** menu.

    The simulation should run until the breakpoint is encountered when the value of the *stop* signal value (shown in the Wave and List windows) changes to '1'. Notice that the *counting* state is entered in the child animated state machine for the *count* state.

    If you have created an animated flow chart notice that the *Store* action box is highlighted in red as the current step.

22. Use the button or choose **Continue** from the **Run** cascade of the **Simulation** menu to continue the simulation until the *stop* signal changes back to zero.

    Notice that the *suspended* state is entered in the animated state machine and the following note is issued in the main simulation window:

```
**Note: Count suspended correctly.
```

23. Run the simulator to the next change of the *stop* signal by using the  button and notice the message:

```
**Note: Alarm asserted correctly
```

24. Use the  button in the test bench block diagram to delete the breakpoint on the *stop* signal and complete the simulation by using the  button.

    The simulation should run to completion and issue the message:

```
**Failure: Timer test completed
```

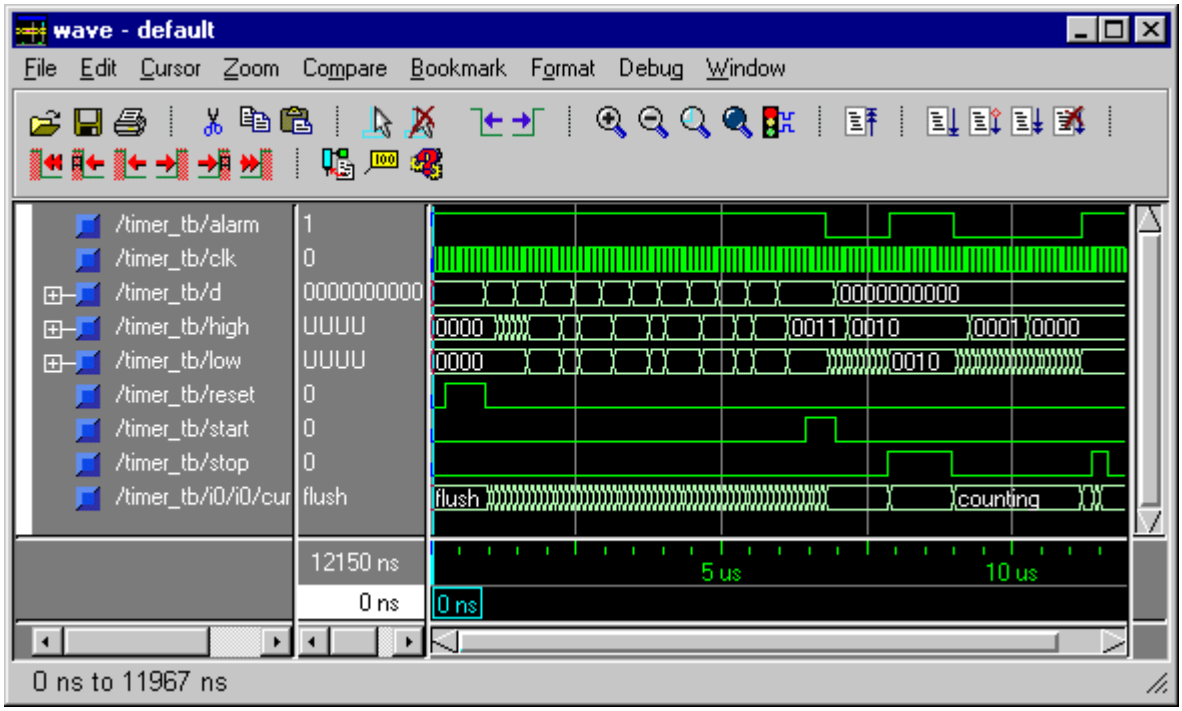
    Examine the results displayed in the Wave window and ensure that the simulation has performed correctly by comparison with the example waveforms shown below.

If any VHDL errors are discovered, correct your design and use the  button to regenerate and recompile the modified diagram. If the diagram is successfully generated and compiled, the flow button automatically restarts the simulation for the current simulation hierarchy.



You can use the  button or choose **Highlight Object** from the **Display** cascade of the simulation menu to highlight all occurrences in the simulation windows of a signal corresponding to any selected graphical object.

Alternatively, you can use the button or choose **Restart Simulator** from the Simulation menu to repeat the simulation.





You can display the full simulation waveform by using the button or choosing **Zoom Full** from the Wave window **Zoom** menu.

Review the Animation

Notice how the animation activity trail in the animated diagrams is highlighted in blue and the exit break point for *Timer test completed* is indexed on the flow chart VHDL architecture displayed in the simulator Source window.

25. If you have enabled animation for the *Timer_tester* flow chart, use the button or choose **Link Diagrams** from the **Animation** menu to link the animated diagrams. When the diagrams are linked, the animation review commands are applied to all the animated flow charts and state diagrams in the simulation hierarchy.

You can review the animation by using the , , , or buttons (or by choosing **Goto Next**, **Goto Previous**, **Goto Time**, **Goto Start** or **Goto Latest** from the **Animation** menu).

The  and  buttons step through each object in the flow chart. However in the state diagram, you can set options in the **Animation** menu (or from a pulldown menu on the toolbar button) to move by change of state  , simulation event  or change of clock edge  . The toolbar button icon changes to indicate the current mode of movement.

You may want to change the activity trail to display the trail from a specified time or specify a fixed length.

The current simulation step (or current state and last transition taken in an animated state diagram) is always shown in red and the previous step (or previous state and transition) in yellow. All other visited objects included in the activity trail are shown in blue. Remember that you can open down the hierarchical action box or hierarchical state to view the animation in the child diagrams.

26. Use the toolbar buttons to move forwards and backward through the animation.

27. Compare the animation with the results displayed in the Waveform window and ensure that the simulation has performed correctly by comparison with the example waveforms on the previous page.

Debugging From ModelSim

If you are using ModelSim, additional toolbar and menu commands are provided for cross-probing to the graphical views. These include the following:

- A  button is available in the Animation toolbar and a **Cause** option from the **Animation** menu. These commands can be used to move the ModelSim Wave and List windows to the current animation time.
- A corresponding  button and **Cause** option (in the **Debug** menu) are available in the ModelSim Wave window. These commands can be used to update all currently open animation windows to the simulation step or event immediately preceding the time marked by the Wave window cursor.
- A **Cause** command is also added to the **Debug** menu in the simulator List window which can be used to update all open animation windows to the simulation step or event corresponding to the selected line in the List window.
- A  button is added to the toolbar (and a **Trace To HDS** option is added to the **Debug** menu) in the Source window which can be used to open the graphic window showing the source object corresponding to the line of HDL code under the cursor.
- The **Trace To HDS** command is available in the **Debug** menu for the Model*Sim* Wave, List and Signals windows (or using the  button in the Wave window). It can be used in these windows to display the source object where the selected

signal is declared. (Typically, this will be the test bench or top level block diagram for the design being simulated.) You can also trace a signal to HDS from the Wave or Signals windows by double-clicking on the signal.

- A  button and **Trace To HDS** command are available from the **Debug** and popup menus in the main simulation window. A **Trace To HDS** command is also added to **Debug** menu in Model*Sim* Structure window. These commands can be used to open the source object corresponding to the selected instance.



If you are using Model*Sim* SE, you can also move through the simulation by using a Model*Sim* cursor. The state machine animation and the values displayed in the simulation probes are setup to track the cursor by default.

These options can be enabled or disabled by using the  button or by setting options in the state diagram **Display** menu and the block diagram **Probes** menu.

Refer to “Cross Probing from Model*Sim*” and “Using the Model*Sim* Source Window” in the *HDL Designer Series Graphical Editors User Manual* for more information about the cross-probing commands added to Model*Sim* when it is used with a HDL Designer Series tool.

28. Exit the simulator (by choosing **Quit** from the **File** menu in the main Model*Sim* window). Any other simulator windows are automatically closed.

29. Close all graphic editor windows by choosing **All Editor Windows** from the **Close** cascade of the **File** menu in the design manager.

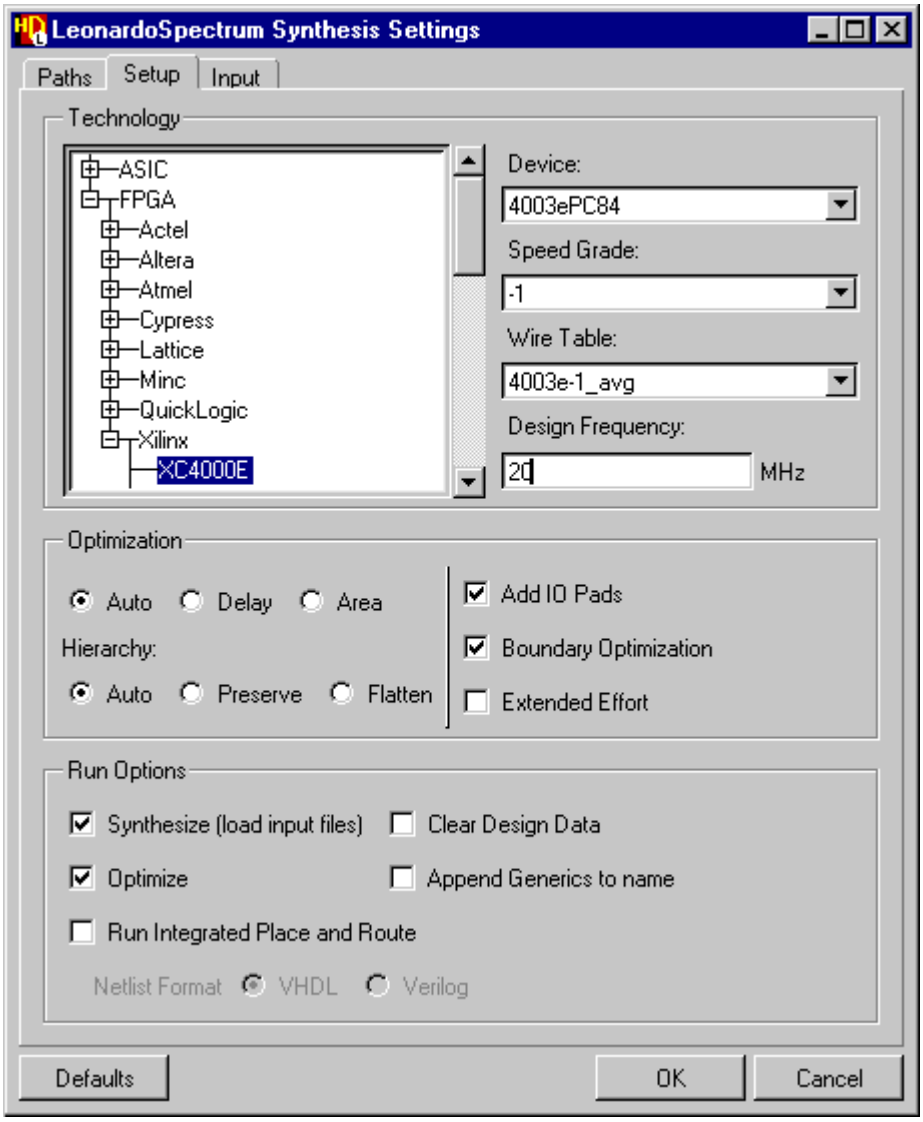
Run the LeonardoSpectrum Flow

Windows, Solaris and HPUX users may have access to the Leonardo Spectrum Flow.

1. Select the *Timer* design unit in the design explorer and use the  button to run the LeonardoSpectrum flow through components.

The tester design unit used in the test bench contains unsynthesizable HDL. Ensure that you have selected the *Timer* and not the *Timer_tb* design unit in the design explorer.

The design will be regenerated if necessary and.the LeonardoSpectrum Synthesis Settings dialog box is displayed:



2. Use the ⊞ icons to expand the FPGA technologies list and select the technology of your choice. For example, Xilinx XC4000E.



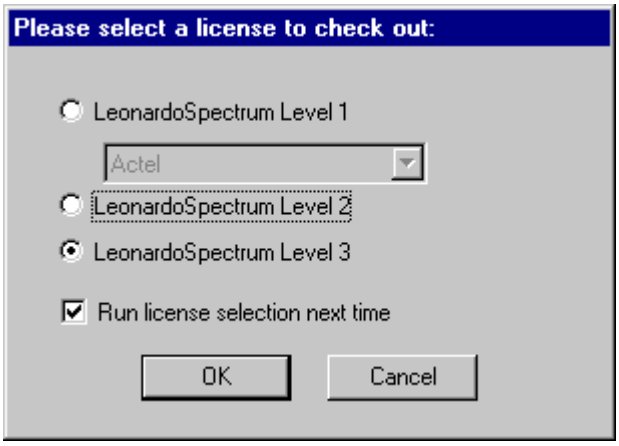
If you have an Exemplar level 3 license, ASIC and FPGA technology libraries are available. If you have a level 2 license, only FPGA libraries are available.

The Device, Speed Grade and Wire Table fields are automatically selected when you have chosen a technology.

3. Complete the dialog box by entering a clock frequency (for example 20 MHz). All other options can be left with their default values.
The options in the other tabs can also be left with their default values.

4. Use the OK button to confirm and close the dialog box.

When you confirm the invoke settings, LeonardoSpectrum is invoked and you are prompted to select an Exemplar level 1, 2 or 3 license.

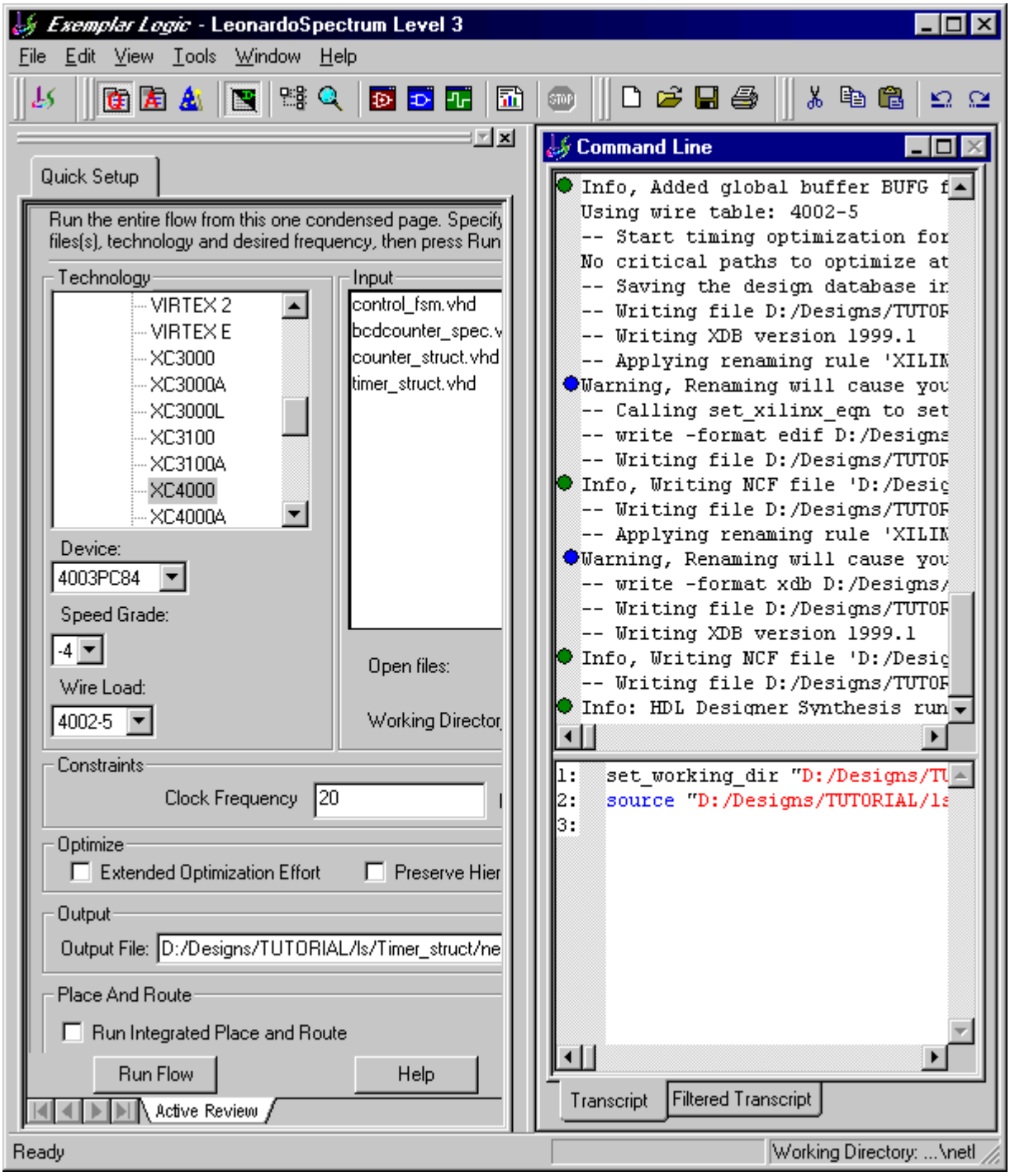




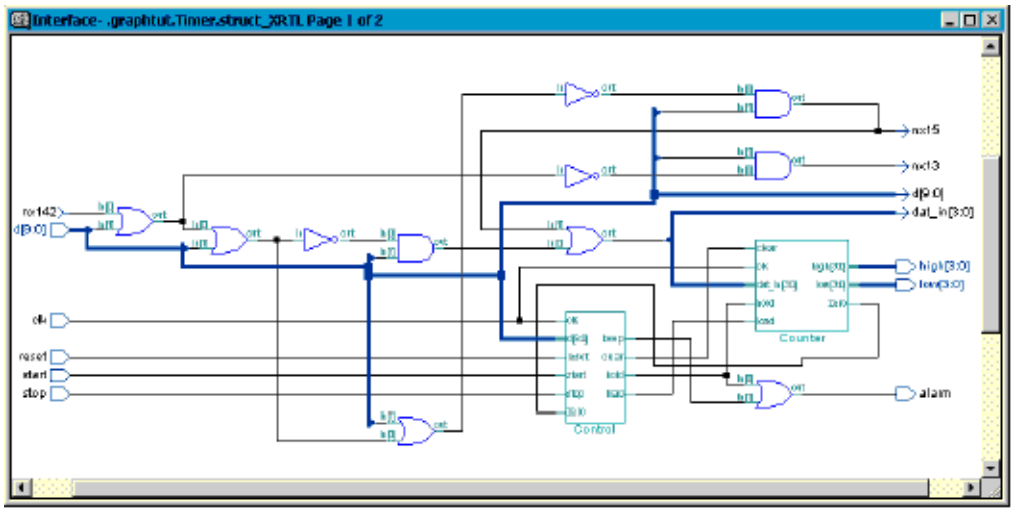
Higher level licenses enable more features but you can choose to run at a lower level using a higher level license.

When you confirm the license, your design is synthesized and optimized for the selected technology.

Status messages are transcripted in the Command window ending with the message "*HDL Designer Synthesis run finished*".:



You can use the button from the LeonardoSpectrum toolbar to display the RTL schematic or the button to display the technology schematic for your design. For example, the following picture shows the RTL schematic:



You can move around the schematic using the scroll bars or the diagram can be enlarged or maximized in the schematic window by choosing the **Zoom In** or **Zoom Fit** options from the **Zoom** cascade of the popup menu.

You can crossprobe to the corresponding source design object by selecting an object in the schematic window and choosing **Trace to HDL Designer** from the popup menu.

5. Exit from the synthesis tool by choosing **Exit** from the LeonardoSpectrum **File** menu and respond No when you are prompted whether to save your project settings.

You have now completed this tutorial.

If you have not successfully verified your design, a completed reference example is available in the examples subdirectory of the HDL Designer Series installation. This example can be accessed by opening the *TIMER_Vhdl* library in the design explorer.

Using the Example VHDL Design

A completed example design can be accessed from the *TIMER_Vhdl* library in the *Examples* project. The example design includes generated HDL but only dummy library mapping for downstream data.

To use the example design, you should change the location of the downstream data directory to a location for which you have write permissions. Refer to Editing Library Mapping in the *HDL Designer Series User Manual* for information about changing library mapping.

Alternatively, you can easily create your own copy of the example design by using the procedure described in Using the Example Designs in the *Start Here Guide for the HDL Designer Series*.



Windows users typically have write access to the installation directories. However, it is recommended that you change the mapping to a suitable directory for user data rather than overwrite the installed examples.

Once you have modified the library mapping, you can browse, compile, simulate and animate the example design by following the procedures from “Browse the Completed Design” on page 1-73.

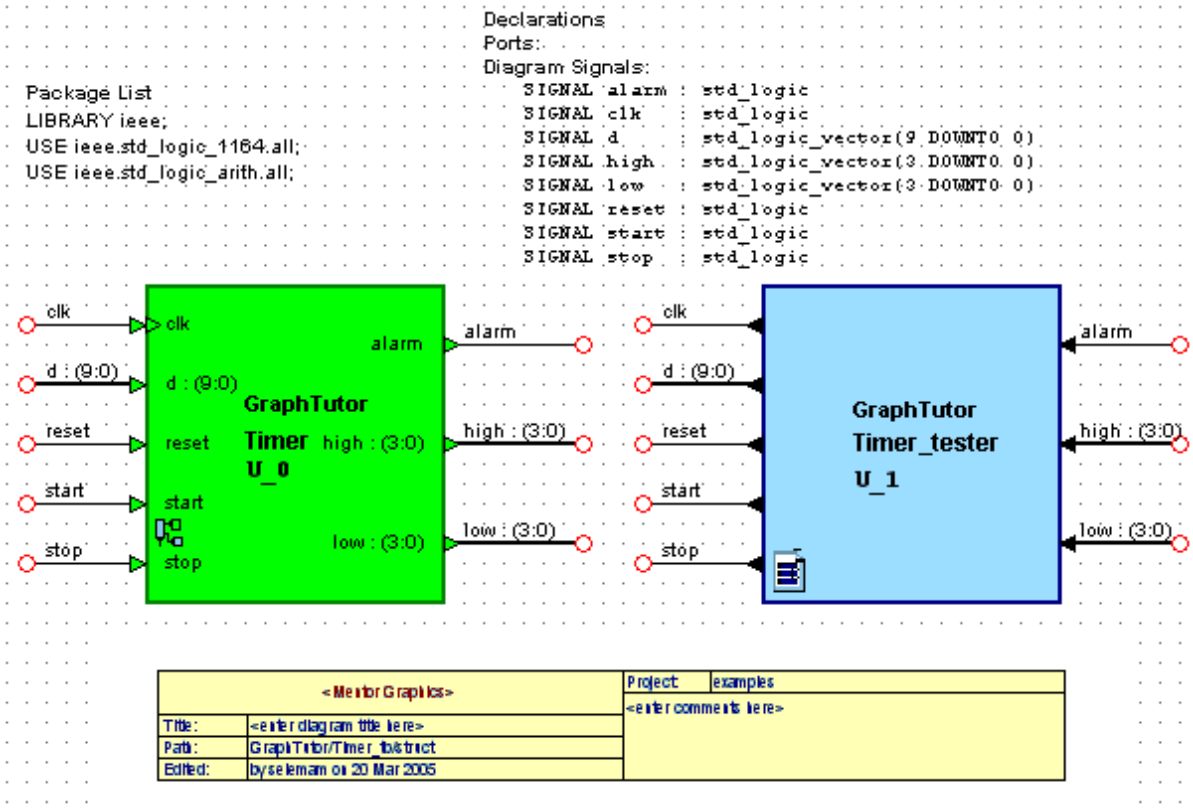
In order to animate the flow chart and state machine in the example design, you should regenerate HDL through components after setting **Instrument HDL for animation** in the state machine and flow chart properties. Refer to “Set Generation Properties” on page 1-39 and “Generate HDL for the Test Bench” on page 1-71.

Chapter 2
Creating a VHDL Flow Chart

Introduction

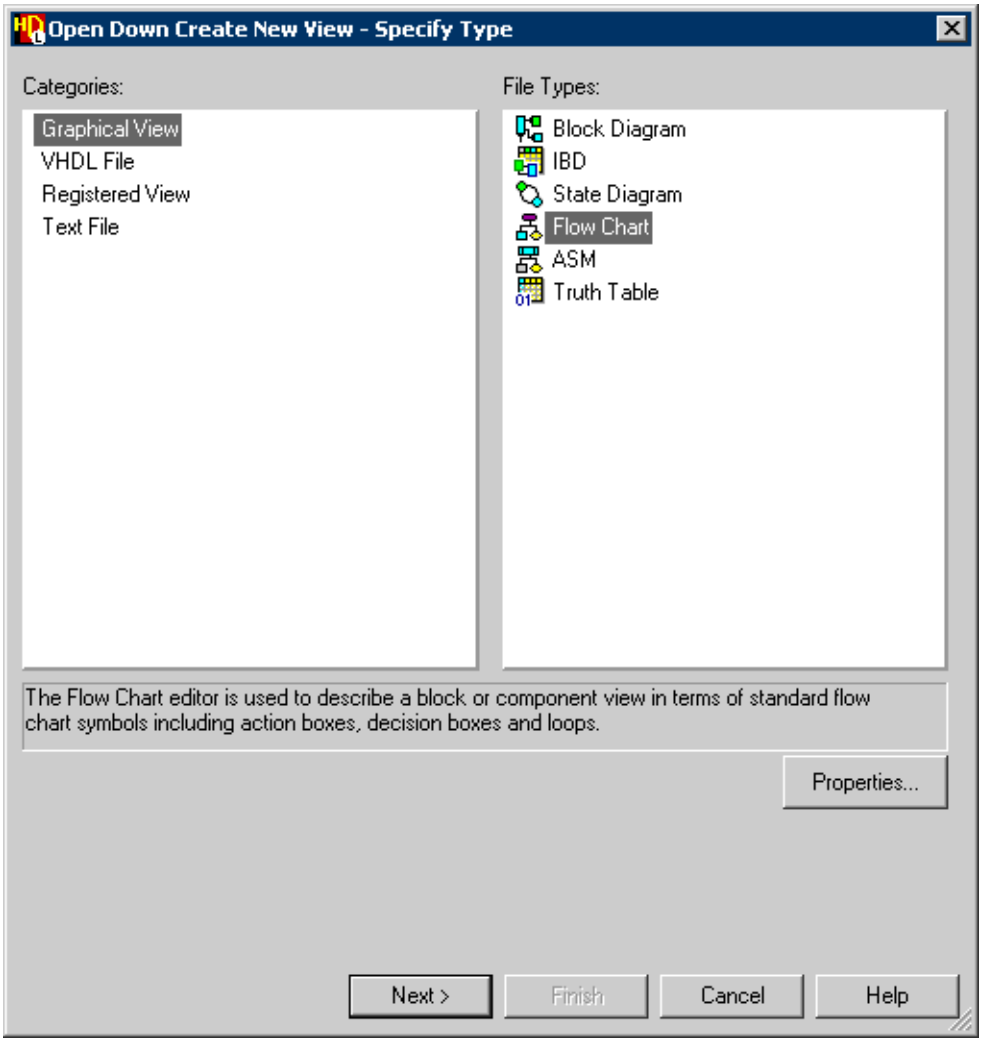
This chapter describes alternative procedures for using a VHDL flow chart instead of importing the *Timer_tester* as a HDL text view.

After you "Create a Test Bench" on page 1-65 the test bench block diagram should look similar to the following picture:



Create the Tester Flow Chart

1. Double-click over the *Timer_tester* block in the test bench block diagram and select a **Flow Chart** view in the Open Down Create New View wizard.

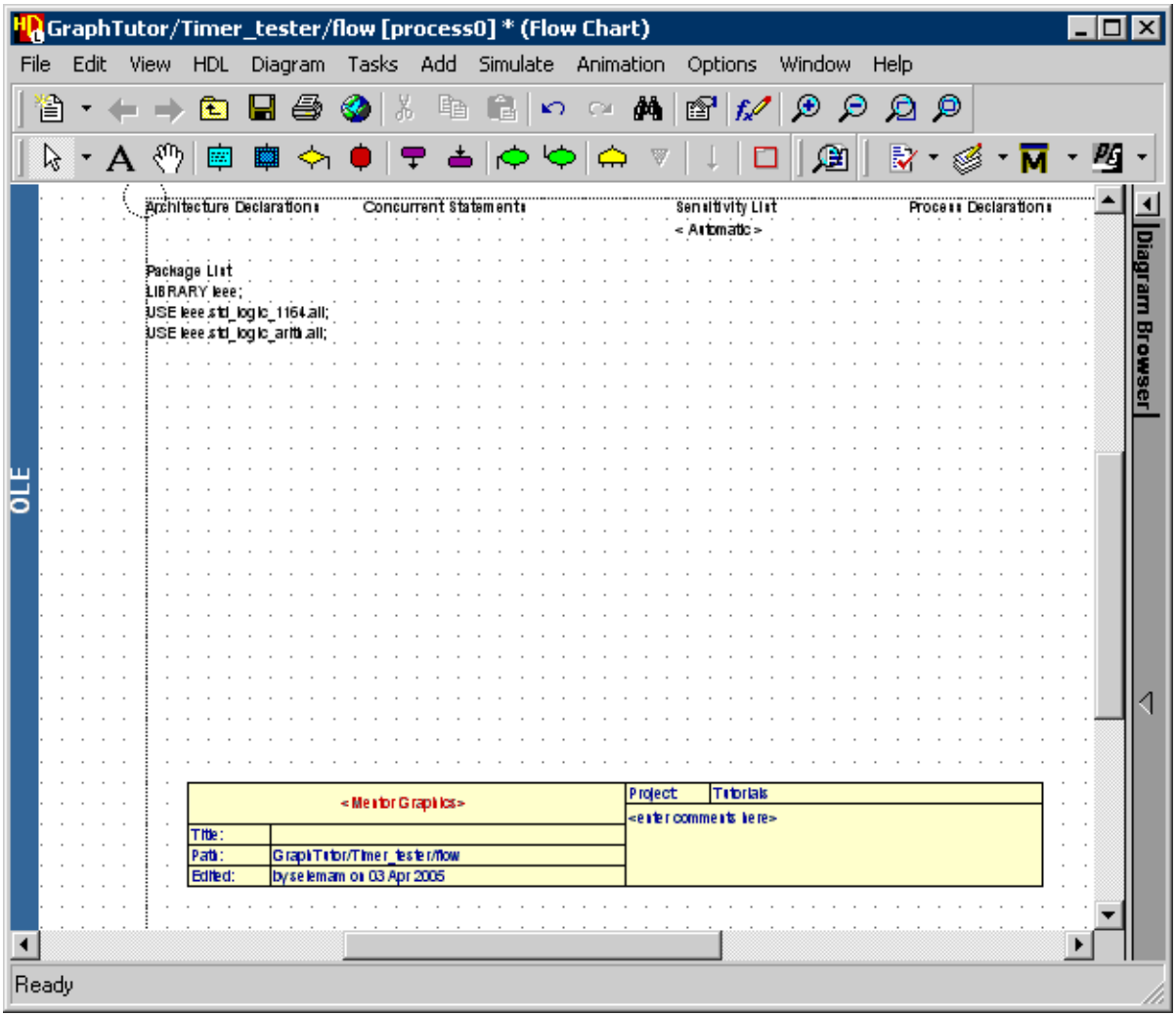


   If you have previously created a view for the *Timer_tester* block, this will open instead. Choose **Flow Chart** from the **Graphical View** cascade of the  button to create a Flow Chart.

2. Use the  button to display the Specify View Name page and confirm the wizard using the default view name *flow.fc*.

   A new flow chart (*GraphTutor/Timer_tester/flow [process0]*) is created as a child view of the *Timer_tester* block.

The new flow chart is initialized with page boundaries set for the default printer, the default VHDL package list, default title block and labels for architecture declarations, concurrent statements, sensitivity list and process declarations:



3. Choose **Rename Flow Chart** from the **Diagram** menu and enter the new name *Monitor*. When you confirm the dialog box, this name replaces the default process name *process0* appended to the design unit and view names.

Set Flow Chart Properties

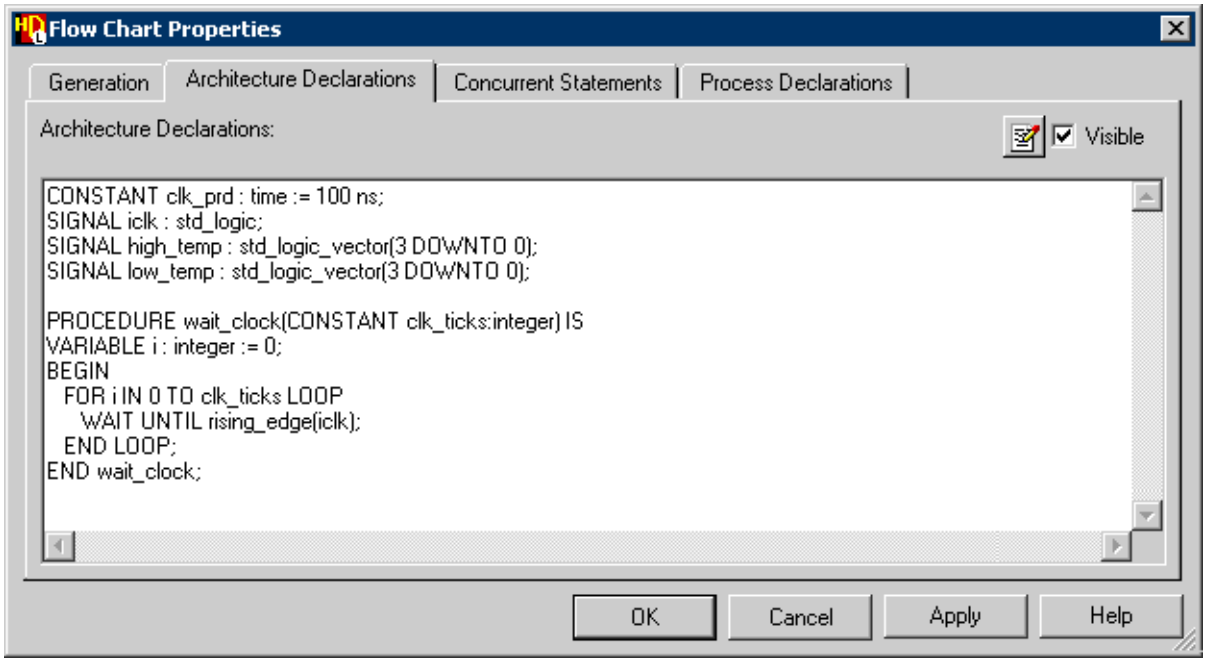
4. Double-click with the mouse over the Architecture Declarations label in the flow chart to display the **Architecture Declarations** tab of the Flow Chart Properties dialog box.

5. Enter the following declarations which set up some internal variables and a simple wait procedure:

```
CONSTANT clk_prd : time := 100 ns;
SIGNAL iclk : std_logic;
SIGNAL high_temp : std_logic_vector(3 DOWNTO 0);
SIGNAL low_temp : std_logic_vector(3 DOWNTO 0);

PROCEDURE wait_clock(CONSTANT clk_ticks:integer) IS
VARIABLE i : integer := 0;
BEGIN
   FOR i IN 0 TO clk_ticks LOOP
      WAIT UNTIL rising_edge(iclk);
   END LOOP;
END wait_clock;
```

The architecture declarations are included at the top of the architecture description in the generated HDL and must be valid VHDL statements terminated by a semi-colon. Comment text can be included provided each comment is preceded by the VHDL comment characters (--).



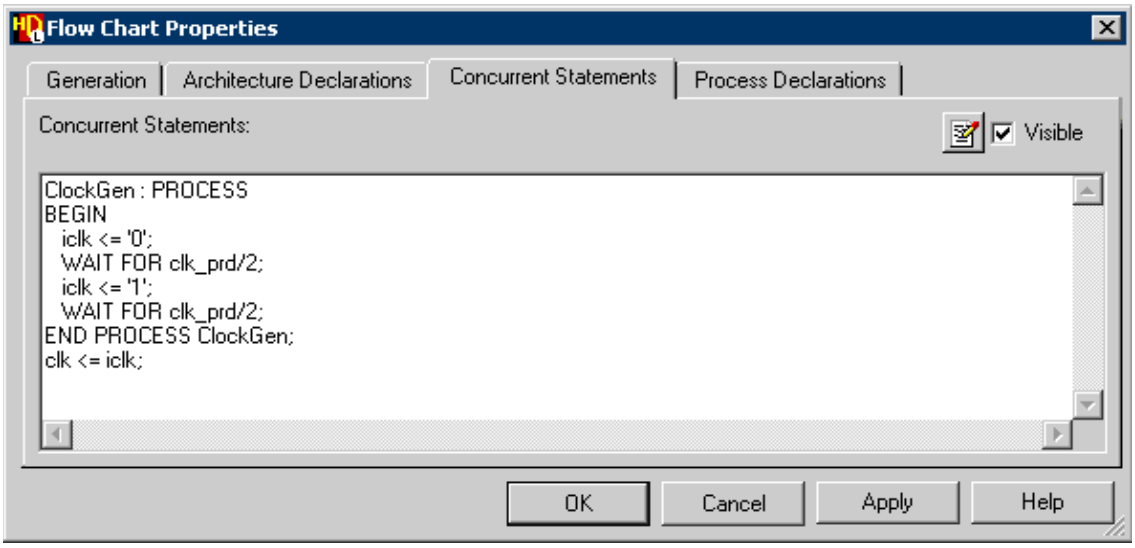
6. Use the Apply button to display these properties on the flow chart. The HDL syntax is checked on entry and you are warned if any errors are encountered.

7. Select the **Concurrent Statements** tab and enter the following statements which generate a clock signal for the test bench:

```
ClockGen : PROCESS
BEGIN
   iclk <= '0';
   WAIT FOR clk_prd/2;
   iclk <= '1';
   WAIT FOR clk_prd/2;
END PROCESS ClockGen;
```

```
clk <= iclk;
```

Concurrent statements are included after the BEGIN statement in the generated HDL for the flow chart and must be valid HDL statements and be terminated by a semi-colon. They are typically used for a concurrent process (such as the clock generator defined by this example).

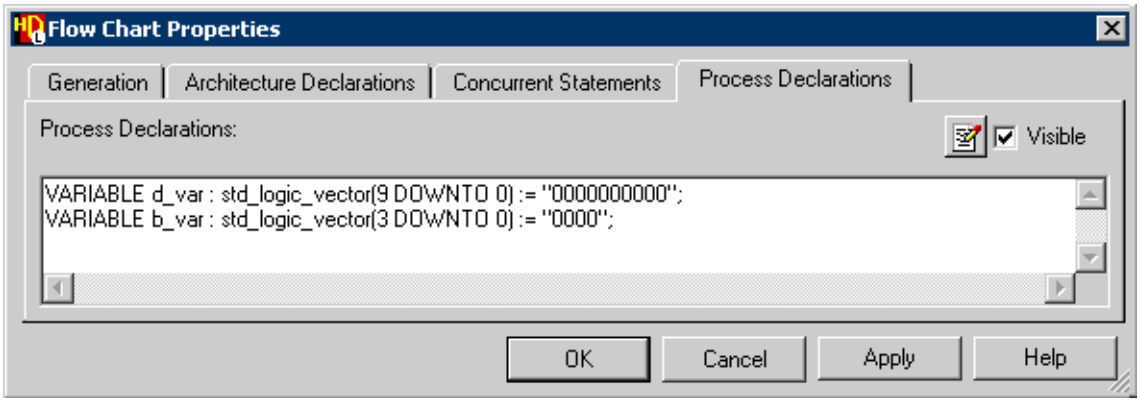


8. Use the Apply button to display these properties on the flow chart.

9. Select the **Process Declarations** tab and enter the following variable declarations:

```
VARIABLE d_var : std_logic_vector(9 DOWNTO 0) := "0000000000";
VARIABLE b_var : std_logic_vector(3 DOWNTO 0) := "0000";
```

Process declarations are included at the top of the main process in the HDL generated for the flow chart and must be valid HDL statements terminated by a semi-colon.



10. Use the OK button to apply these properties to the flow chart and dismiss the dialog box.

Add a Start Point and Action Box

11. Use the  button to add a start point near the top of the flow chart.
12. Use the  button to add an action box below the start point.

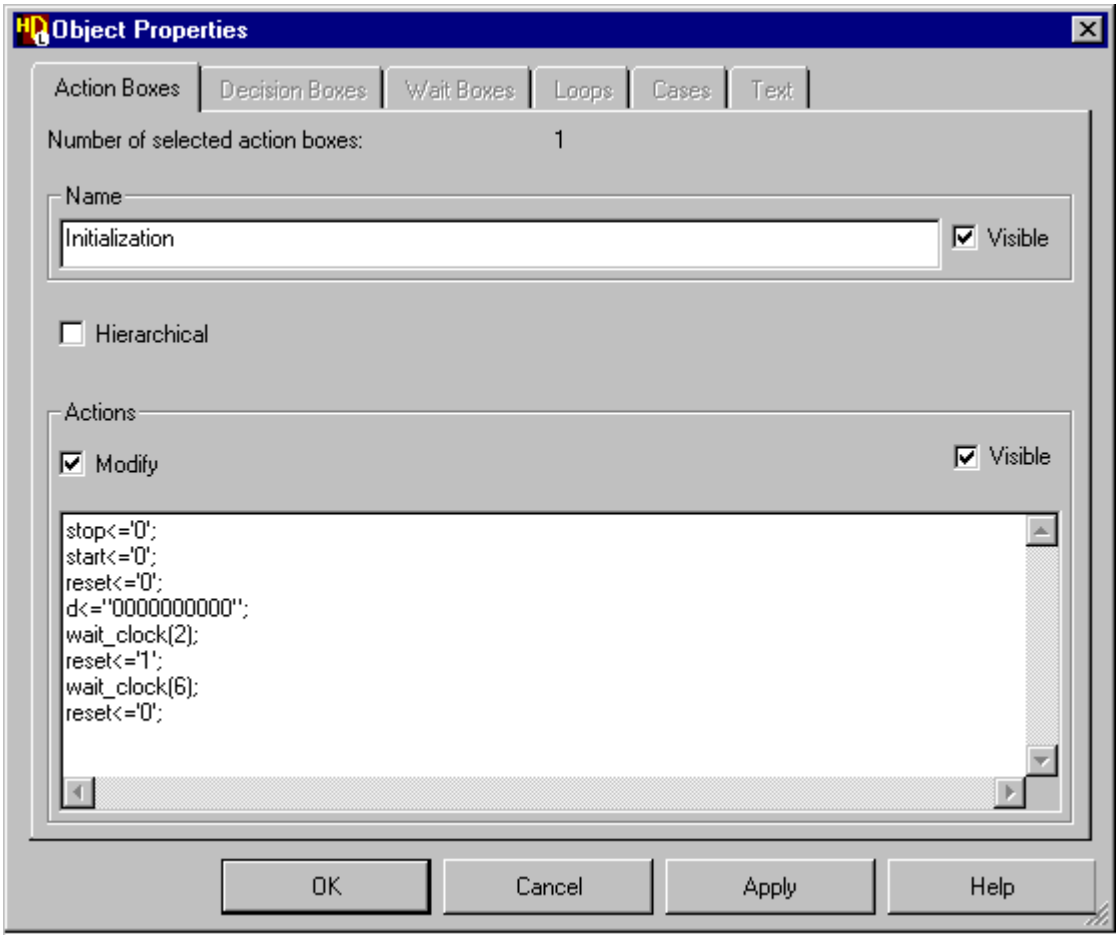
    Notice that the start point and action box are automatically connected.

13. Double-click with the mouse over the action box (or use the  button) to display the **Action Boxes** tab of the Flow Chart Object Properties dialog box.

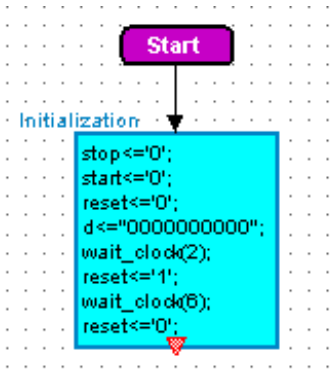
    You can change the action box name and enter action statements. However, the name must be unique on the flow chart and cannot be changed when more than one action box is selected.

14. Change the default name in the dialog box to *Initialization* and enter the following actions:

```
stop<='0';
start<='0';
reset<='0';
d<="0000000000";
wait_clock(2);
reset<='1';
wait_clock(6);
reset<='0';
```



15. Use the Apply button to update the flow chart.



If you drag one side of an action box, the mid-point is preserved.

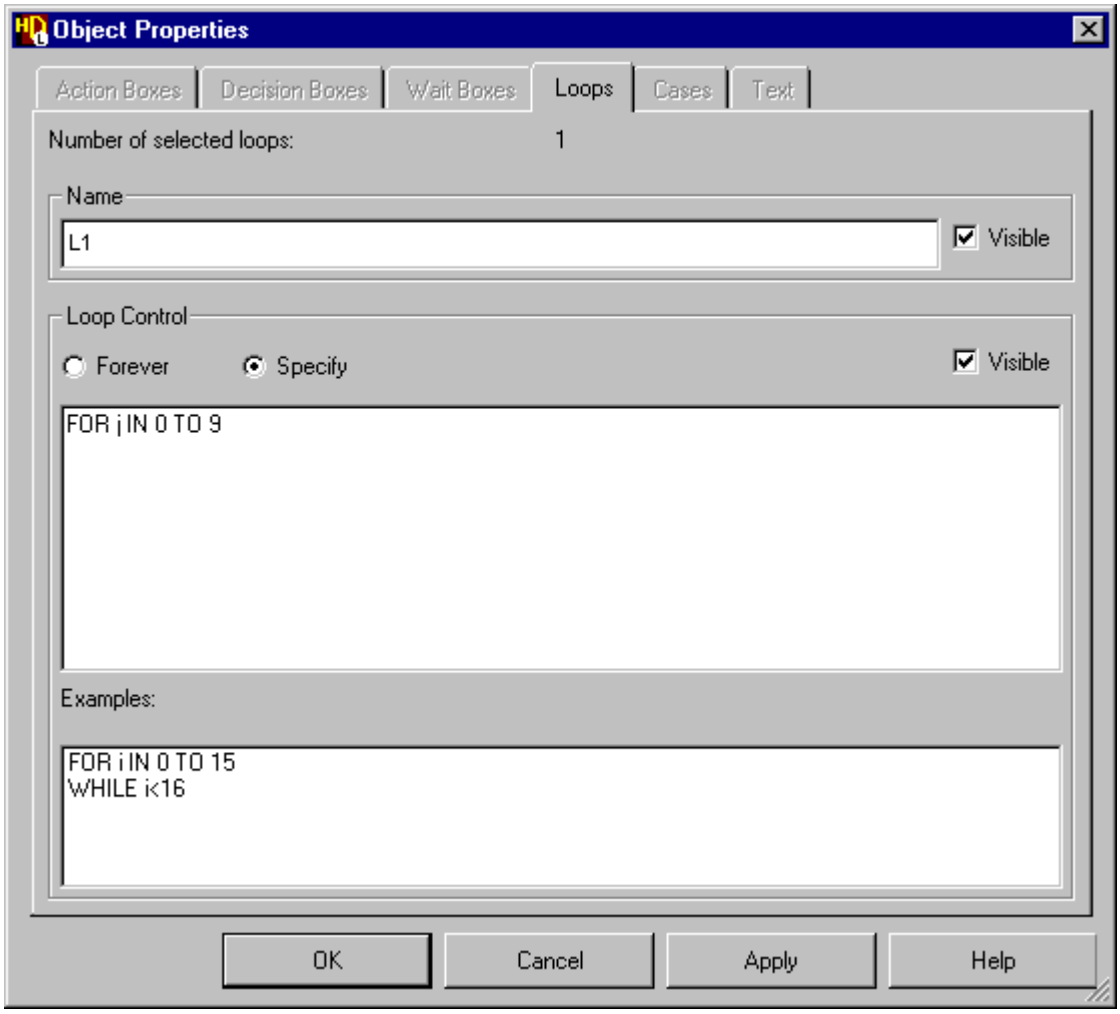
Add a Loop and an Associated Comment

16. Use the  button to add a start loop below the *Initialization* action box.

17. Double-click the start loop to display the **Loops** tab of the Flow Chart Object Properties dialog box. Choose the **Specify** button and enter the following loop control statement:

```
FOR j IN 0 TO 9
```

18. Rename the loop *L1* and use the Apply button to update the flow chart.



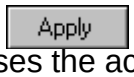


By default, a loop will be repeated forever. However, if you choose the **Specify** option, the dialog box discloses an entry box for loop control statements. Template examples are provided which you can copy into the entry box by clicking on one of the examples with the mouse.

19. Use the  button to add an action box below the start loop.

20. Click with the mouse to select the new action box and use the **Action Boxes** tab of the Flow Chart Object Properties dialog box to enter the following actions:

```
d_var:="0000000000";
d_var(j):='1';
d<=d_var;
wait_clock(4);
ASSERT ((high=b_var)AND(low=b_var))
REPORT "Decoder or Load failure."
SEVERITY failure;
b_var:=unsigned(b_var)+1;
```

21. Change the name of the action box to *Decoder* and use the 

button to update the flow chart. Resize the action box so that it encloses the actions text.

22. Use the  button to add an end loop below the *Decoder* action box.

23. Use the  button to enter text mode. Click in a free space near the loop and enter the following text in the comment text entry box:

```
Loop through all
possible decoder
values and check
output values
```



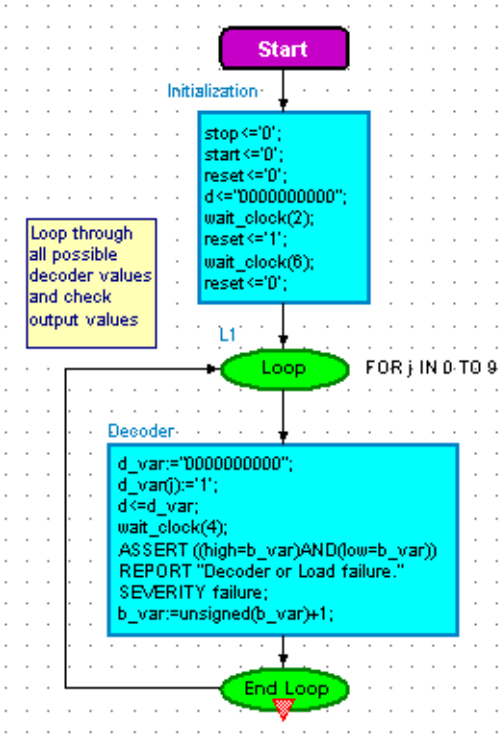
You can enter free-format text including line breaks and spaces which will be preserved on the diagram.

24. Click the left mouse button outside the text entry box to complete the text. Right click to deselect the text area.

25. Select the comment text and choose **Include in HDL** from the popup menu. Select **Before Object** from the popup menu. An anchor is attached to the cursor. Click the left mouse button with the cursor over the start loop object to terminate the anchor and associate the notes with the loop.

    The comment text will be included immediately before the loop statements in the generated HDL for the flow chart.

The flow chart should look similar to the following picture:



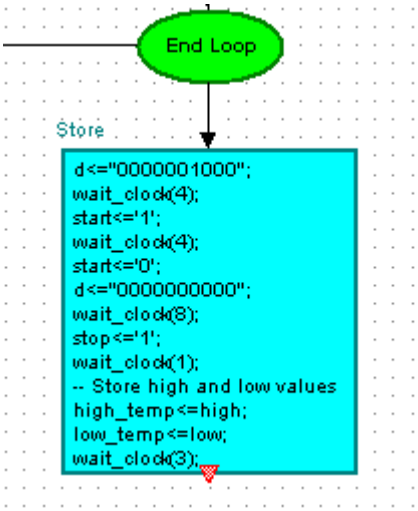
Add an Action Box

26. Use the button to add another action box below the end loop and use the Flow Chart Object Properties dialog box to rename the action box *Store* and apply the following actions:

```
d<="0000001000";
wait_clock(4);
start<='1';
wait_clock(4);
start<='0';
d<="0000000000";
wait_clock(8);
stop<='1';
wait_clock(1);
--Store high and low values
high_temp<=high;
low_temp<=low;
wait_clock(3);
```



You can include comment text directly in an actions box by using HDL comment characters. For example the comment text *--Store high and low values* in the example above.



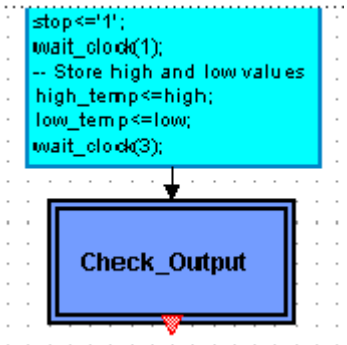
Add a Hierarchical Action Box

When a flow chart contains a large number of procedural statements it can rapidly become a very large diagram which can be difficult to read or print. However, such a chart can be represented as a number of separate hierarchical charts by using hierarchical action boxes.



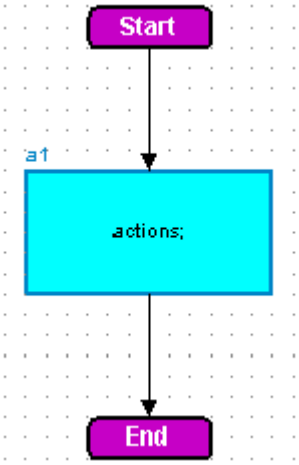
Flow chart hierarchy does not effect the generated HDL and is not used if you use HDL import to recover the *Timer_tester* flow chart.

27. Use the  button to add a hierarchical action box below the *Store* action box and use direct text editing to change the name of the hierarchical action box to *Check_Output*.



28. Double-click on the *Check_Output* hierarchical action box to open a new child flow chart (or select the hierarchical action box and choose **Open Down** from the **File** menu).

The child flow chart is initialized in a new window tab as an action box connected by flows to a start point and end point.



Add a Decision Box

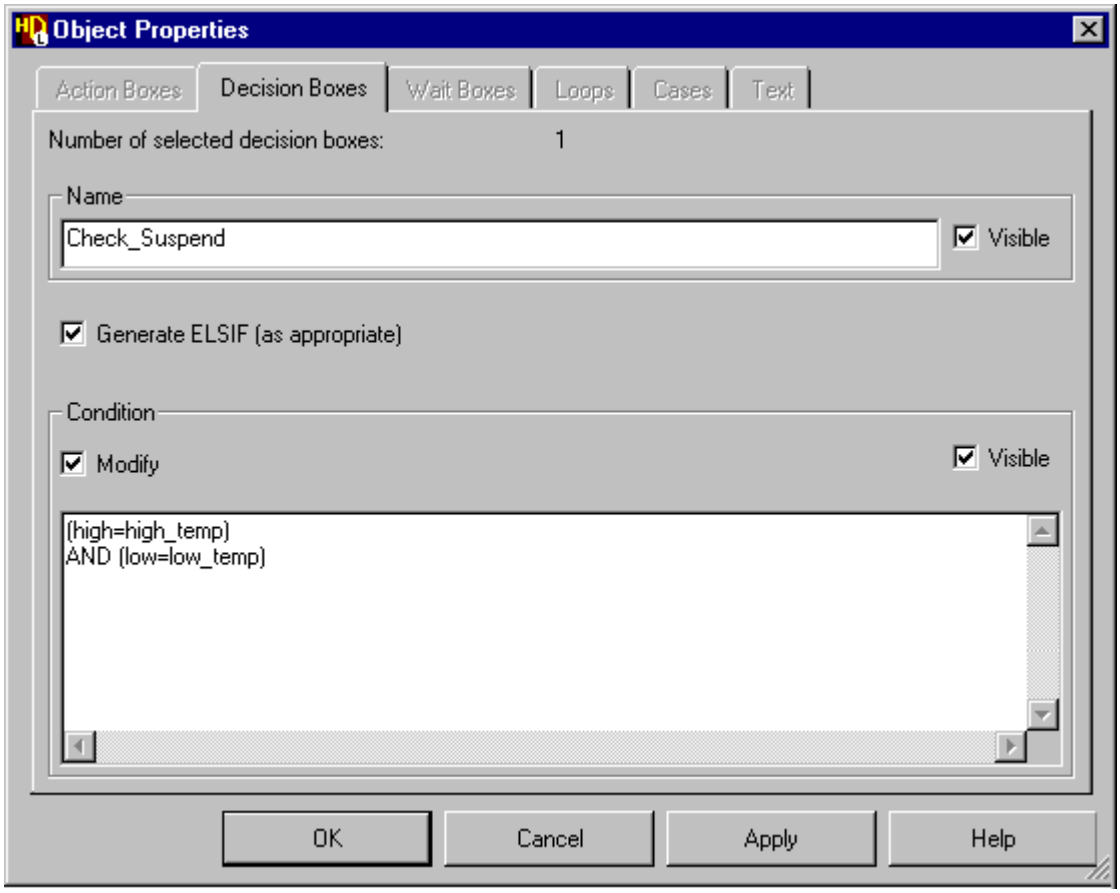
29. Select the flows connected to the start point and end point in the child flow chart and use the Del key to delete them.

You can use Shift key with the left mouse button to select both flows.

30. Use the button to add a decision box below the start point in the *Check_Output* flow chart.

31. Use the **Decision Boxes** tab of the Flow Chart Object Properties dialog box to change the name of the decision box to *Check_Suspend* and enter the following condition:

```
(high=high_temp)
AND (low=low_temp)
```



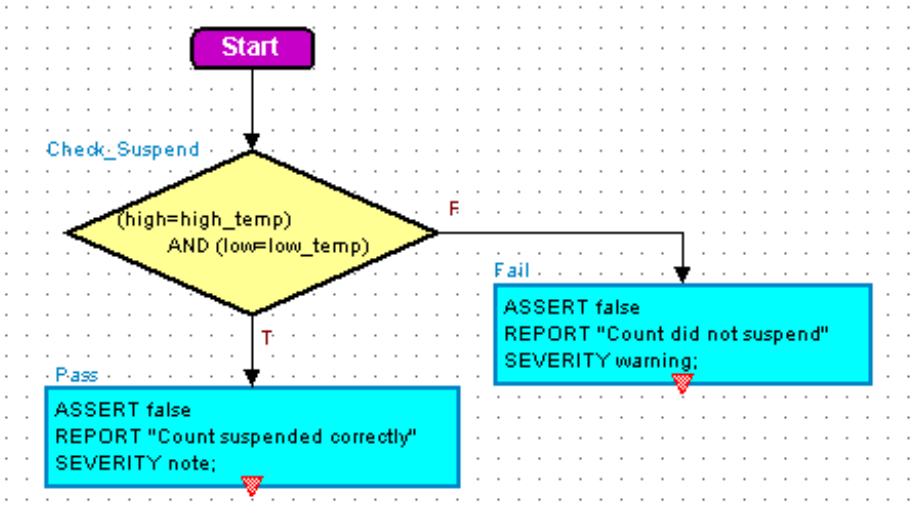
32. Select the existing action box. Use the **Action Boxes** tab of the Flow Chart Object Properties dialog box to change its name to *Pass* and enter the following actions:

```
-- Pass
ASSERT false
REPORT "Count suspended correctly"
SEVERITY note;
```

33. Use the button to add an action box for the False (F) condition. Use the **Action Boxes** tab to change its name to *Fail* and enter the following actions:

```
-- Fail
ASSERT false
REPORT "Count did not suspend"
SEVERITY warning;
```

34. Reposition the action boxes by dragging with the mouse and use the ↓ button to connect flows between the decision box and the pass action box..



Notice that the start point and Fail action boxes have been automatically connected to the decision box.

When a decision box object is selected, the popup menu includes an option to swap the True and False outputs.

Add Wait Boxes

35. Use the button to add an action box below the *Pass* action box and use the **Action Boxes** tab of the Flow Chart Object Properties dialog box to enter the following actions:

```
wait_clock(4);
stop<='0';
```

36. Change the name of this action box to *Continue*.

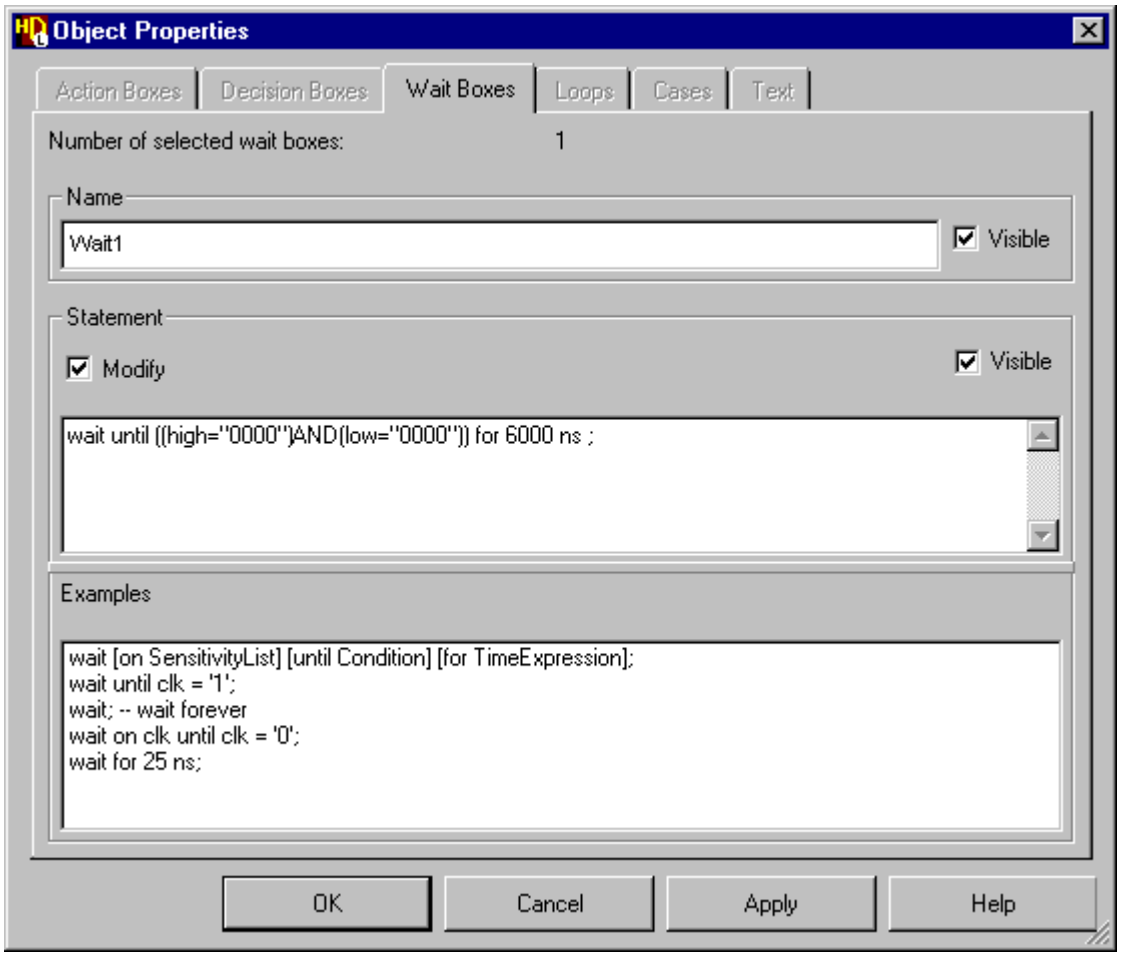
37. Use the button to add two wait boxes below it.

38. Click the mouse over the first wait box (or use the button) to display the **Wait Boxes** tab of the Flow Chart Object Properties dialog box:

39. Use the **Wait Box** tab of the Flow Chart Object Properties dialog box to rename the first wait box *Wait1* and specify the following wait condition:

```
wait until ((high="0000")AND(low="0000")) for 6000 ns ;
```

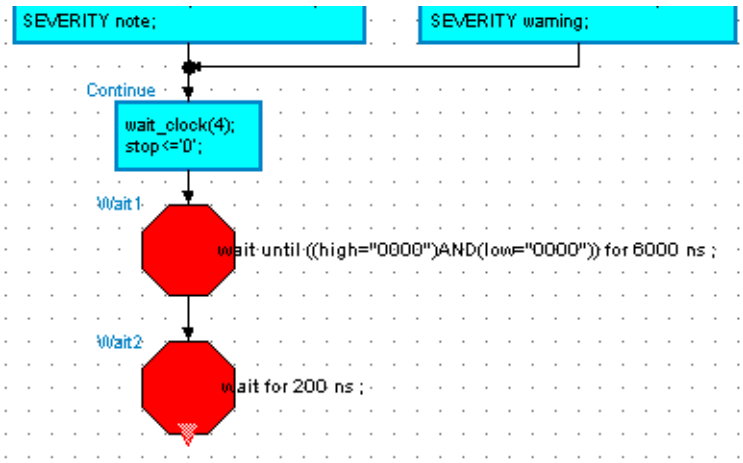
You can click on one of the examples shown in the dialog box to use it as a template. Any valid combination of **on**, **until** and **for** can be used in the wait statement which must be terminated by a semi-colon.



40. Use the **Wait Box** tab to rename the second wait box *Wait2* and specify the following wait condition:

```
wait for 200 ns ;
```

41. Use the ↓ button to connect the two flows from the *Pass* and *Fail* action boxes below the *Pass* action box. Notice that a flow join is automatically created when you connect the two flows together.



Copy the Decision Tree

The child hierarchical flow chart can be completed by using similar procedures to those described in the last two topics. However, the next section can be done more quickly by copying the decision tree objects which have already been created.

42. Select the *Check_Suspend* decision box, plus the *Pass* and *Fail* action boxes and use the button (or choose **Copy** from the popup or **Edit** menu).

43. Scroll the window down and click the Left mouse button below the last wait box to de-select the objects. Use the button (or choose **Paste** from the popup or **Edit** menu) to paste a copy of the objects from the clipboard in the middle of the window. Use the mouse to drag the objects to the required position.

    Notice that each of the pasted objects is automatically given a unique name.

44. Use the Flow Chart Object Properties dialog box to change the names, conditions and actions where required.

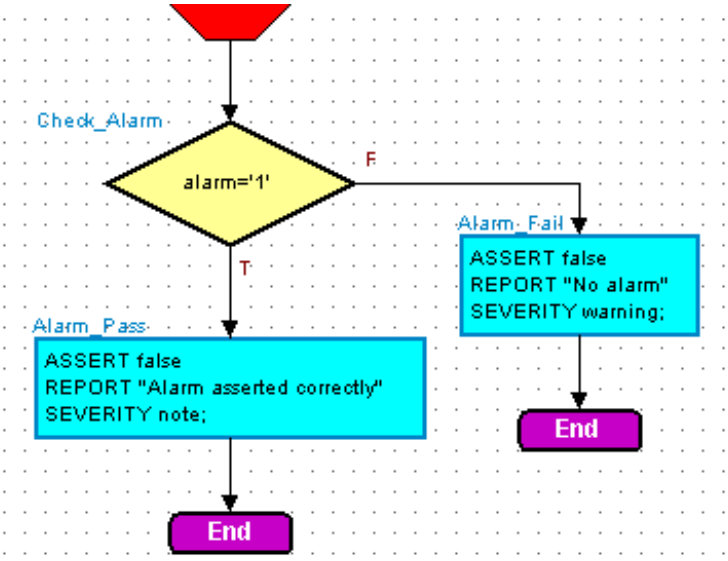
```
Check_Alarm
alarm='1'

Alarm_Pass
ASSERT false
REPORT "Alarm asserted correctly"
SEVERITY note;

Alarm_Fail
ASSERT false
REPORT "No alarm"
SEVERITY warning;
```

45. Drag the end point beneath the other objects and use the ↓ button to connect the remaining flows on the child flow chart.

    Multiple end points can be used. For example, in the picture below, a separate end point has been attached to the *Alarm_Pass* and *Alarm_Fail* action boxes.



Completing the Flow Chart

46. Use the button (or choose **Open Up** from the **File** menu) to display the parent flow chart and the button to add an action box below the hierarchical action box.

47. Use the Flow Chart Object Properties dialog box to change the name of this action box to *Clear* and enter the following actions:

```
stop<='1';
wait_clock(2);
stop<='0';
wait_clock(4);
ASSERT false
REPORT "Timer test completed"
SEVERITY Failure;
```



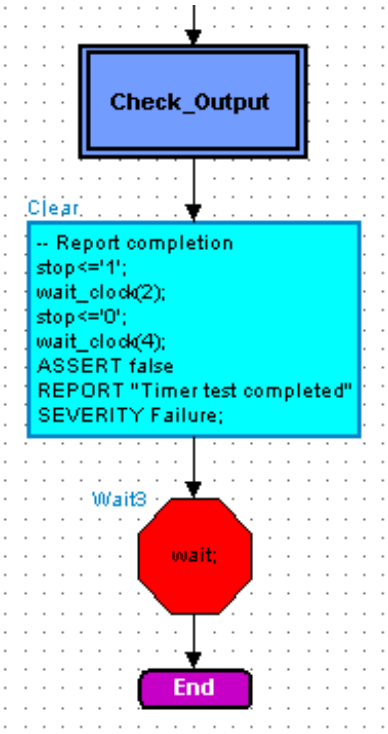
These actions are executed when the timer test has completed successfully but are given *Failure* severity to stop the simulator execution after the completion message is issued.

48. Choose the OK button to confirm the Flow Chart Object Properties box and dismiss it.

49. Use the button to add a wait box below the action box. Change the name of this wait box to *Wait3* with the default wait condition (*wait;*).

    This condition corresponds to a wait for interaction at the completion of the test sequence.

50. Use the button to add an end point and use the button to connect any remaining flows.



51. Complete the flow chart by editing the title, project and comments fields in the title block.

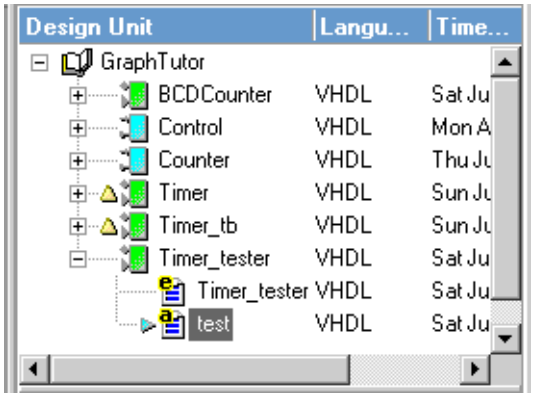
52. Use the button to save the flow chart.

Return to "Generate HDL for the Test Bench" on page 1-71 in the main VHDL tutorial.

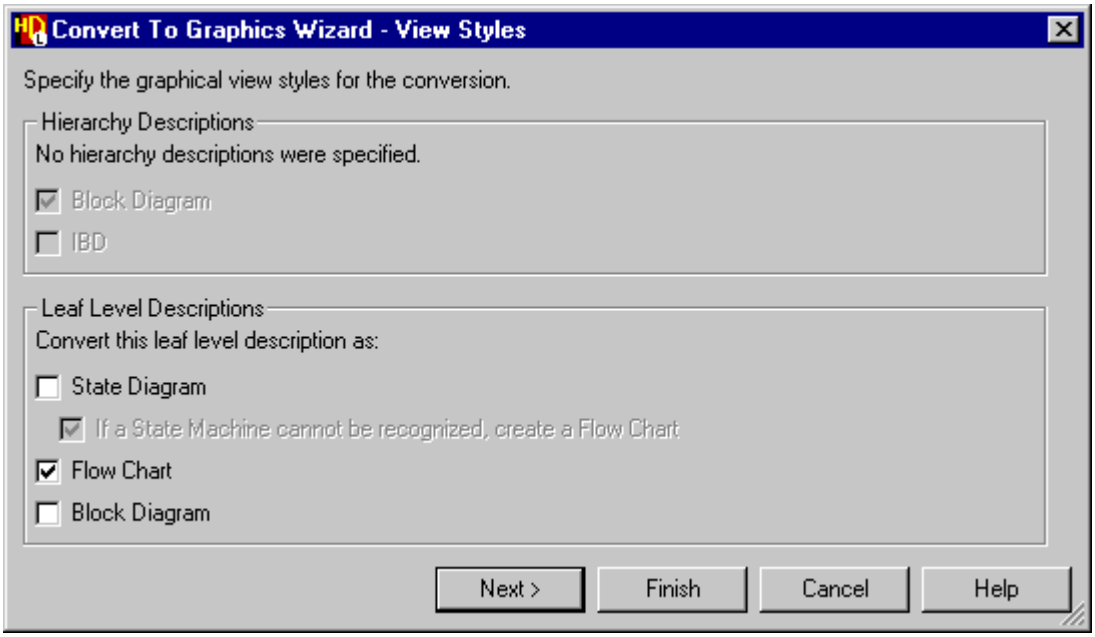
Using HDL2Graphics

If you are using **HDL Designer**, you can use HDL2Graphics to convert the HDL text view of the *Timer_tester* design unit to a graphical flow chart.

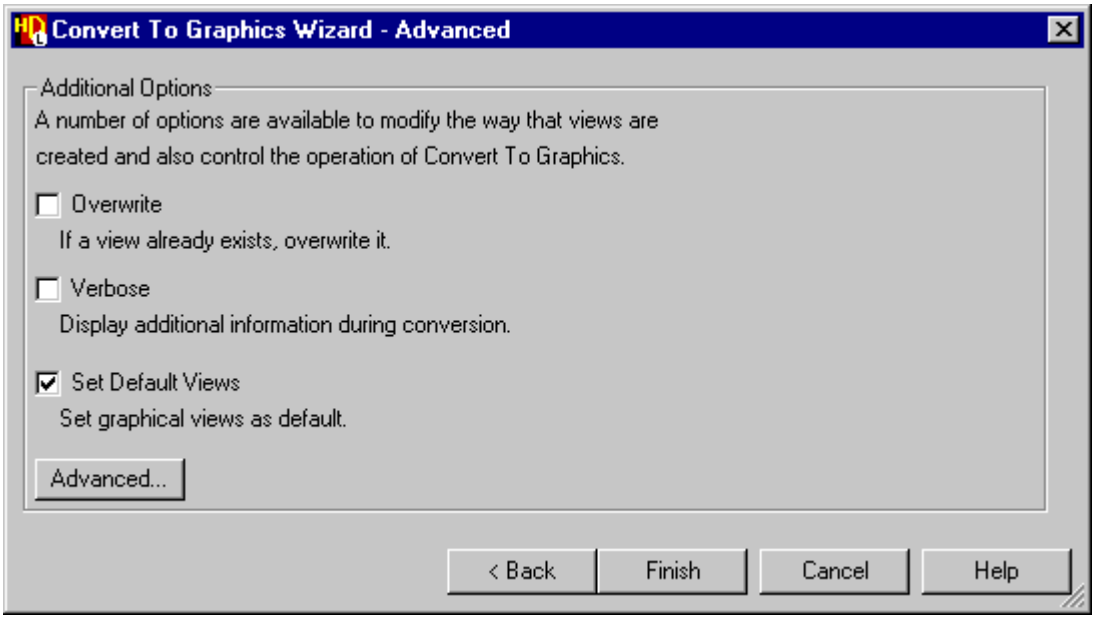
1. Select the VHDL architecture *test {Timer_tester.vhd}* for the *Timer_tester* design unit in the *Design Units* view of the design explorer:



2. Use the  button or choose **Single Level** from the **Convert To Graphics** cascade of the **HDL** menu to display the View Styles page of the Convert To Graphics wizard:



3. Set the **Flow Chart** option in the Leaf-level Descriptions section of this page and use the Next > button to display the Advanced page of the wizard:



4. Set the **Set Default Views** option and use the Finish button to convert the VHDL architecture view to a flow chart.

   You are prompted to confirm that you wish to continue. When you confirm the prompt dialog box, the new graphical view is made the default view but the HDL text view is retained as an alternative view for the design unit.

   Notice that a graphical symbol is also created for the *Timer_tester* design unit.

Return to "Instantiate the Imported Tester" on page 1-69 in the main VHDL tutorial.

Trademark Information

The following are trademarks of Mentor Graphics Corporation:

3D Design, ABIST, ARC4, Arithmetic BIST, Accelerated Technology®, AccuPARTner, AccuParts, AccuSim®, ADEPT, ADVance, ADVance MS, ADVance RFIC, AMPLE, Analog Analyst, Analog Station, Ares®, ARTgrid, ArtRouter, ARTshape, ASICPlan, ASICVector Interfaces, Aspire, AuthExpress, AutoActive®, AutoCells, AutoDissolve, AutoFilter, AutoFlow, AutoLib, AutoLinear, AutoLink, AutoLogic, AutoLogic BLOCKS, AutoLogic FPGA, AutoLogic VHDL®, AutomotiveLib, AutoPAR®, AutoTherm®, AutoTherm Duo, Auto ThermMCM, AutoView, Autowire Station, AXEL, AXEL Symbol Genie, AX-GMAC, BISTArchitect, BLAST, Blaze, BlazeRouter, Board Station Consumer, Board Architect, Board Designer, Board Layout, Board Process Library, BoardSim®, Board Station®, BOLD Administrator, BOLD Browser, BOLD Composer, Bridgepoint®, BSDArchitect, BSPBuilder, Buy on Demand, Cable Analyzer, Cable Station, CAECO Designer, CAEFORM, Calibre®, Calibre DRC, Calibre DRC-H, Calibre DESIGNrev, Calibre CB, Calibre FRACTUREh, Calibre FRACTUREj Calibre FRACTUREm, Calibre FRACTUREt, Calibre FRACTUREk, Calibre LITHOview, Calibre LVS-H, Calibre MTflex, Calibre OPCsbar, Calibre OPCpro, Calibre ORC, Calibre PRINTimage, Calibre PSMgate, Calibre PSMcheck, Calibre TDopc, Calibre WORKbench, Calibre RVE, Calibre MGC, Calibre Interactive, Calibre MDPview, Calibre xRC, Capital®, Capital Analysis, Capital Archive, Capital Bridges, Capital Documents, Capital H®, Capital H the complete desktop engineer®, Capital Harness, Capital Harness Systems, Capital Insight, Capital Integration, Capital Manager, Capital Manufacture, Capital Support, Capital Systems, Capture Station®, Catapult, Celaro, Cell Builder, Cell Station®, CellFloor, CellGen,CellGraph, CellPlace, CellPower, CellRoute, Centricity, CEOC, ChaseX, CheckMate, CHEOS, Chip Station®, ChipGraph, ChipLister, Circuit PathFinder, Co-Verification Environment, COLsim, Code/lab®, CommLib, CommLib BMC, Concurrent Design Environment, Connectivity Dataport, Continuum, Continuum Power Analyst, CoreAlliance, CoreBIST, Core Builder, Core Factory, CTIntegrator, DataCentric Model, DataFusion, Datapath, Data Solvent, dBUG, Debug Detective, DC Analyzer, Deltacore®, DeltaV, Design Architect®, Design Architect®-IC, Design Architect Elite, Design Capture, Design Exchange®, Design Manager, Design Station®, DesignBook®, DesignView, DesktopASIC, Destination PCB®, Destiny RE, DFTAdvisor, DFTArchitect, DFTInsight, DMS, DMS Xchange,, DxAnalog, DxDataBook, DxDesigner, DxLibraryStudio, DxParts, DxPDF, DxViewOnly, DxVariantManager, Direct System Verification, DSV, Documentation Station, DSS (Decision Support System), DxAnalog, DxDataManager, DxDesigner, DxEnterprise for Agile, DxMatrix, DxViewDraw, E3Lcable, EDA Tech ForumEDT, Eldo®, EldoNet, ePartners, EParts®, EPlanner®, eProduct Designer, EProduct Services®, Empowering Solutions, Engineer's Desktop, EngineerView, Enterprise Librarian, ENRead, ENWrite, ESim, Exemplar, ExemplarLogic, Expedition, Explorer CAECO Layout, Explorer CheckMate, Explorer Datapath, Explorer Lsim, Explorer Lsim-C, Explorer Lsim-S, Explorer Ltime, Explorer Schematic, Explorer VHDLsim, ExpressI/O, EZwave, FabLink, Falcon®, Falcon Framework®, FastScan, FastStart, Fire, First-Pass Design Success, First-Pass Success, FlexSim, FlexTest, FDL (Flow Definition Language), FlowTabs, FlowXpert, FORMA, FormalPro, FPGA Advantage®, FPGAdvisor, FPGA BoardLink, FPGA Builder, FPGASim, FPGA Station®, FPGA Xchange, FrameConnect, Fusion, Galileo®, Gate Station®, GateGraph, GatePlace, GateRoute, GDT®, GDT Core, GDT Designer, GDT Developer, GENIE, GenWare, Geom Genie, HDL2Graphics, HDL Architect, HDL Architect Station, HDL Assistant, HDL Author, HDL Designer, HDL Designer Series, HDL Detective, HDL Inventor, HDL Link, HDL Pilot, HDL Processor, HDLSim, HDLWrite, Hierarchial Injection, Hierarchy Injection, HIC rules, Hardware Modeling Library, HotPlot®, Hybrid Designer, Hybrid Station®, HyperLynx®, HyperSuite®, IC Design Station, IC Designer, IC Layout Station, Library Manager, IC Station®, Streamview, ICbasic, ICblocks, ICcheck, ICcompact, ICdevice, ICextract, ICGen, ICgraph, ICLink, IClister, ICplan, ICRT Controller Lcompiler, ICrules, IC Studio, ICtrace, ICverify, ICview, ICX®, ICX Active, ICX Plan, ICX Pro, ICX Sentry, ICX Tau, ICX Verify, ICX Vision, ICX Custom Model, ICX Custom Modeling, ICX Project Modeling, ICX Standard Library, IDEA Series, Idea Station®, IKOS®, In All The Right Places, INFORM®, IFX, Inexia, Innovate PCB, Innoveda, Integrated Product Development, Integra Station®, Integration Tool Kit, IT, INTELLITEST®, Interactive LAYOUT, Interconnect Table, Interface-Based Design, IB, Inventra, Inventra IPX, Inventra Soft Cores, IP Engine, IP Evaluation Kit, IP Factory, IP-PCB, IP QuickUse, IPSim, IS Analyzer, IS Floorplanner, IS MultiBoard, IS Optimizer, IS Synthesizer, iSolve, IV'locity, Language Neutral Licensing, Latium®, LAYOUT, LNL, LBIST, LBISTArchitect, Lc, Lcore, Leaf Cell Toolkit, Led, LED Layout, Leonardo®, LeonardoInsight, LeonardoSpectrum, Librarian, Library Builder, LineSim®, Logic Analyzer on a Chip, Logic Builder, Logical Cable, LogicLib, logio, Lsim, Lsim DSM, Lsim Gate, LsimNet, Lsim Power Analyst, Lsim Review, Lsim Switch, Lsim XL, Mach PA, Mach TA, ManufactureView, Manufacturing Advisor, Manufacturing Cable, MaskCompose, MBIST, MBISTArchitect, MBIST Full-Speed, MBIST Flex, MBIST In-Place, MBIST Manager, MCM Designer, MCM Station®, MDV, MegaFunction, Memory Builder, Memory Builder Conductor, Memory Builder Mozart, Memory Designer, Memory Model Builder, Mentor®, Mentor Graphics®, MicroPlan, MicroRoute, Microtec®, Mixed-Signal Pro, ModelEditor, ModelSim®, ModelSim LN, ModelSim VHDL, ModelSim VLOG, ModelSim SE, ModelStation®, Model Technology, ModelViewer, ModelViewerPlus, MODGEN, Monet®, Mslab, Msview, MS Analyzer, MS Architect, MS-Express, MSIMON, Nanokernal®, NetCheck, NETED, Nucleus®, Nucleus All You Need® Opsim, OutNet, P&RIntegrator, PACKAGE, PADS®, PARADE, ParallelRoute-Autocells, ParallelRoute-MicroRoute, Part Foundry, Parts SpeciaList, PathLink, PCB-Gen, PCB-Generator, PCB IGES, PCB Mechanical Interface, PCI Express, PDLSim, PE-GMAC, PE-MAC, Personal Learning Program, Physical Cable, Physical Test Manager:SITE, PLA Lcompiler, Platform Express, PX, PLDSynthesis, PLDSynthesis II, Power Analyst, Power Analyst Station,PowerLogic, PowerPCB®, Precision®, Pre-Silicon, ProjectXpert, ProtoBoard, ProtoView, QDS, QNet, QualityIBIS, QuickCheck, QuickFault, QuickConnect, QuickGrade, QuickHDL, QuickHDL Express, QuickHDL Pro, QuickPart Builder, QuickPart Tables, QuickParts, QuickPath, QuickSim, QuickStart, QuickUse, QuickUse Development System, QuickVHDL, Quiet, Quiet Expert, RAM Lcompiler, RC-Delay, RC-Reduction, RapidExpert, REAL Time Solutions!, Registrar, Reliability Advisor, Reliability Manager, REMEDI, Renoir®, RF Architect, RF Gateway, RISE, ROM Lcompiler, RTL X-Press, Satellite PCB Station, Scalable Verification, ScaleableModels, SCAP, Scan-Sequential, Scepter, Scepter DFF, Schematic View Compiler SVC, Schemgen, SDF (Software Data Formatter), SDL2000 Lcompiler, Seamless®, Seamless ASAP, Seamless C-Bridge, Selective Promotion, Sheet Planner, Signal Spy, Signal Vision, SignaMask OPC, Signature Synthesis, Simulation Manager, SimPilot, SimView, Smartgrid, SmartMask, SmartParts, SmartRouter, SmartScripts, Smartshape, SNX, SneakPath Analyzer,Spectra®, SpeedGate, SpeedGate DSV, SpeedWave, SOS Initiative, Source Explorer, SpiceNet, SST Velocity®, Standard Power Model Format (SPMF), Structure Recovery, Super C, Supermax® ECAD, Super IC Station, Supermax ECAD®, Symbol Genie, Symbolscript, SymGen, SYMED, SynthesisWizard, System Architect, System Design Station, System Modeling Blocks, Systems on Board Initiative, SystemVision, Target Manager, Tau®, TeamPCB, TeraCell, TeraPlace, TeraPlace-GF, TechNotes, TestKompress®, Test Station®, Test Structure Builder, The Ultimate Site For HDL Simulation, The Ultimate Tool For HDL Simulation, TimeCloser, Timing Builder, TNX, ToolBuilder, Transcable®, TransDesign, Truetiming, Utopia, Vlog, V-Express, V-Net, VHDLnet, VHDLwrite, Verinex, ViewBase®, ViewDraw®, ViewCreator, ViewLogic®, ViewSim®, ViewWare®, Viking, Virtual Library, VirtuaLogic, Virtual Target, Virtual Test Manager:TOP, VirtualWires, Voyager®, VRTX®, VRTXmc, VRTXoc, VRTXsa, VRTX32®, VStation, VStation-30M, Waveform DataPort, We Make TMN Easy, Wiz-o-matic, WorkXpert, xCalibre®, xCalibrate, Xconfig, XlibCreator, XML2AXEL, Xpert, Xpert API, XperBuilder, Xpert Dialogs, Xpert Profiler, XRAY®, XRAY MasterWorks®, XSH®, Xtrace®, Xtrace Daemon, Xtrace Protocol, Xtreme Design Client, Xtreme Design Session, XtremePCB, XTK, Zeelan®, Zero Tolerance Verification and ZLibs are trademarks or registered trademarks of Mentor Graphics Corporation or its affiliated companies in the United States and other countries.

The following are service marks of Mentor Graphics Corporation:

A World of Learning, ADAPT, AppNotes, Assess2000, BIST Compiler, BIST-In-Place, BIST-Ready, Concurrent Board Process, DirectConnect, ECO Immunity, EDGE (Engineering Design Guide for Excellence), Expert2000, FastTrack Consulting, IntraStep, ISD Creation, It's More than Just Tools, Knowledge Center, Knowledge-Sourcing, Mentor Graphics Support CD, Mentor Graphics SupportBulletin, Mentor Graphics SupportCenter, Mentor Graphics SupportFax, Mentor Graphics SupportNet-Email, Mentor Graphics SupportNet-FTP, Mentor Graphics SupportNet-Telnet, Mentor Graphics We Mean Business, MTPI, Online Knowledge Center, OpenDoor, Reinstatement 2000, SiteLine2000, SupportNet KnowlegeBase, Support Services BaseLine, Support Services ClassLine, Support Services Latitudes, Support Services OpenLine, Support Services PrivateLine, Support Services SiteLine, Support Services TechLine, Support Services RemoteLine, and VR-Process.

Mentor Graphics' trademarks may only be used with express written permission from Mentor Graphics. Fair use of Mentor Graphics' trademarks in advertising and promotion of Mentor Graphics products requires proper acknowledgement.

Mentor sometimes refers to products by using trademarks ("Marks") owned by a third-party and such use is not an attempt to indicate Mentor Graphics as a source of that product, but is intended to indicate a product from, or associated with, a respective third party. Use of third-party Marks is intended to inure to the benefit of the respective third-party.

Rev. 040628

End-User License Agreement

IMPORTANT - USE OF THIS SOFTWARE IS SUBJECT TO LICENSE RESTRICTIONS. CAREFULLY READ THIS LICENSE AGREEMENT BEFORE USING THE SOFTWARE.

This license is a legal "Agreement" concerning the use of Software between you, the end user, either individually or as an authorized representative of the company acquiring the license, and Mentor Graphics Corporation and Mentor Graphics (Ireland) Limited acting directly or through their subsidiaries or authorized distributors (collectively "Mentor Graphics"). USE OF SOFTWARE INDICATES YOUR COMPLETE AND UNCONDITIONAL ACCEPTANCE OF THE TERMS AND CONDITIONS SET FORTH IN THIS AGREEMENT. If you do not agree to these terms and conditions, promptly return, or, if received electronically, certify destruction of Software and all accompanying items within five days after receipt of Software and receive a full refund of any license fee paid.

END-USER LICENSE AGREEMENT

1. **GRANT OF LICENSE.** The software programs you are installing, downloading, or have acquired with this Agreement, including any updates, modifications, revisions, copies, documentation and design data ("Software") are copyrighted, trade secret and confidential information of Mentor Graphics or its licensors who maintain exclusive title to all Software and retain all rights not expressly granted by this Agreement. Mentor Graphics grants to you, subject to payment of appropriate license fees, a nontransferable, nonexclusive license to use Software solely: (a) in machine-readable, object-code form; (b) for your internal business purposes; and (c) on the computer hardware or at the site for which an applicable license fee is paid, or as authorized by Mentor Graphics. A site is restricted to a one-half mile (800 meter) radius. Mentor Graphics' standard policies and programs, which vary depending on Software, license fees paid or service plan purchased, apply to the following and are subject to change: (a) relocation of Software; (b) use of Software, which may be limited, for example, to execution of a single session by a single user on the authorized hardware or for a restricted period of time (such limitations may be communicated and technically implemented through the use of authorization codes or similar devices); (c) support services provided, including eligibility to receive telephone support, updates, modifications, and revisions. Current standard policies and programs are available upon request.

2. **ESD SOFTWARE.** If you purchased a license to use embedded software development ("ESD") Software, Mentor Graphics grants to you a nontransferable, nonexclusive license to reproduce and distribute executable files created using ESD compilers, including the ESD run-time libraries distributed with ESD C and C++ compiler Software that are linked into a composite program as an integral part of your compiled computer program, provided that you distribute these files only in conjunction with your compiled computer program. Mentor Graphics does NOT grant you any right to duplicate or incorporate copies of Mentor Graphics' real-time operating systems or other ESD Software, except those explicitly granted in this section, into your products without first signing a separate agreement with Mentor Graphics for such purpose.

3. **BETA CODE.** Portions or all of certain Software may contain code for experimental testing and evaluation ("Beta Code"), which may not be used without Mentor Graphics' explicit authorization. Upon Mentor Graphics' authorization, Mentor Graphics grants to you a temporary, nontransferable, nonexclusive license for experimental use to test and evaluate the Beta Code without charge for a limited period of time specified by Mentor Graphics. This grant and your use of the Beta Code shall not be construed as marketing or offering to sell a license to the Beta Code, which Mentor Graphics may choose not to release commercially in any form. If Mentor Graphics authorizes you to use the Beta Code, you agree to evaluate and test the Beta Code under normal conditions as directed by Mentor Graphics. You will contact Mentor Graphics periodically during your use of the Beta Code to discuss any malfunctions or suggested improvements. Upon completion of your evaluation and testing, you will send to Mentor Graphics a written evaluation of the Beta Code, including its strengths, weaknesses and recommended improvements. You agree that any written evaluations and all inventions, product improvements, modifications or developments that Mentor Graphics conceived or made during or subsequent to this Agreement, including those based partly or wholly on your feedback, will be the exclusive property of Mentor Graphics. Mentor Graphics will have exclusive rights, title and interest in all such property. The provisions of this subsection shall survive termination or expiration of this Agreement.

4. **RESTRICTIONS ON USE.** You may copy Software only as reasonably necessary to support the authorized use. Each copy must include all notices and legends embedded in Software and affixed to its medium and container as received from Mentor Graphics. All copies shall remain the property of Mentor Graphics or its licensors. You shall maintain a record of the number and primary location of all copies of Software, including copies merged with other software, and shall make those records available to Mentor Graphics upon request. You shall not make Software available in any form to any

person other than employees and contractors, excluding Mentor Graphics' competitors, whose job performance requires access. You shall take appropriate action to protect the confidentiality of Software and ensure that any person permitted access to Software does not disclose it or use it except as permitted by this Agreement. Except as otherwise permitted for purposes of interoperability as specified by applicable and mandatory local law, you shall not reverse-assemble, reverse-compile, reverse-engineer or in any way derive from Software any source code. You may not sublicense, assign or otherwise transfer Software, this Agreement or the rights under it, whether by operation of law or otherwise ("attempted transfer"), without Mentor Graphics' prior written consent and payment of Mentor Graphics' then-current applicable transfer charges. Any attempted transfer without Mentor Graphics' prior written consent shall be a material breach of this Agreement and may, at Mentor Graphics' option, result in the immediate termination of the Agreement and licenses granted under this Agreement.

The terms of this Agreement, including without limitation, the licensing and assignment provisions shall be binding upon your heirs, successors in interest and assigns. The provisions of this section 4 shall survive the termination or expiration of this Agreement.

5. LIMITED WARRANTY.

5.1. Mentor Graphics warrants that during the warranty period Software, when properly installed, will substantially conform to the functional specifications set forth in the applicable user manual. Mentor Graphics does not warrant that Software will meet your requirements or that operation of Software will be uninterrupted or error free. The warranty period is 90 days starting on the 15th day after delivery or upon installation, whichever first occurs. You must notify Mentor Graphics in writing of any nonconformity within the warranty period. This warranty shall not be valid if Software has been subject to misuse, unauthorized modification or installation. MENTOR GRAPHICS' ENTIRE LIABILITY AND YOUR EXCLUSIVE REMEDY SHALL BE, AT MENTOR GRAPHICS' OPTION, EITHER (A) REFUND OF THE PRICE PAID UPON RETURN OF SOFTWARE TO MENTOR GRAPHICS OR (B) MODIFICATION OR REPLACEMENT OF SOFTWARE THAT DOES NOT MEET THIS LIMITED WARRANTY, PROVIDED YOU HAVE OTHERWISE COMPLIED WITH THIS AGREEMENT. MENTOR GRAPHICS MAKES NO WARRANTIES WITH RESPECT TO: (A) SERVICES; (B) SOFTWARE WHICH IS LICENSED TO YOU FOR A LIMITED TERM OR LICENSED AT NO COST; OR (C) EXPERIMENTAL BETA CODE; ALL OF WHICH ARE PROVIDED "AS IS."

5.2. THE WARRANTIES SET FORTH IN THIS SECTION 5 ARE EXCLUSIVE. NEITHER MENTOR GRAPHICS NOR ITS LICENSORS MAKE ANY OTHER WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, WITH RESPECT TO SOFTWARE OR OTHER MATERIAL PROVIDED UNDER THIS AGREEMENT. MENTOR GRAPHICS AND ITS LICENSORS SPECIFICALLY DISCLAIM ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY.

6. **LIMITATION OF LIABILITY.** EXCEPT WHERE THIS EXCLUSION OR RESTRICTION OF LIABILITY WOULD BE VOID OR INEFFECTIVE UNDER APPLICABLE LAW, IN NO EVENT SHALL MENTOR GRAPHICS OR ITS LICENSORS BE LIABLE FOR INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES (INCLUDING LOST PROFITS OR SAVINGS) WHETHER BASED ON CONTRACT, TORT OR ANY OTHER LEGAL THEORY, EVEN IF MENTOR GRAPHICS OR ITS LICENSORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. IN NO EVENT SHALL MENTOR GRAPHICS' OR ITS LICENSORS' LIABILITY UNDER THIS AGREEMENT EXCEED THE AMOUNT PAID BY YOU FOR THE SOFTWARE OR SERVICE GIVING RISE TO THE CLAIM. IN THE CASE WHERE NO AMOUNT WAS PAID, MENTOR GRAPHICS AND ITS LICENSORS SHALL HAVE NO LIABILITY FOR ANY DAMAGES WHATSOEVER.

7. **LIFE ENDANGERING ACTIVITIES.** NEITHER MENTOR GRAPHICS NOR ITS LICENSORS SHALL BE LIABLE FOR ANY DAMAGES RESULTING FROM OR IN CONNECTION WITH THE USE OF SOFTWARE IN ANY APPLICATION WHERE THE FAILURE OR INACCURACY OF THE SOFTWARE MIGHT RESULT IN DEATH OR PERSONAL INJURY.

8. **INDEMNIFICATION.** YOU AGREE TO INDEMNIFY AND HOLD HARMLESS MENTOR GRAPHICS AND ITS LICENSORS FROM ANY CLAIMS, LOSS, COST, DAMAGE, EXPENSE, OR LIABILITY, INCLUDING ATTORNEYS' FEES, ARISING OUT OF OR IN CONNECTION WITH YOUR USE OF SOFTWARE AS DESCRIBED IN SECTION 7.

9. INFRINGEMENT.

9.1. Mentor Graphics will defend or settle, at its option and expense, any action brought against you alleging that Software infringes a patent or copyright or misappropriates a trade secret in the United States, Canada, Japan, or

member state of the European Patent Office. Mentor Graphics will pay any costs and damages finally awarded against you that are attributable to the infringement action. You understand and agree that as conditions to Mentor Graphics' obligations under this section you must: (a) notify Mentor Graphics promptly in writing of the action; (b) provide Mentor Graphics all reasonable information and assistance to defend or settle the action; and (c) grant Mentor Graphics sole authority and control of the defense or settlement of the action.

9.2. If an infringement claim is made, Mentor Graphics may, at its option and expense: (a) replace or modify Software so that it becomes noninfringing; (b) procure for you the right to continue using Software; or (c) require the return of Software and refund to you any license fee paid, less a reasonable allowance for use.

9.3. Mentor Graphics has no liability to you if infringement is based upon: (a) the combination of Software with any product not furnished by Mentor Graphics; (b) the modification of Software other than by Mentor Graphics; (c) the use of other than a current unaltered release of Software; (d) the use of Software as part of an infringing process; (e) a product that you make, use or sell; (f) any Beta Code contained in Software; (g) any Software provided by Mentor Graphics' licensors who do not provide such indemnification to Mentor Graphics' customers; or (h) infringement by you that is deemed willful. In the case of (h) you shall reimburse Mentor Graphics for its attorney fees and other costs related to the action upon a final judgment.

9.4. THIS SECTION 9 STATES THE ENTIRE LIABILITY OF MENTOR GRAPHICS AND ITS LICENSORS AND YOUR SOLE AND EXCLUSIVE REMEDY WITH RESPECT TO ANY ALLEGED PATENT OR COPYRIGHT INFRINGEMENT OR TRADE SECRET MISAPPROPRIATION BY ANY SOFTWARE LICENSED UNDER THIS AGREEMENT.

10. **TERM.** This Agreement remains effective until expiration or termination. This Agreement will immediately terminate upon notice if you exceed the scope of license granted or otherwise fail to comply with the provisions of Sections 1, 2, or 4. For any other material breach under this Agreement, Mentor Graphics may terminate this Agreement upon 30 days written notice if you are in material breach and fail to cure such breach within the 30-day notice period. If Software was provided for limited term use, this Agreement will automatically expire at the end of the authorized term. Upon any termination or expiration, you agree to cease all use of Software and return it to Mentor Graphics or certify deletion and destruction of Software, including all copies, to Mentor Graphics' reasonable satisfaction.

11. **EXPORT.** Software is subject to regulation by local laws and United States government agencies, which prohibit export or diversion of certain products, information about the products, and direct products of the products to certain countries and certain persons. You agree that you will not export any Software or direct product of Software in any manner without first obtaining all necessary approval from appropriate local and United States government agencies.

12. **RESTRICTED RIGHTS NOTICE.** Software was developed entirely at private expense and is commercial computer software provided with RESTRICTED RIGHTS. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement under which Software was obtained pursuant to DFARS 227.7202-3(a) or as set forth in subparagraphs (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable. Contractor/manufacturer is Mentor Graphics Corporation, 8005 SW Boeckman Road, Wilsonville, Oregon 97070-7777 USA.

13. **THIRD PARTY BENEFICIARY.** For any Software under this Agreement licensed by Mentor Graphics from Microsoft or other licensors, Microsoft or the applicable licensor is a third party beneficiary of this Agreement with the right to enforce the obligations set forth herein.

14. **AUDIT RIGHTS.** With reasonable prior notice, Mentor Graphics shall have the right to audit during your normal business hours all records and accounts as may contain information regarding your compliance with the terms of this Agreement. Mentor Graphics shall keep in confidence all information gained as a result of any audit. Mentor Graphics shall only use or disclose such information as necessary to enforce its rights under this Agreement.

15. **CONTROLLING LAW AND JURISDICTION.** THIS AGREEMENT SHALL BE GOVERNED BY AND CONSTRUED UNDER THE LAWS OF THE STATE OF OREGON, USA, IF YOU ARE LOCATED IN NORTH OR SOUTH AMERICA, AND THE LAWS OF IRELAND IF YOU ARE LOCATED OUTSIDE OF NORTH AND SOUTH AMERICA. All disputes arising out of or in relation to this Agreement shall be submitted to the exclusive jurisdiction of Dublin, Ireland when the laws of Ireland apply, or Wilsonville, Oregon when the laws of Oregon apply. This section shall not restrict Mentor Graphics' right to bring an action against you in the jurisdiction where your place of business is located. The United Nations Convention on Contracts for the International Sale of Goods does not apply to this Agreement.

16. **SEVERABILITY.** If any provision of this Agreement is held by a court of competent jurisdiction to be void, invalid, unenforceable or illegal, such provision shall be severed from this Agreement and the remaining provisions will remain in full force and effect.

17. **PAYMENT TERMS AND MISCELLANEOUS.** You will pay amounts invoiced, in the currency specified on the applicable invoice, within 30 days from the date of such invoice. This Agreement contains the parties' entire understanding relating to its subject matter and supersedes all prior or contemporaneous agreements, including but not limited to any purchase order terms and conditions, except valid license agreements related to the subject matter of this Agreement (which are physically signed by you and an authorized agent of Mentor Graphics) either referenced in the purchase order or otherwise governing this subject matter. This Agreement may only be modified in writing by authorized representatives of the parties. Waiver of terms or excuse of breach must be in writing and shall not constitute subsequent consent, waiver or excuse. The prevailing party in any legal action regarding the subject matter of this Agreement shall be entitled to recover, in addition to other relief, reasonable attorneys' fees and expenses.

Rev. 040401, Part Number 221417